
COSC2673 (Computational) Machine Learning

Assignment 3 - Image Classification By Alissa Nguyen (s4085541)

Introduction

This notebook develops a machine learning workflow for an image classification task using a modified version of the CRCHistoPhenotypes dataset. The dataset contains small 27×27 RGB histology image patches of colon cells.

The assignment includes two supervised classification tasks:

1. **Cancer Classification**
Predict whether a cell image is cancerous using the target variable `isCancerous`.
2. **Cell-Type Classification**
Predict the cell category using the target variable `cellTypeName`.

The dataset provides two label files:

- `data_labels_mainData.csv`, which contains labels for both `isCancerous` and `cellTypeName`.
- `data_labels_extraData.csv`, which contains labels for `isCancerous` only.

Therefore, the `isCancerous` task uses both the main and extra datasets, while the `cellTypeName` task uses only the main dataset.

A key requirement of this assessment is that the test set must contain unseen patients. For this reason, the data is split by `patientID`, rather than by randomly splitting individual images. This avoids patient leakage and gives a more realistic evaluation of model generalisation.

The final supervised models used in this notebook are:

- `LogisticRegression`, used as a linear supervised baseline.
- `RandomForestClassifier`, used as a non-linear tree-based ensemble model.
- A Convolutional Neural Network (`CNN`), used as the image-specific deep learning model.

This model selection creates a controlled comparison between a linear classifier, a non-linear ensemble method, and a neural network architecture designed for image data.

Running Instructions and Reproducibility Notes

Before running this notebook, ensure that:

1. A Python environment is installed, such as Anaconda or standard Python with Jupyter Notebook/JupyterLab
2. Place the `data` folder in the same directory as this notebook
3. The following Python libraries are installed: pandas, numpy, matplotlib, Pillow, scikit-learn, torch, torchvision. To run this notebook:
4. Start Jupyter Notebook or JupyterLab
5. Run the cells from top to bottom order
6. The notebook will load the image labels and image files, inspect and analyse the dataset, create patient-level train/validation/test splits, preprocess the image data, train and evaluate classical machine learning models and CNN models, compare their performance, apply advanced techniques, select the final models, and evaluate them
7. The final evaluation results are displayed inside the notebook using metric tables, classification reports, confusion matrices, and training/validation plots.

1. Import Libraries

This section imports the libraries used for data handling, image loading, visualisation, classical machine learning, unsupervised learning, and deep learning. The notebook mainly uses `pandas`, `numpy`, `matplotlib`, `PIL`, `scikit-learn`, and `PyTorch`. These libraries support the full workflow required for the assignment, including data inspection, preprocessing, supervised classification, unsupervised exploratory analysis, model evaluation, and neural network training.

```
import os
import random
import warnings
import copy
from pathlib import Path
from collections import Counter

import numpy as np
import pandas as pd

#visualisation
import matplotlib.pyplot as plt

#image loading
from PIL import Image

#Scikit-learn: splitting, preprocessing, models, evaluation
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.metrics import (
    accuracy_score,
    f1_score,
    precision_score,
    recall_score,
```

```

        classification_report,
        confusion_matrix,
        ConfusionMatrixDisplay
    )

#classical supervised learning models
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

from sklearn.utils.class_weight import compute_class_weight

#unsupervised learning
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

#PyTorch deep learning
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader

warnings.filterwarnings("ignore")

print("Libraries imported successfully.")
print("PyTorch version:", torch.__version__)

Libraries imported successfully.
PyTorch version: 2.12.0

```

2. File Paths and Reproducibility Settings

This section defines the file paths based on the local folder structure. A fixed random seed is used so that data splitting and model experiments are more reproducible. Reproducibility is important because the assessment evaluates not only model performance, but also the correctness and reproducibility of predictions.

```

#local folder path
DATA_DIR = Path("data")
IMAGE_DIR = DATA_DIR / "images"

#label file paths
MAIN_LABELS_PATH = DATA_DIR / "data_labels_mainData.csv"
EXTRA_LABELS_PATH = DATA_DIR / "data_labels_extraData.csv"

#reproducibility
RANDOM_STATE = 42

random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)

```

```

torch.manual_seed(RANDOM_STATE)

if torch.cuda.is_available():
    torch.cuda.manual_seed(RANDOM_STATE)
    torch.cuda.manual_seed_all(RANDOM_STATE)

#device setting for PyTorch
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Random seed:", RANDOM_STATE)
print("Using device:", device)

print("\nMain labels path:", MAIN_LABELS_PATH)
print("Extra labels path:", EXTRA_LABELS_PATH)
print("Image folder path:", IMAGE_DIR)

Random seed: 42
Using device: cpu

Main labels path: data/data_labels_mainData.csv
Extra labels path: data/data_labels_extraData.csv
Image folder path: data/images

# Check that the expected folders and files exist
print("DATA_DIR exists:", DATA_DIR.exists())
print("IMAGE_DIR exists:", IMAGE_DIR.exists())
print("Main labels file exists:", MAIN_LABELS_PATH.exists())
print("Extra labels file exists:", EXTRA_LABELS_PATH.exists())

DATA_DIR exists: True
IMAGE_DIR exists: True
Main labels file exists: True
Extra labels file exists: True

QUICK_RUN = False

if QUICK_RUN:
    print("QUICK_RUN is ON: smaller samples or fewer epochs may be
    used for development.")
else:
    print("QUICK_RUN is OFF: full settings will be used for final
    results.")

QUICK_RUN is OFF: full settings will be used for final results.

```

3. Locate Images

The image files are stored in the `data/images` folder. This section locates all `.png` files and creates a dictionary that maps each image filename to its full file path. This step is necessary because the label files store image filenames, while the models require the actual image files to be loaded from disk.

```

all_image_paths = sorted(list(IMAGE_DIR.glob("*.png")))

print("Number of PNG images found:", len(all_image_paths))

# Display the first few paths
all_image_paths[:10]

Number of PNG images found: 20280

[PosixPath('data/images/1.png'),
 PosixPath('data/images/10.png'),
 PosixPath('data/images/100.png'),
 PosixPath('data/images/1000.png'),
 PosixPath('data/images/10000.png'),
 PosixPath('data/images/10001.png'),
 PosixPath('data/images/10002.png'),
 PosixPath('data/images/10003.png'),
 PosixPath('data/images/10004.png'),
 PosixPath('data/images/10005.png')]

image_path_dict = {path.name: path for path in all_image_paths}

print("Number of unique image filenames:", len(image_path_dict))

if len(image_path_dict) == len(all_image_paths):
    print("No duplicate image filenames detected.")
else:
    print("Warning: duplicate image filenames may exist.")

Number of unique image filenames: 20280
No duplicate image filenames detected.

#display one example image to confirm that image loading works
if len(all_image_paths) > 0:
    example_path = all_image_paths[0]
    example_img = Image.open(example_path)

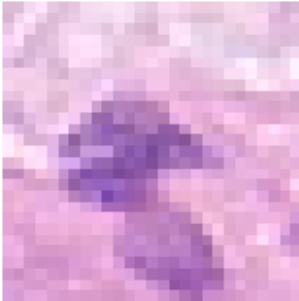
    print("Example image path:", example_path)
    print("Image size:", example_img.size)
    print("Image mode:", example_img.mode)

    plt.figure(figsize=(2, 2))
    plt.imshow(example_img)
    plt.axis("off")
    plt.title("Example image")
    plt.show()
else:
    print("No images found. Please check IMAGE_DIR.")

```

Example image path: data/images/1.png
Image size: (27, 27)
Image mode: RGB

Example image



4. Load Labels

This section loads the two label files used in the assignment. The main data contains labels for both classification tasks. The extra data only contains labels for the cancer classification task. For this reason, the two tasks are later prepared using different combinations of the label files:

- `isCancerous`: uses both main and extra data.
- `cellTypeName`: uses only main data.

A `source` column is added to preserve whether each row came from the main or extra label file. This makes the workflow easier to inspect and verify.

```
main_df = pd.read_csv(MAIN_LABELS_PATH)
extra_df = pd.read_csv(EXTRA_LABELS_PATH)

print("Main data shape:", main_df.shape)
print("Extra data shape:", extra_df.shape)
```

```
display(main_df.head())
display(extra_df.head())
```

```
Main data shape: (9896, 6)
Extra data shape: (10384, 4)
```

	InstanceID	patientID	ImageName	cellTypeName	cellType
isCancerous					
0	22405	1	22405.png	fibroblast	0
0					
1	22406	1	22406.png	fibroblast	0
0					
2	22407	1	22407.png	fibroblast	0
0					
3	22408	1	22408.png	fibroblast	0
0					

4	22409	1	22409.png	fibroblast	0
---	-------	---	-----------	------------	---

	InstanceID	patientID	ImageName	isCancerous
0	12681	61	12681.png	0
1	12682	61	12682.png	0
2	12683	61	12683.png	0
3	12684	61	12684.png	0
4	12685	61	12685.png	0

```
print("Main data columns:")
print(main_df.columns.tolist())
```

```
print("\nExtra data columns:")
print(extra_df.columns.tolist())
```

```
Main data columns:
['InstanceID', 'patientID', 'ImageName', 'cellTypeName', 'cellType', 'isCancerous']
```

```
Extra data columns:
['InstanceID', 'patientID', 'ImageName', 'isCancerous']
```

```
main_df = main_df.copy()
extra_df = extra_df.copy()
```

```
main_df["source"] = "main"
extra_df["source"] = "extra"
```

```
display(main_df.head())
display(extra_df.head())
```

	InstanceID	patientID	ImageName	cellTypeName	cellType
0	22405	1	22405.png	fibroblast	0
0	main				
1	22406	1	22406.png	fibroblast	0
0	main				
2	22407	1	22407.png	fibroblast	0
0	main				
3	22408	1	22408.png	fibroblast	0
0	main				
4	22409	1	22409.png	fibroblast	0
0	main				

	InstanceID	patientID	ImageName	isCancerous	source
0	12681	61	12681.png	0	extra
1	12682	61	12682.png	0	extra
2	12683	61	12683.png	0	extra
3	12684	61	12684.png	0	extra
4	12685	61	12685.png	0	extra

5. Data Inspection

Before modelling, the label files are inspected to understand the structure and quality of the dataset.

This section checks:

- dataframe shape and column types
- missing values
- duplicate rows
- target distributions
- number of patients
- number of images per patient
- whether all labelled images can be matched to files in the image folder

These checks are important because data issues at this stage could affect all later modelling results. They are also especially important for this assignment because the data must be split by `patientID` to avoid patient leakage.

```
print("Main dataframe information:")
main_df.info()
```

```
print("\nExtra dataframe information:")
extra_df.info()
```

```
Main dataframe information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9896 entries, 0 to 9895
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InstanceID      9896 non-null   int64
1   patientID       9896 non-null   int64
2   ImageName       9896 non-null   object
3   cellTypeName    9896 non-null   object
4   cellType        9896 non-null   int64
5   isCancerous     9896 non-null   int64
6   source          9896 non-null   object
dtypes: int64(4), object(3)
memory usage: 541.3+ KB
```

```
Extra dataframe information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10384 entries, 0 to 10383
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InstanceID      10384 non-null  int64
1   patientID       10384 non-null  int64
```

```
2   ImageName      10384 non-null  object
3   isCancerous    10384 non-null  int64
4   source         10384 non-null  object
```

```
dtypes: int64(3), object(2)
```

```
memory usage: 405.8+ KB
```

```
#check missing values in both label files
```

```
print("Missing values in main data:")
```

```
display(main_df.isna().sum())
```

```
print("\nMissing values in extra data:")
```

```
display(extra_df.isna().sum())
```

```
Missing values in main data:
```

```
InstanceID      0
patientID       0
ImageName       0
cellTypeName    0
cellType        0
isCancerous     0
source          0
dtype: int64
```

```
Missing values in extra data:
```

```
InstanceID      0
patientID       0
ImageName       0
isCancerous     0
source          0
dtype: int64
```

```
#check duplicate rows
```

```
print("Duplicate rows in main data:", main_df.duplicated().sum())
```

```
print("Duplicate rows in extra data:", extra_df.duplicated().sum())
```

```
Duplicate rows in main data: 0
```

```
Duplicate rows in extra data: 0
```

```
#check target distributions
```

```
print("Main data - isCancerous distribution:")
```

```
display(main_df["isCancerous"].value_counts(dropna=False))
```

```
print("\nExtra data - isCancerous distribution:")
```

```
display(extra_df["isCancerous"].value_counts(dropna=False))
```

```
print("\nMain data - cellTypeName distribution:")
```

```
display(main_df["cellTypeName"].value_counts(dropna=False))
```

```
Main data - isCancerous distribution:
```

```
isCancerous
0      5817
1      4079
Name: count, dtype: int64
```

Extra data - isCancerous distribution:

```
isCancerous
0      7394
1      2990
Name: count, dtype: int64
```

Main data - cellTypeName distribution:

```
cellTypeName
epithelial      4079
inflammatory    2543
fibroblast      1888
others          1386
Name: count, dtype: int64
```

```
#check patient counts and images per patient
print("Number of patients in main data:",
main_df["patientID"].nunique())
print("Images per patient in main data:")
display(main_df["patientID"].value_counts().describe())

print("\nNumber of patients in extra data:",
extra_df["patientID"].nunique())
print("Images per patient in extra data:")
display(extra_df["patientID"].value_counts().describe())
```

Number of patients in main data: 60
Images per patient in main data:

```
count      60.000000
mean       164.933333
std        92.104927
min        11.000000
25%        114.000000
50%        148.000000
75%        207.500000
max        389.000000
Name: count, dtype: float64
```

Number of patients in extra data: 38
Images per patient in extra data:

```

count      38.000000
mean       273.263158
std        203.522717
min         6.000000
25%        86.000000
50%       321.000000
75%       397.750000
max       699.000000
Name: count, dtype: float64

```

5.1 Build Task DataFrames

The assignment contains two prediction tasks, so two separate modelling dataframes are created. The first dataframe, `cancer_df`, is used for the `isCancerous` task. It combines the main and extra label files because both contain cancer labels. The second dataframe, `celltype_df`, is used for the `cellTypeName` task. It uses only the main label file because the extra label file does not contain cell-type labels. Creating separate dataframes keeps the workflow clear and prevents the cell-type model from accidentally using rows that do not have cell-type labels.

```

#combine main and extra data for the cancer classification task
cancer_df = pd.concat([main_df, extra_df], ignore_index=True)

```

```

#use only main data for the celltype classification task
celltype_df = main_df.copy()

```

```

print("Cancer task dataframe shape:", cancer_df.shape)
print("Celltype task dataframe shape:", celltype_df.shape)

```

```

display(cancer_df.head())
display(celltype_df.head())

```

```

Cancer task dataframe shape: (20280, 7)
Celltype task dataframe shape: (9896, 7)

```

	InstanceID	patientID	ImageName	cellTypeName	cellType
isCancerous	source				
0	22405	1	22405.png	fibroblast	0.0
0	main				
1	22406	1	22406.png	fibroblast	0.0
0	main				
2	22407	1	22407.png	fibroblast	0.0
0	main				
3	22408	1	22408.png	fibroblast	0.0
0	main				
4	22409	1	22409.png	fibroblast	0.0
0	main				

	InstanceID	patientID	ImageName	cellTypeName	cellType
isCancerous	source				

0		22405	1	22405.png	fibroblast	0
0	main					
1		22406	1	22406.png	fibroblast	0
0	main					
2		22407	1	22407.png	fibroblast	0
0	main					
3		22408	1	22408.png	fibroblast	0
0	main					
4		22409	1	22409.png	fibroblast	0
0	main					

5.3 Attach Image Paths

The label files contain image filenames, while the image folder contains the actual image files. This section standardises the filename column and attaches the full image path to each row. A row can only be used for modelling if its image file is found successfully.

```
#find the image filename column automatically
def find_filename_column(df):
    """
    Find the column that contains image filenames.
    """
    possible_cols = [
        "imageFile",
        "image_filename",
        "filename",
        "fileName",
        "ImageName",
        "imageName"
    ]

    for col in possible_cols:
        if col in df.columns:
            return col

    for col in df.columns:
        if df[col].astype(str).str.contains(".png",
regex=False).any():
            return col

    raise ValueError("No image filename column found. Please inspect
the dataframe columns.")

filename_col_cancer = find_filename_column(cancer_df)
filename_col_celltype = find_filename_column(celltype_df)

print("Filename column in cancer dataframe:", filename_col_cancer)
```

```
print("Filename column in cell-type dataframe:",
      filename_col_celltype)
```

```
Filename column in cancer dataframe: ImageName
Filename column in cell-type dataframe: ImageName
```

```
#standardise filename column name to image_filename
cancer_df = cancer_df.rename(columns={filename_col_cancer:
"image_filename"})
celltype_df = celltype_df.rename(columns={filename_col_celltype:
"image_filename"})
```

```
display(cancer_df.head())
display(celltype_df.head())
```

	InstanceID	patientID	image_filename	cellTypeName	cellType
isCancerous \					
0	22405	1	22405.png	fibroblast	0.0
0					
1	22406	1	22406.png	fibroblast	0.0
0					
2	22407	1	22407.png	fibroblast	0.0
0					
3	22408	1	22408.png	fibroblast	0.0
0					
4	22409	1	22409.png	fibroblast	0.0
0					

	source
0	main
1	main
2	main
3	main
4	main

	InstanceID	patientID	image_filename	cellTypeName	cellType
isCancerous \					
0	22405	1	22405.png	fibroblast	0
0					
1	22406	1	22406.png	fibroblast	0
0					
2	22407	1	22407.png	fibroblast	0
0					
3	22408	1	22408.png	fibroblast	0
0					
4	22409	1	22409.png	fibroblast	0
0					

	source
0	main

```

1  main
2  main
3  main
4  main

#attach image paths by matching each filename to the image dictionary created earlier
def attach_image_paths(df, image_dict):
    df = df.copy()
    df["image_path"] = df["image_filename"].map(image_dict)
    return df

cancer_df = attach_image_paths(cancer_df, image_path_dict)
celltype_df = attach_image_paths(celltype_df, image_path_dict)

print("Cancer task missing image paths:",
      cancer_df["image_path"].isna().sum())
print("Cell-type task missing image paths:",
      celltype_df["image_path"].isna().sum())

Cancer task missing image paths: 0
Cell-type task missing image paths: 0

```

5.4 Final Data Integrity Checks

Rows with missing image paths are removed if any are found. The final target distributions and patient counts are then checked again. This confirms that the final modelling dataframes are valid before moving into problem definition, splitting, and model development.

```

#remove rows with missing image paths, if any
#if the count is zero, this step will not change the data

before_cancer = len(cancer_df)
before_celltype = len(celltype_df)

cancer_df =
cancer_df.dropna(subset=["image_path"]).reset_index(drop=True)
celltype_df =
celltype_df.dropna(subset=["image_path"]).reset_index(drop=True)

print("Cancer task rows before image-path filtering:", before_cancer)
print("Cancer task rows after image-path filtering:", len(cancer_df))

print("\nCell-type task rows before image-path filtering:",
      before_celltype)
print("Cell-type task rows after image-path filtering:",
      len(celltype_df))

```

Cancer task rows before image-path filtering: 20280

Cancer task rows after image-path filtering: 20280

Cell-type task rows before image-path filtering: 9896

Cell-type task rows after image-path filtering: 9896

#final target distribution checks after image-path filtering

```
print("Final cancer task target distribution:")
```

```
display(cancer_df["isCancerous"].value_counts(dropna=False))
```

```
print("\nFinal cell-type task target distribution:")
```

```
display(celltype_df["cellTypeName"].value_counts(dropna=False))
```

Final cancer task target distribution:

isCancerous

0 13211

1 7069

Name: count, dtype: int64

Final cell-type task target distribution:

cellTypeName

epithelial 4079

inflammatory 2543

fibroblast 1888

others 1386

Name: count, dtype: int64

#final patient count checks

```
print("Cancer task number of patients:",
```

```
cancer_df["patientID"].nunique())
```

```
print("Cell-type task number of patients:",
```

```
celltype_df["patientID"].nunique())
```

Cancer task number of patients: 98

Cell-type task number of patients: 60

#final dataframe preview

```
print("Cancer task dataframe:")
```

```
display(cancer_df.head())
```

```
print("\nCell-type task dataframe:")
```

```
display(celltype_df.head())
```

Cancer task dataframe:

	InstanceID	patientID	image_filename	cellTypeName	cellType
isCancerous \					
0	22405	1	22405.png	fibroblast	0.0
0					

```

1      22406      1      22406.png      fibroblast      0.0
0
2      22407      1      22407.png      fibroblast      0.0
0
3      22408      1      22408.png      fibroblast      0.0
0
4      22409      1      22409.png      fibroblast      0.0
0

```

```

      source      image_path
0    main  data/images/22405.png
1    main  data/images/22406.png
2    main  data/images/22407.png
3    main  data/images/22408.png
4    main  data/images/22409.png

```

Cell-type task dataframe:

```

      InstanceID  patientID  image_filename  cellTypeName  cellType
isCancerous \
0      22405      1      22405.png      fibroblast      0
0
1      22406      1      22406.png      fibroblast      0
0
2      22407      1      22407.png      fibroblast      0
0
3      22408      1      22408.png      fibroblast      0
0
4      22409      1      22409.png      fibroblast      0
0

```

```

      source      image_path
0    main  data/images/22405.png
1    main  data/images/22406.png
2    main  data/images/22407.png
3    main  data/images/22408.png
4    main  data/images/22409.png

```

6. Problem Definition

This section defines the machine learning problem before model training. Clearly defining the task, input, target variable, learning experience, and evaluation measures helps keep the modelling process focused and justified. The two tasks use the same image input format but predict different targets. The first task is a binary classification problem, while the second task is a multi-class classification problem.

6.1 Task 1: Cancer Classification

The first task is to predict whether a colon cell image is cancerous.

- **Task:** binary image classification.
- **Input:** a `27 × 27 × 3` RGB histology image.
- **Target variable:** `isCancerous`.
- **Classes:** `0` for non-cancerous and `1` for cancerous.
- **Learning experience:** labelled examples from both `data_labels_mainData.csv` and `data_labels_extraData.csv`.
- **Evaluation focus:** model performance on unseen patients.

This task is clinically important because false negatives may indicate cancerous examples being missed. Therefore, recall and macro F1-score are important alongside accuracy.

6.2 Task 2: Cell-Type Classification

The second task is to classify each image into a cell-type category.

- **Task:** multi-class image classification.
- **Input:** a `27 × 27 × 3` RGB histology image.
- **Target variable:** `cellTypeName`.
- **Classes:** `epithelial`, `fibroblast`, `inflammatory`, and `others`.
- **Learning experience:** labelled examples from `data_labels_mainData.csv`.
- **Evaluation focus:** model performance across all cell-type classes on unseen patients.

This task is more detailed than the cancer classification task because the model must distinguish between four visually similar categories. The confusion matrix is especially useful here because it shows which cell types are most often confused.

6.3 Evaluation Measures

The models are evaluated using several classification metrics:

- `accuracy`
- `macro_f1`
- `weighted_f1`
- `macro_precision`
- `macro_recall`
- `classification report`
- `confusion matrix`

Accuracy is useful as a general measure, but it can be misleading when the dataset is imbalanced. For this reason, macro-averaged metrics are used as the main comparison measures. `macro_f1` is used as the primary validation metric because it gives equal weight to each class. This is important because both tasks contain class imbalance, and the model should not be judged only by its performance on the majority class. Confusion matrices are used to support deeper error analysis. They show where each model struggles, such as whether cancerous samples are missed or whether one cell type is frequently predicted as another.

6.4 Model Selection Rationale

This notebook trains the same three model families for both prediction tasks:

1. `LogisticRegression`
2. `RandomForestClassifier`
3. Convolutional Neural Network (CNN)

This design keeps the comparison controlled. Both tasks use the same input type, but they differ in the prediction target. Using the same model families for both tasks makes it easier to compare how model behaviour changes between binary cancer classification and multi-class cell-type classification.

- `LogisticRegression` is used as the linear supervised baseline. It tests whether the classes can be separated using a simple linear decision boundary after the images are flattened into pixel vectors. This gives a clear starting point for comparison.
- `RandomForestClassifier` is used as the non-linear tree-based ensemble model. It can model more complex relationships than a linear classifier by combining multiple decision trees. This is useful because image pixels may interact in non-linear ways.

The CNN is used as the image-specific deep learning model. Unlike the two classical models, the CNN keeps the spatial image structure and learns local visual patterns through convolutional layers. This is appropriate for image classification because neighbouring pixels contain meaningful spatial information.

Together, these three models provide a structured comparison between a linear model, a non-linear ensemble model, and a deep learning model designed for image data.

```
#DEFINE TASKS, TARGETS, AND MODEL PLAN
TASKS = {
    "cancer": {
        "name": "Cancer Classification",
        "dataframe": cancer_df,
        "target": "isCancerous",
        "description": "Predict whether the given colon cell image is
cancerous."
    },
    "celltype": {
        "name": "Cell-Type Classification",
        "dataframe": celltype_df,
        "target": "cellTypeName",
        "description": "Predict the cell type of the given colon cell
image."
    }
}

MODEL_PLAN = {
    "Classical Linear Model": "LogisticRegression",
```

```

    "Classical Non-linear Ensemble Model": "RandomForestClassifier",
    "Deep Learning Image Model": "Convolutional Neural Network"
}

for task_key, task_info in TASKS.items():
    df = task_info["dataframe"]
    target = task_info["target"]

    print("=" * 70)
    print(task_info["name"])
    print(task_info["description"])
    print("Target column:", target)
    print("Number of samples:", len(df))
    print("Number of patients:", df["patientID"].nunique())
    print("Number of target classes:", df[target].nunique())
    print("\nTarget distribution:")
    display(df[target].value_counts())

print("=" * 70)
print("Final supervised model plan:")
for model_type, model_name in MODEL_PLAN.items():
    print(f"{model_type}: {model_name}")

```

```

=====
Cancer Classification
Predict whether the given colon cell image is cancerous.
Target column: isCancerous
Number of samples: 20280
Number of patients: 98
Number of target classes: 2

```

Target distribution:

```

isCancerous
0      13211
1       7069
Name: count, dtype: int64

```

```

=====
Cell-Type Classification
Predict the cell type of the given colon cell image.
Target column: cellTypeName
Number of samples: 9896
Number of patients: 60
Number of target classes: 4

```

Target distribution:

```

cellTypeName
epithelial      4079
inflammatory    2543

```

```
fibroblast      1888
others          1386
Name: count, dtype: int64
```

```
=====
Final supervised model plan:
Classical Linear Model: LogisticRegression
Classical Non-linear Ensemble Model: RandomForestClassifier
Deep Learning Image Model: Convolutional Neural Network
```

7. Patient-Level Train/Validation/Test Split

The assessment requires the final test set to contain patients that are not seen during model training. Therefore, the data is split by `patientID` rather than by individual image rows. This is an important design decision. If images from the same patient appeared in both the training and test sets, the model could learn patient-specific patterns rather than general features of the cell images. This would create data leakage and may make the model appear more accurate than it would be on genuinely unseen patients.

The split used in this notebook is:

- training set: approximately 64% of patients
- validation set: approximately 16% of patients
- test set: approximately 20% of patients

The validation set is used for model comparison and tuning. The test set is kept separate until the final evaluation stage.

```
def patient_level_split(
    df,
    patient_col="patientID",
    test_size=0.20,
    val_size=0.20,
    random_state=42
):
    """
    Split a dataframe by patient ID.

    The function first separates test patients from the full patient
    list.
    It then splits the remaining patients into training and validation
    groups.

    This ensures that no patient appears in more than one split.
    """
    patients = df[patient_col].drop_duplicates()

    train_val_patients, test_patients = train_test_split(
        patients,
```

```

        test_size=test_size,
        random_state=random_state,
        shuffle=True
    )

    train_patients, val_patients = train_test_split(
        train_val_patients,
        test_size=val_size,
        random_state=random_state,
        shuffle=True
    )

    train_df =
df[df[patient_col].isin(train_patients)].reset_index(drop=True)
    val_df =
df[df[patient_col].isin(val_patients)].reset_index(drop=True)
    test_df =
df[df[patient_col].isin(test_patients)].reset_index(drop=True)

    return train_df, val_df, test_df

```

7.2 Leakage Checking Function

After splitting, a leakage check is performed to confirm that there is no patient overlap between the training, validation, and test sets. This check is included because patient leakage would make the evaluation unreliable.

```

def check_patient_leakage(train_df, val_df, test_df,
    patient_col="patientID"):
    """
    Check whether any patient appears in more than one split.
    """
    train_patients = set(train_df[patient_col].unique())
    val_patients = set(val_df[patient_col].unique())
    test_patients = set(test_df[patient_col].unique())

    train_val_overlap = train_patients.intersection(val_patients)
    train_test_overlap = train_patients.intersection(test_patients)
    val_test_overlap = val_patients.intersection(test_patients)

    print("Train/Validation patient overlap:", len(train_val_overlap))
    print("Train/Test patient overlap:", len(train_test_overlap))
    print("Validation/Test patient overlap:", len(val_test_overlap))

    if (
        len(train_val_overlap) == 0
        and len(train_test_overlap) == 0
        and len(val_test_overlap) == 0
    ):

```

```

        print("No patient leakage detected.")
    else:
        print("Warning: patient leakage detected.")

```

7.3 Cancer Classification Split

For the `isCancerous` task, the split is performed on `cancer_df`, which contains both the main and extra datasets. The split is based on `patientID`, so the cancer classification model is evaluated on patients that were not used during training.

```

cancer_train_df, cancer_val_df, cancer_test_df = patient_level_split(
    cancer_df,
    patient_col="patientID",
    test_size=0.20,
    val_size=0.20,
    random_state=RANDOM_STATE
)

print("Cancer classification split sizes:")
print("Training samples:", len(cancer_train_df))
print("Validation samples:", len(cancer_val_df))
print("Test samples:", len(cancer_test_df))

print("\nCancer classification patient counts:")
print("Training patients:", cancer_train_df["patientID"].nunique())
print("Validation patients:", cancer_val_df["patientID"].nunique())
print("Test patients:", cancer_test_df["patientID"].nunique())

Cancer classification split sizes:
Training samples: 12319
Validation samples: 4005
Test samples: 3956

Cancer classification patient counts:
Training patients: 62
Validation patients: 16
Test patients: 20

print("Cancer classification leakage check:")
check_patient_leakage(
    cancer_train_df,
    cancer_val_df,
    cancer_test_df,
    patient_col="patientID"
)

Cancer classification leakage check:
Train/Validation patient overlap: 0
Train/Test patient overlap: 0

```

Validation/Test patient overlap: 0
No patient leakage detected.

```
print("Cancer task target distribution by split:")

cancer_split_distribution = pd.DataFrame({
    "train":
cancer_train_df["isCancerous"].value_counts(normalize=True),
    "validation":
cancer_val_df["isCancerous"].value_counts(normalize=True),
    "test": cancer_test_df["isCancerous"].value_counts(normalize=True)
}).fillna(0)
```

```
display(cancer_split_distribution)
```

Cancer task target distribution by split:

	train	validation	test
isCancerous			
0	0.648916	0.57628	0.735339
1	0.351084	0.42372	0.264661

7.4 Celltype Classification Split

For the `celltypeName` task, the split is performed on `celltype_df`, which only contains the main dataset. The same patient-level splitting strategy is used so that the cell-type classification model is also evaluated on unseen patients.

```
celltype_train_df, celltype_val_df, celltype_test_df =
patient_level_split(
    celltype_df,
    patient_col="patientID",
    test_size=0.20,
    val_size=0.20,
    random_state=RANDOM_STATE
)

print("Cell-type classification split sizes:")
print("Training samples:", len(celltype_train_df))
print("Validation samples:", len(celltype_val_df))
print("Test samples:", len(celltype_test_df))

print("\nCell-type classification patient counts:")
print("Training patients:", celltype_train_df["patientID"].nunique())
print("Validation patients:", celltype_val_df["patientID"].nunique())
print("Test patients:", celltype_test_df["patientID"].nunique())
```

Cell-type classification split sizes:

Training samples: 6215

Validation samples: 1842

Test samples: 1839

Cell-type classification patient counts:

Training patients: 38

Validation patients: 10

Test patients: 12

```
print("Cell-type classification leakage check:")
check_patient_leakage(
    celltype_train_df,
    celltype_val_df,
    celltype_test_df,
    patient_col="patientID"
)
```

Cell-type classification leakage check:

Train/Validation patient overlap: 0

Train/Test patient overlap: 0

Validation/Test patient overlap: 0

No patient leakage detected.

```
print("Cell-type task target distribution by split:")
```

```
celltype_split_distribution = pd.DataFrame({
    "train":
celltype_train_df["cellTypeName"].value_counts(normalize=True),
    "validation":
celltype_val_df["cellTypeName"].value_counts(normalize=True),
    "test":
celltype_test_df["cellTypeName"].value_counts(normalize=True)
}).fillna(0)
```

```
display(celltype_split_distribution)
```

Cell-type task target distribution by split:

	train	validation	test
cellTypeName			
epithelial	0.366050	0.477199	0.502991
fibroblast	0.212550	0.166124	0.141925
inflammatory	0.301368	0.129207	0.234910
others	0.120032	0.227470	0.120174

7.5 Split Distribution Analysis

The following plots compare the class distributions across the training, validation, and test sets. Because the split is performed by patient rather than by individual image, the class proportions may not be perfectly balanced across the three splits. This is expected. The priority is to prevent patient leakage and produce a more realistic evaluation of generalisation.

```

def plot_split_distribution(train_df, val_df, test_df, target_col,
title):
    """
    Plot target class distribution across train, validation, and test
    splits.
    """
    distribution_df = pd.DataFrame({
        "Train": train_df[target_col].value_counts(normalize=True),
        "Validation": val_df[target_col].value_counts(normalize=True),
        "Test": test_df[target_col].value_counts(normalize=True)
    }).fillna(0)

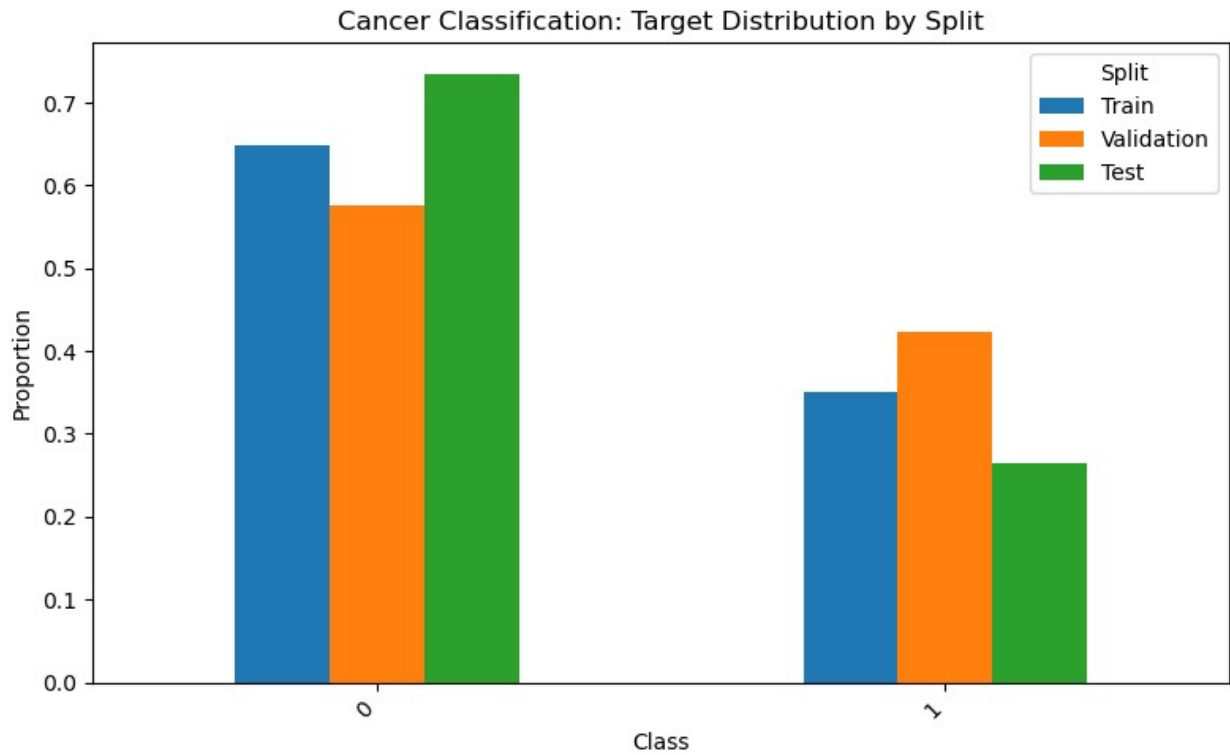
    ax = distribution_df.plot(kind="bar", figsize=(8, 5))
    ax.set_title(title)
    ax.set_xlabel("Class")
    ax.set_ylabel("Proportion")
    ax.legend(title="Split")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()

    return distribution_df

cancer_distribution_table = plot_split_distribution(
    cancer_train_df,
    cancer_val_df,
    cancer_test_df,
    target_col="isCancerous",
    title="Cancer Classification: Target Distribution by Split"
)

display(cancer_distribution_table)

```



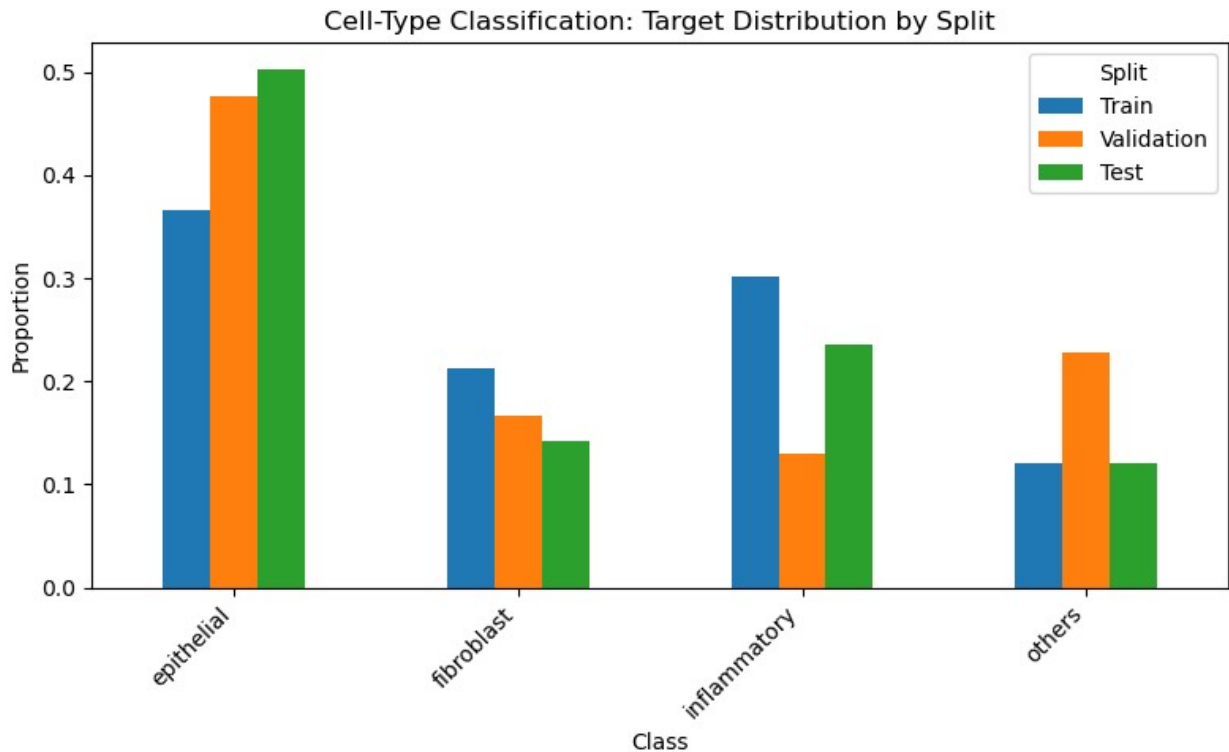
	Train	Validation	Test
isCancerous			
0	0.648916	0.57628	0.735339
1	0.351084	0.42372	0.264661

```

celltype_distribution_table = plot_split_distribution(
    celltype_train_df,
    celltype_val_df,
    celltype_test_df,
    target_col="cellTypeName",
    title="Cell-Type Classification: Target Distribution by Split"
)

display(celltype_distribution_table)

```



	Train	Validation	Test
cellTypeName			
epithelial	0.366050	0.477199	0.502991
fibroblast	0.212550	0.166124	0.141925
inflammatory	0.301368	0.129207	0.234910
others	0.120032	0.227470	0.120174

7.6 Split Summary

The final split summary is stored in a table so that the sample and patient counts can be referenced later in the report. This also confirms that both tasks have separate training, validation, and test sets.

```
split_summary = pd.DataFrame({
    "Task": [
        "Cancer Classification",
        "Cancer Classification",
        "Cancer Classification",
        "Cell-Type Classification",
        "Cell-Type Classification",
        "Cell-Type Classification"
    ],
    "Split": [
        "Train",
        "Validation",
```

```

        "Test",
        "Train",
        "Validation",
        "Test"
    ],
    "Samples": [
        len(cancer_train_df),
        len(cancer_val_df),
        len(cancer_test_df),
        len(celltype_train_df),
        len(celltype_val_df),
        len(celltype_test_df)
    ],
    "Patients": [
        cancer_train_df["patientID"].nunique(),
        cancer_val_df["patientID"].nunique(),
        cancer_test_df["patientID"].nunique(),
        celltype_train_df["patientID"].nunique(),
        celltype_val_df["patientID"].nunique(),
        celltype_test_df["patientID"].nunique()
    ]
})

display(split_summary)

```

	Task	Split	Samples	Patients
0	Cancer Classification	Train	12319	62
1	Cancer Classification	Validation	4005	16
2	Cancer Classification	Test	3956	20
3	Cell-Type Classification	Train	6215	38
4	Cell-Type Classification	Validation	1842	10
5	Cell-Type Classification	Test	1839	12

8. Exploratory Data Analysis

This section explores the dataset before model training. The purpose of EDA is to identify patterns and possible modelling challenges that should influence later decisions.

The analysis focuses on:

- class distributions
- patient-level image distributions
- sample images from each class
- RGB channel and intensity patterns
- average images by class

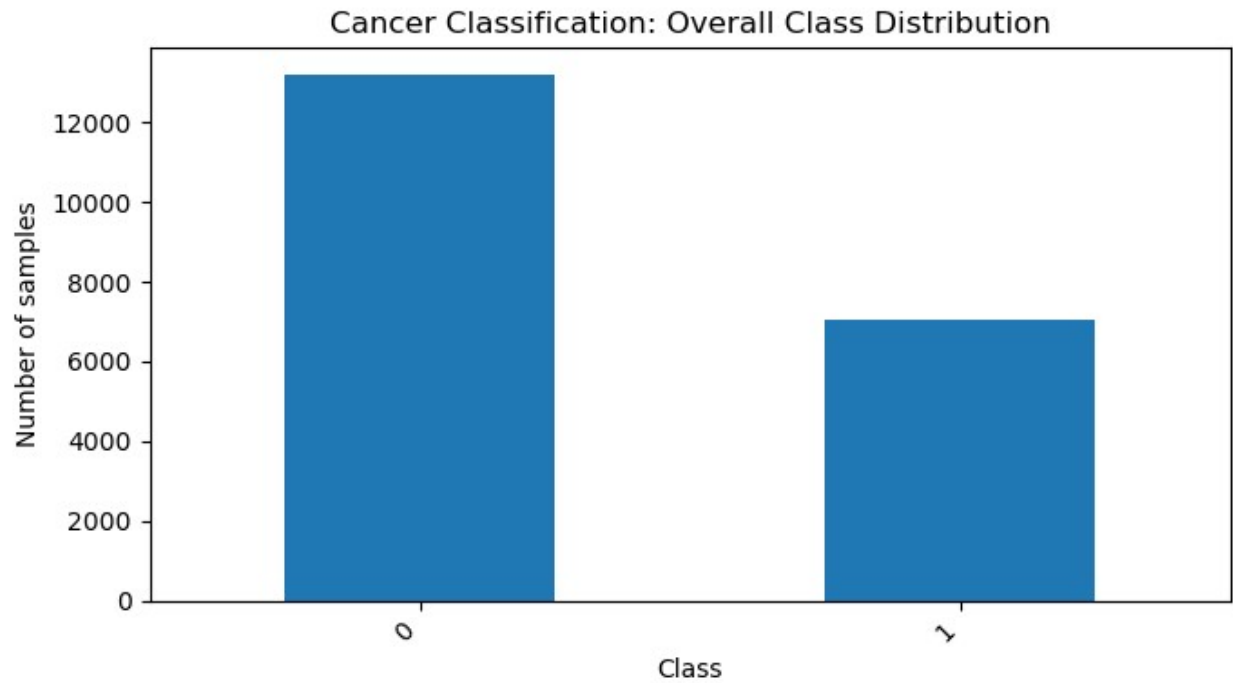
The EDA is used to support modelling decisions rather than only describing the dataset. In particular, it helps identify class imbalance, patient imbalance, and visual similarity between classes.

8.1 Class Distribution

The class distributions are examined for both prediction tasks. This is important because class imbalance can affect model training and evaluation. A model may achieve reasonable accuracy by favouring the majority class, while still performing poorly on minority classes. For this reason, macro-averaged metrics are used later during model comparison.

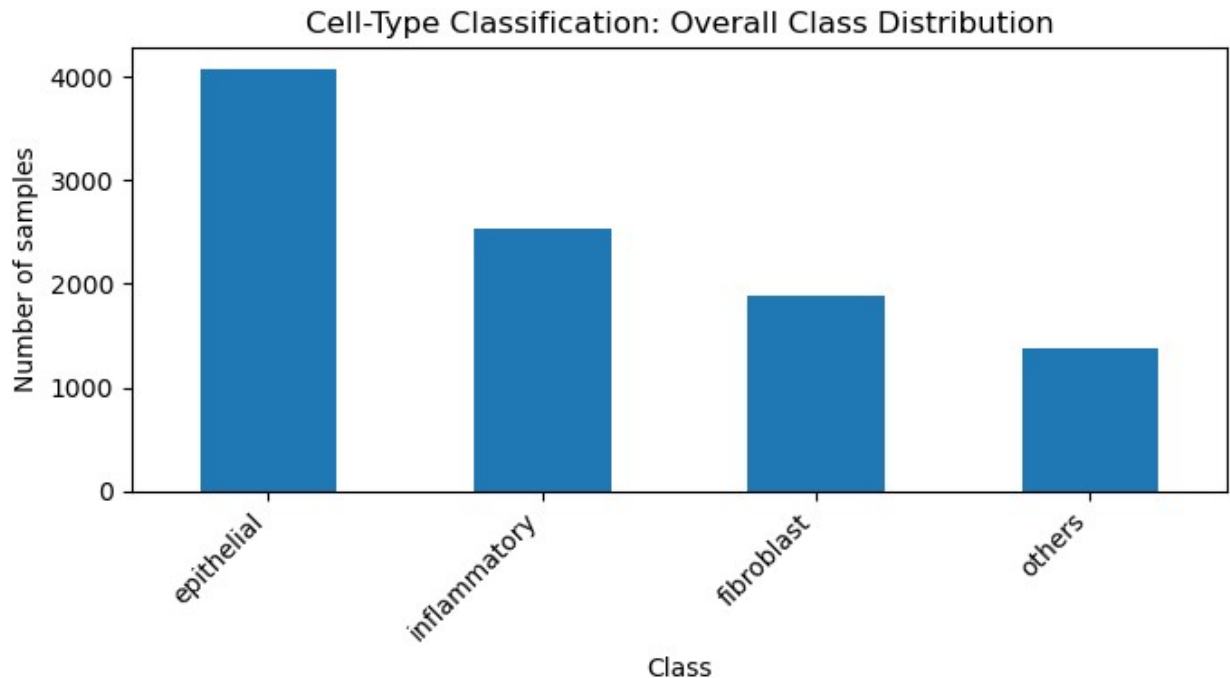
```
def plot_class_distribution(df, target_col, title):  
    """  
    Plot count and percentage distribution of a target column.  
    """  
    counts = df[target_col].value_counts()  
    percentages = df[target_col].value_counts(normalize=True) * 100  
  
    distribution_df = pd.DataFrame({  
        "count": counts,  
        "percentage": percentages.round(2)  
    })  
  
    display(distribution_df)  
  
    ax = counts.plot(kind="bar", figsize=(7, 4))  
    ax.set_title(title)  
    ax.set_xlabel("Class")  
    ax.set_ylabel("Number of samples")  
    plt.xticks(rotation=45, ha="right")  
    plt.tight_layout()  
    plt.show()  
  
    return distribution_df  
  
cancer_class_distribution = plot_class_distribution(  
    cancer_df,  
    target_col="isCancerous",  
    title="Cancer Classification: Overall Class Distribution"  
)
```

	count	percentage
isCancerous		
0	13211	65.14
1	7069	34.86



```
celltype_class_distribution = plot_class_distribution(  
    celltype_df,  
    target_col="cellTypeName",  
    title="Cell-Type Classification: Overall Class Distribution"  
)
```

cellTypeName	count	percentage
epithelial	4079	41.22
inflammatory	2543	25.70
fibroblast	1888	19.08
others	1386	14.01



Interpretation: The cancer classification task contains two classes, but the non-cancerous class is more frequent than the cancerous class. This means accuracy alone may not fully describe model quality, especially if the model performs unevenly across the two classes. The cell-type classification task contains four classes. The distribution is also imbalanced, with some cell types appearing more frequently than others. This supports the use of `macro_f1`, `macro_precision`, and `macro_recall` later in the model comparison.

8.2 Patient-Level Image Distribution

The number of images per patient is inspected because the assignment evaluates generalisation to unseen patients. If some patients contribute many more images than others, the dataset is patient-imbalanced. This can affect model evaluation because performance may depend on which patients are assigned to the training, validation, and test sets.

```
def plot_images_per_patient(df, title):  
    """  
    Plot the distribution of image counts per patient.  
    """  
    patient_counts = df["patientID"].value_counts()  
  
    print(title)  
    display(patient_counts.describe())  
  
    ax = patient_counts.plot(kind="hist", bins=20, figsize=(7, 4))  
    ax.set_title(title)  
    ax.set_xlabel("Number of images per patient")  
    ax.set_ylabel("Number of patients")  
    plt.tight_layout()
```

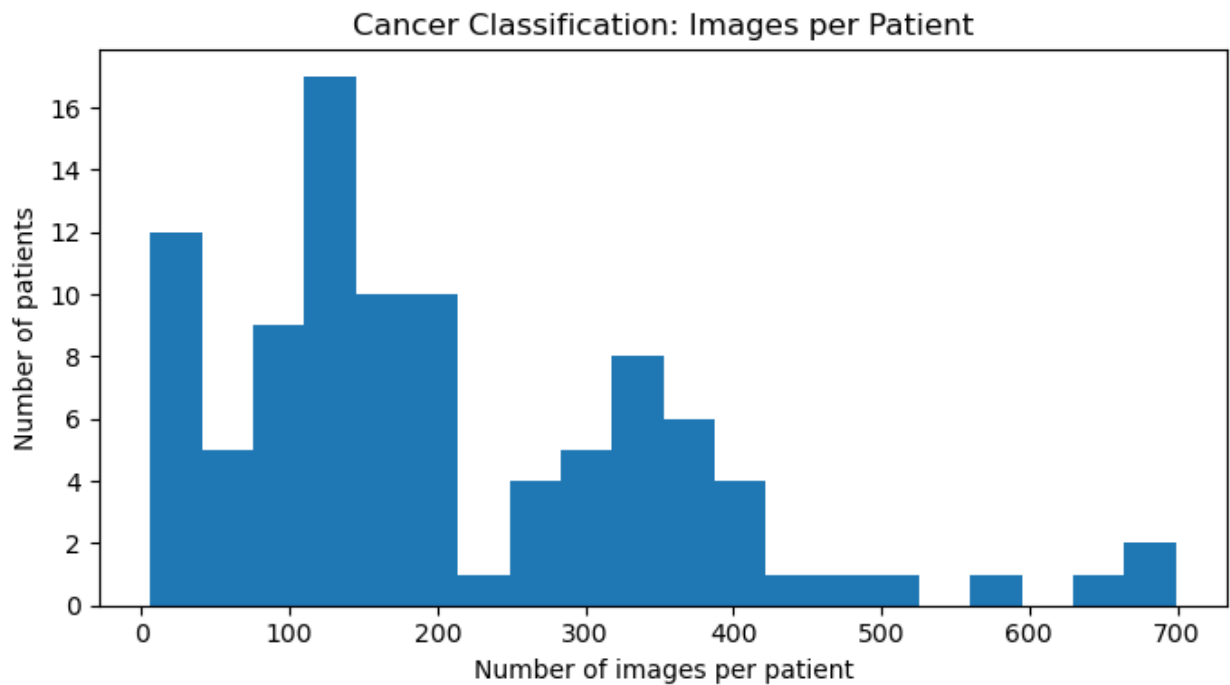
```
plt.show()

return patient_counts

cancer_patient_counts = plot_images_per_patient(
    cancer_df,
    title="Cancer Classification: Images per Patient"
)
```

Cancer Classification: Images per Patient

```
count      98.000000
mean       206.938776
std        154.190190
min         6.000000
25%        100.500000
50%        162.000000
75%        319.000000
max        699.000000
Name: count, dtype: float64
```



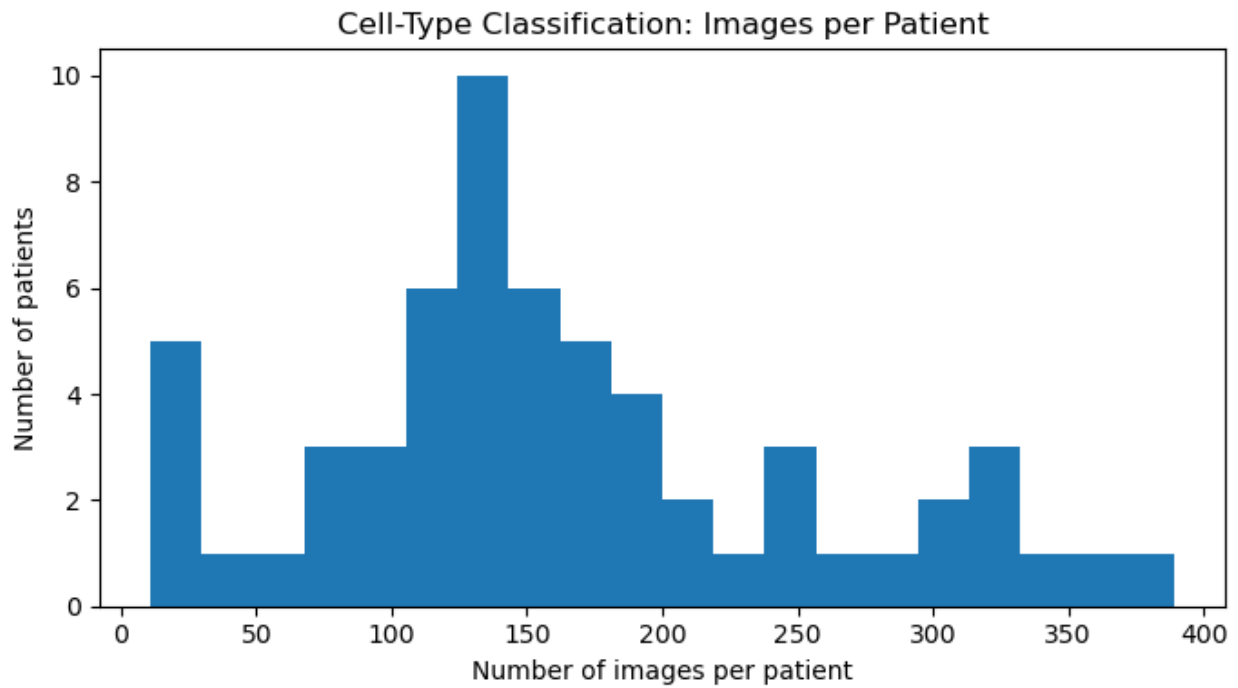
```
celltype_patient_counts = plot_images_per_patient(
    celltype_df,
    title="Cell-Type Classification: Images per Patient"
)
```

Cell-Type Classification: Images per Patient

```

count      60.000000
mean       164.933333
std        92.104927
min        11.000000
25%        114.000000
50%        148.000000
75%        207.500000
max        389.000000
Name: count, dtype: float64

```



```

print("Cancer task: top 10 patients by number of images")
display(cancer_patient_counts.head(10))

print("\nCell-type task: top 10 patients by number of images")
display(celltype_patient_counts.head(10))

```

Cancer task: top 10 patients by number of images

```

patientID
79      699
77      696
91      642
92      571
80      507
89      480
68      422
90      416
78      409

```

```
85      405
Name: count, dtype: int64
```

Cell-type task: top 10 patients by number of images

```
patientID
54      389
29      355
9       348
8       332
20      325
18      320
17      310
10      302
51      286
55      263
Name: count, dtype: int64
```

Interpretation: The number of images per patient varies substantially. This supports the decision to split by `patientID` rather than by individual image rows.

An image-level split could place images from the same patient into multiple splits, creating data leakage. The patient-level split is stricter and more realistic, although it may also cause class proportions to vary between the training, validation, and test sets.

8.3 Sample Images by Class

Sample images are displayed for each class to inspect the visual characteristics of the dataset. This helps show whether the classes appear visually distinct or whether there is overlap between them. Since the images are only 27×27 pixels, many important differences may be subtle and difficult to identify visually.

```
def show_sample_images_by_class(df, target_col, n_per_class=5,
                                random_state=42):
    """
    Display sample images for each class in the target column.
    """
    classes = sorted(df[target_col].dropna().unique())

    for class_label in classes:
        class_df = df[df[target_col] == class_label]

        if len(class_df) == 0:
            continue

        sample_df = class_df.sample(
            n=min(n_per_class, len(class_df)),
            random_state=random_state
        )
```

```

plt.figure(figsize=(n_per_class * 2, 2))
plt.suptitle(f"{target_col}: {class_label}", y=1.05)

for i, (_, row) in enumerate(sample_df.iterrows()):
    img = Image.open(row["image_path"]).convert("RGB")

    plt.subplot(1, n_per_class, i + 1)
    plt.imshow(img)
    plt.axis("off")
    plt.title(str(row["image_filename"]))

plt.tight_layout()
plt.show()

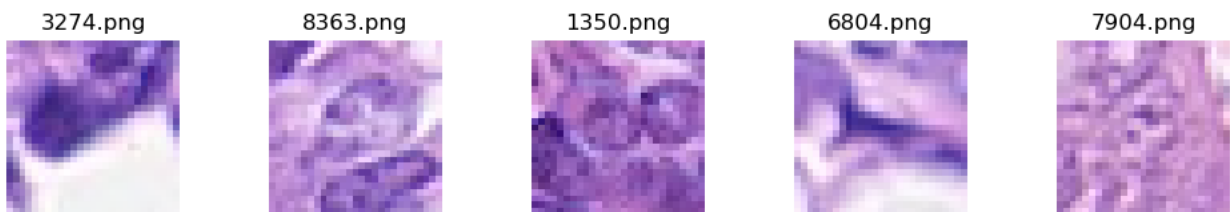
show_sample_images_by_class(
    cancer_df,
    target_col="isCancerous",
    n_per_class=5,
    random_state=RANDOM_STATE
)

```

isCancerous: 0



isCancerous: 1

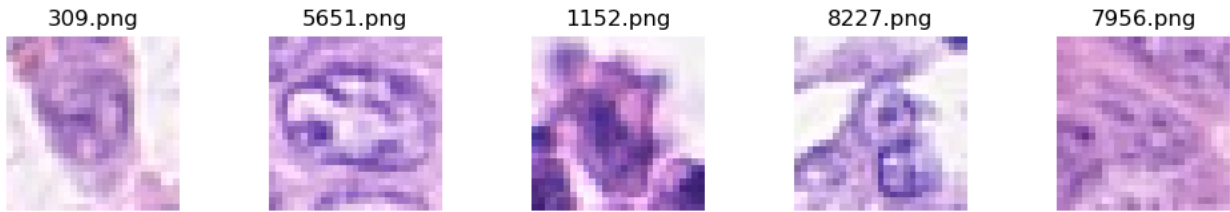


```

show_sample_images_by_class(
    celltype_df,
    target_col="cellTypeName",
    n_per_class=5,
    random_state=RANDOM_STATE
)

```

cellTypeName: epithelial



cellTypeName: fibroblast



cellTypeName: inflammatory



cellTypeName: others



Interpretation: The sample images show that the classification task is visually challenging. Many images share similar pink and purple histology staining, and the image resolution is small. This suggests that the models may need to learn subtle differences in colour, texture, and local cell structure. It also supports the use of a CNN later, because CNNs are designed to learn local spatial image features.

8.4 RGB Channel and Intensity Features

Simple image summary features are calculated to describe the colour and brightness patterns in the dataset. For each image, the mean red, green, and blue channel values are calculated, along with the overall mean pixel intensity. These features are used for EDA only. The final models use either flattened pixel values or image tensors.

```

def compute_image_summary_features(df):
    """
    Compute simple RGB and intensity summary features for each image.
    """
    rows = []

    for _, row in df.iterrows():
        img = Image.open(row["image_path"]).convert("RGB")
        arr = np.array(img).astype(np.float32) / 255.0

        rows.append({
            "image_filename": row["image_filename"],
            "patientID": row["patientID"],
            "red_mean": arr[:, :, 0].mean(),
            "green_mean": arr[:, :, 1].mean(),
            "blue_mean": arr[:, :, 2].mean(),
            "overall_intensity": arr.mean()
        })

    return pd.DataFrame(rows)

#to keep the notebook fast during development, use a sample when
QUICK_RUN is on
if QUICK_RUN:
    cancer_eda_sample = cancer_df.sample(n=min(3000, len(cancer_df)),
    random_state=RANDOM_STATE)
    celltype_eda_sample = celltype_df.sample(n=min(3000,
    len(celltype_df)), random_state=RANDOM_STATE)
else:
    cancer_eda_sample = cancer_df.copy()
    celltype_eda_sample = celltype_df.copy()

print("Cancer EDA sample size:", len(cancer_eda_sample))
print("Cell-type EDA sample size:", len(celltype_eda_sample))

Cancer EDA sample size: 20280
Cell-type EDA sample size: 9896

cancer_rgb_features =
compute_image_summary_features(cancer_eda_sample)
celltype_rgb_features =
compute_image_summary_features(celltype_eda_sample)

cancer_rgb_features = cancer_rgb_features.merge(
    cancer_eda_sample[["image_filename", "isCancerous"]],
    on="image_filename",
    how="left"
)

celltype_rgb_features = celltype_rgb_features.merge(
    celltype_eda_sample[["image_filename", "cellTypeName"]],

```

```

    on="image_filename",
    how="left"
)

display(cancer_rgb_features.head())
display(celltype_rgb_features.head())

```

	image_filename	patientID	red_mean	green_mean	blue_mean	\
0	22405.png	1	0.838925	0.669502	0.844735	
1	22406.png	1	0.826741	0.660018	0.834762	
2	22407.png	1	0.792706	0.633352	0.814497	
3	22408.png	1	0.889443	0.815401	0.900707	
4	22409.png	1	0.871164	0.761860	0.874429	

	overall_intensity	isCancerous
0	0.784387	0
1	0.773840	0
2	0.746852	0
3	0.868517	0
4	0.835818	0

	image_filename	patientID	red_mean	green_mean	blue_mean	\
0	22405.png	1	0.838925	0.669502	0.844735	
1	22406.png	1	0.826741	0.660018	0.834762	
2	22407.png	1	0.792706	0.633352	0.814497	
3	22408.png	1	0.889443	0.815401	0.900707	
4	22409.png	1	0.871164	0.761860	0.874429	

	overall_intensity	cellTypeName
0	0.784387	fibroblast
1	0.773840	fibroblast
2	0.746852	fibroblast
3	0.868517	fibroblast
4	0.835818	fibroblast

```

def plot_feature_by_class(feature_df, target_col, feature_col, title):
    """
    Plot a numeric image summary feature by class
    """
    classes = sorted(feature_df[target_col].dropna().unique())

    data = [
        feature_df[feature_df[target_col] == class_label][feature_col]
        for class_label in classes
    ]

    plt.figure(figsize=(7, 4))
    plt.boxplot(data, labels=classes)
    plt.title(title)
    plt.xlabel("Class")

```

```

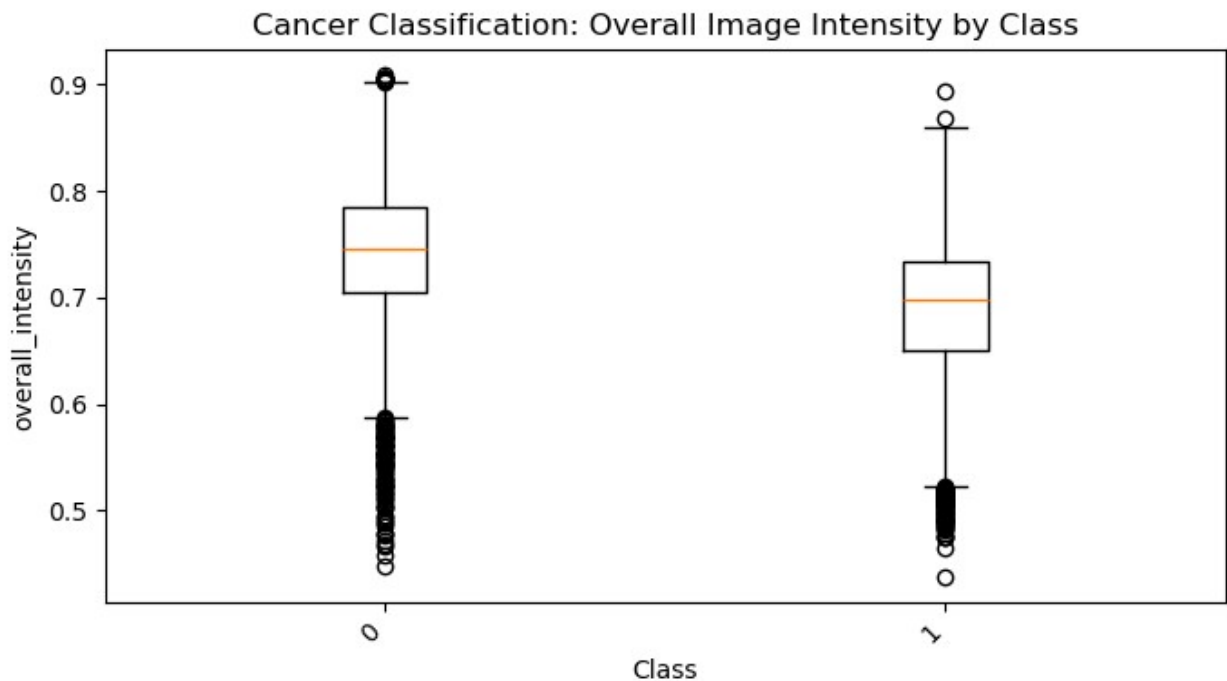
plt.ylabel(feature_col)
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

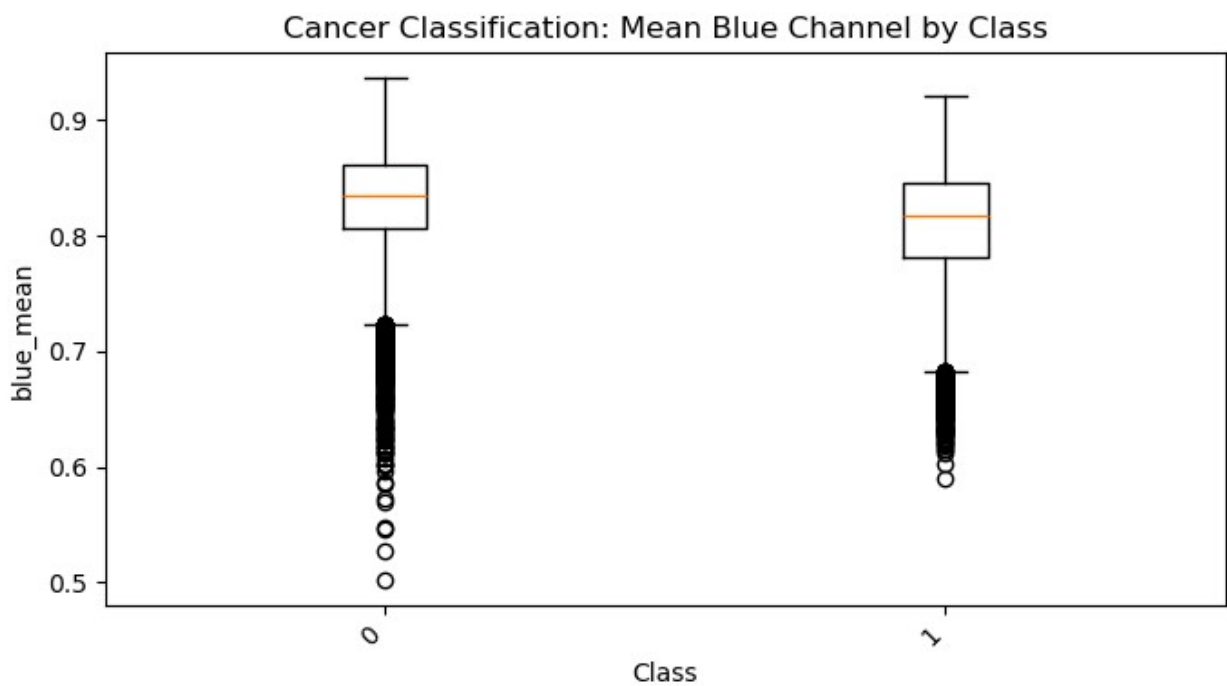
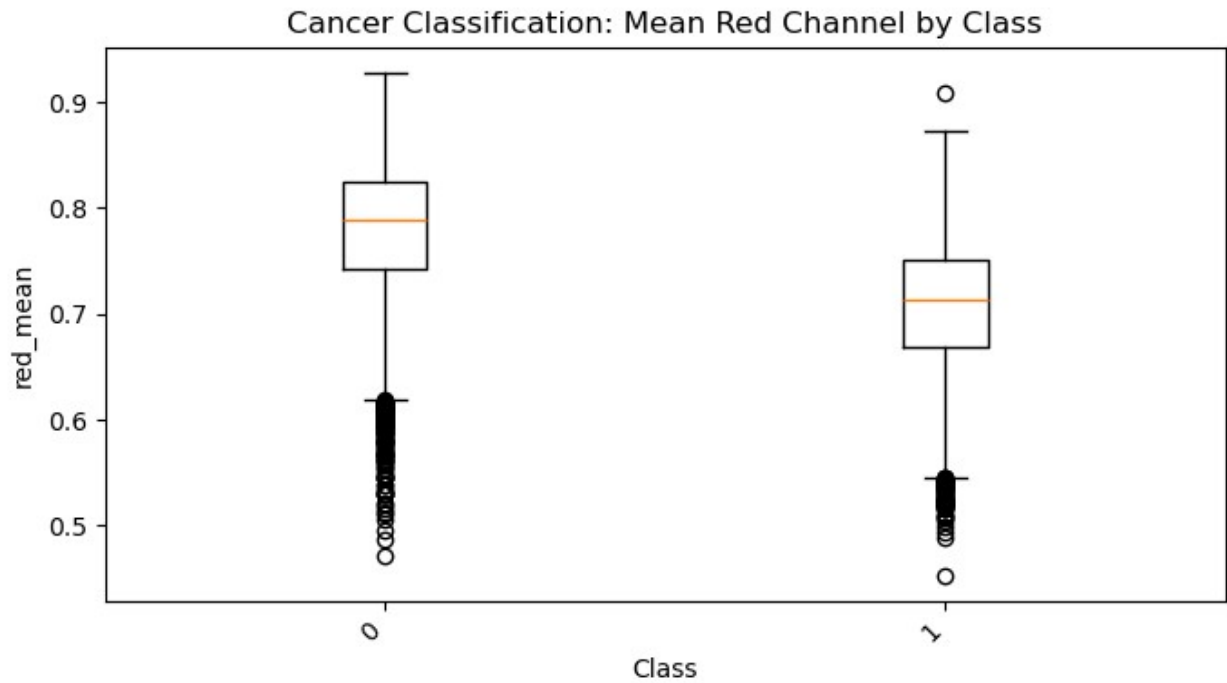
plot_feature_by_class(
    cancer_rgb_features,
    target_col="isCancerous",
    feature_col="overall_intensity",
    title="Cancer Classification: Overall Image Intensity by Class"
)

plot_feature_by_class(
    cancer_rgb_features,
    target_col="isCancerous",
    feature_col="red_mean",
    title="Cancer Classification: Mean Red Channel by Class"
)

plot_feature_by_class(
    cancer_rgb_features,
    target_col="isCancerous",
    feature_col="blue_mean",
    title="Cancer Classification: Mean Blue Channel by Class"
)

```





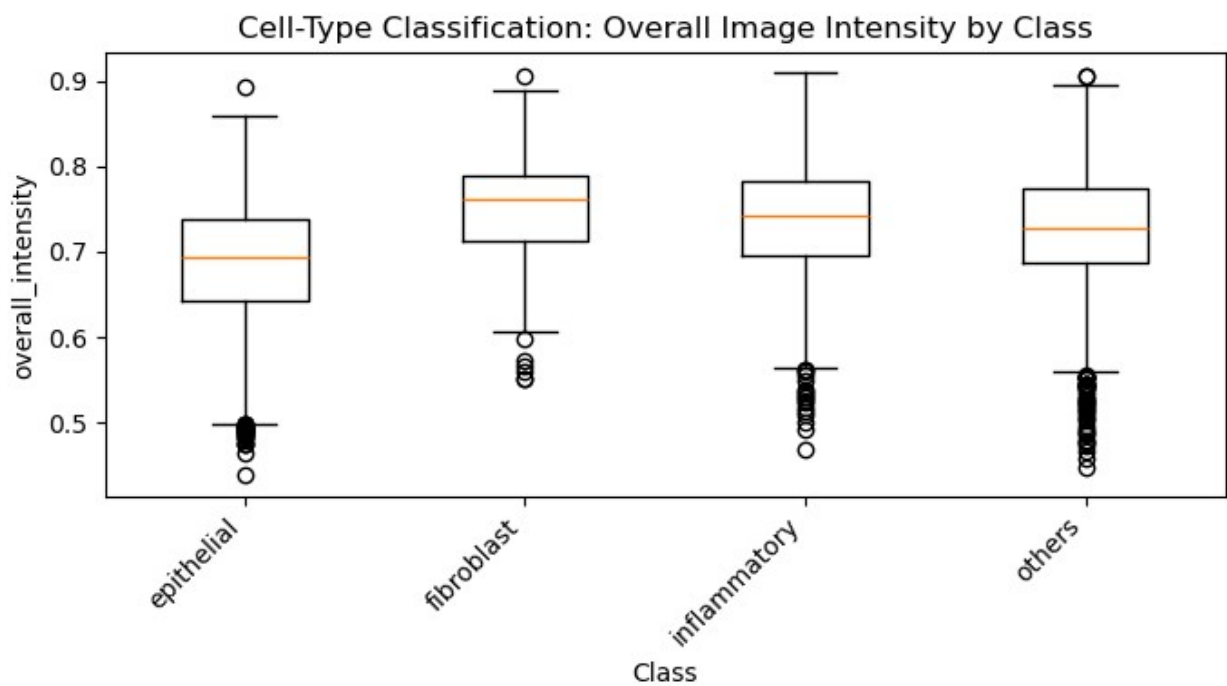
```
plot_feature_by_class(  
    celltype_rgb_features,  
    target_col="cellTypeName",  
    feature_col="overall_intensity",  
    title="Cell-Type Classification: Overall Image Intensity by Class"  
)
```

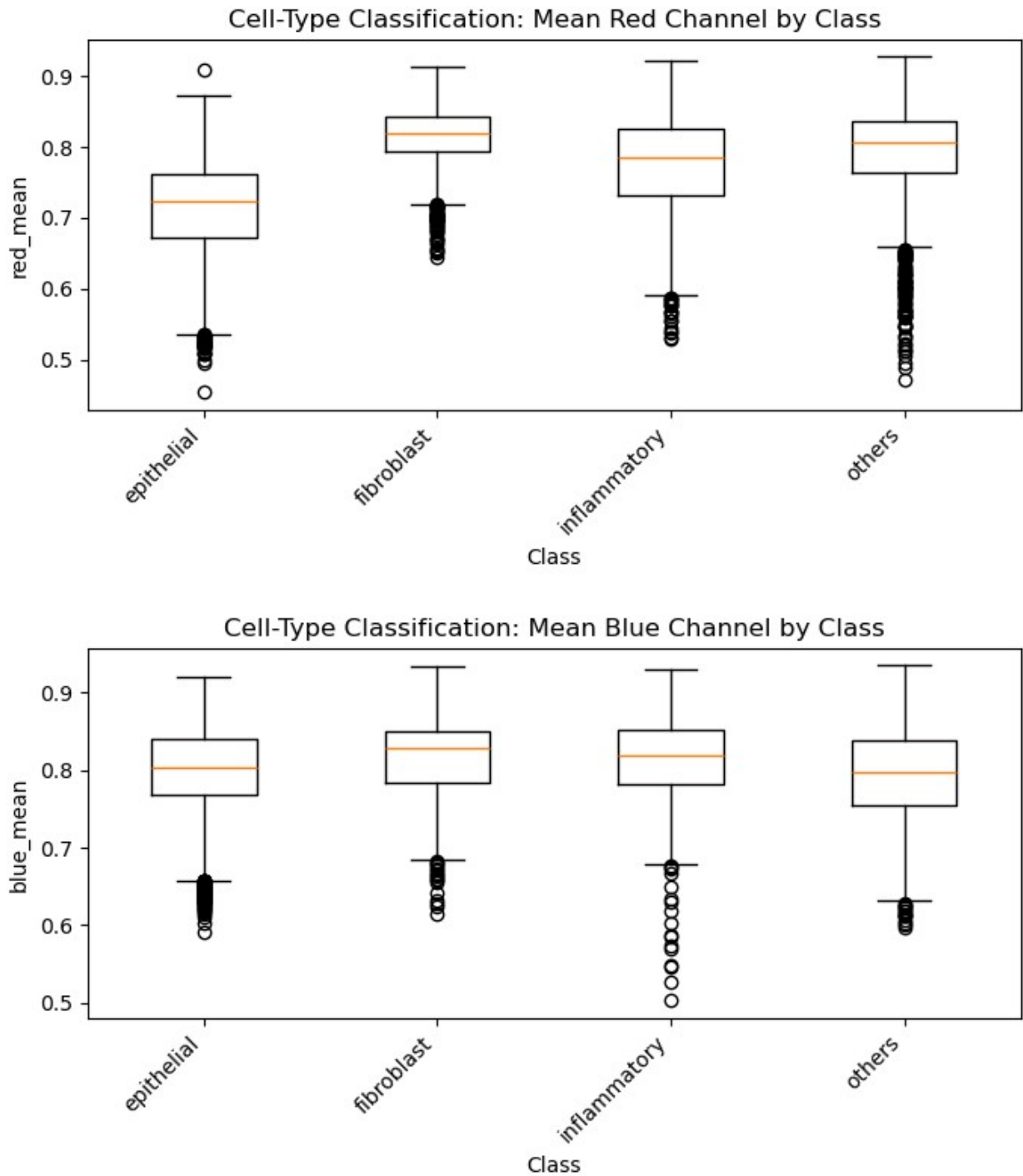
```

plot_feature_by_class(
    celltype_rgb_features,
    target_col="cellTypeName",
    feature_col="red_mean",
    title="Cell-Type Classification: Mean Red Channel by Class"
)

plot_feature_by_class(
    celltype_rgb_features,
    target_col="cellTypeName",
    feature_col="blue_mean",
    title="Cell-Type Classification: Mean Blue Channel by Class"
)

```





Interpretation: the RGB and intensity plots show whether classes differ in simple colour or brightness characteristics. If the distributions overlap strongly, then colour statistics alone are unlikely to be sufficient for strong classification. This supports the use of full pixel-based models and CNN models. The classical models use flattened pixel information, while the CNN preserves the spatial image structure and can learn local image patterns.

8.5 Average Image by Class

Average images are calculated for each class. This gives a high-level visual summary of colour and structure patterns. However, averaging also removes local texture and cell-level detail. Therefore, similar average images do not mean that the classes are impossible to distinguish. Instead, it suggests that useful patterns may be subtle and local.

```
def plot_average_image_by_class(df, target_col,
max_images_per_class=1000):
    """
    Compute and display the average image for each class.
    """
    classes = sorted(df[target_col].dropna().unique())

    plt.figure(figsize=(4 * len(classes), 4))

    for i, class_label in enumerate(classes):
        class_df = df[df[target_col] == class_label]

        if len(class_df) > max_images_per_class:
            class_df = class_df.sample(
                n=max_images_per_class,
                random_state=RANDOM_STATE
            )

        images = []

        for _, row in class_df.iterrows():
            img = Image.open(row["image_path"]).convert("RGB")
            arr = np.array(img).astype(np.float32) / 255.0
            images.append(arr)

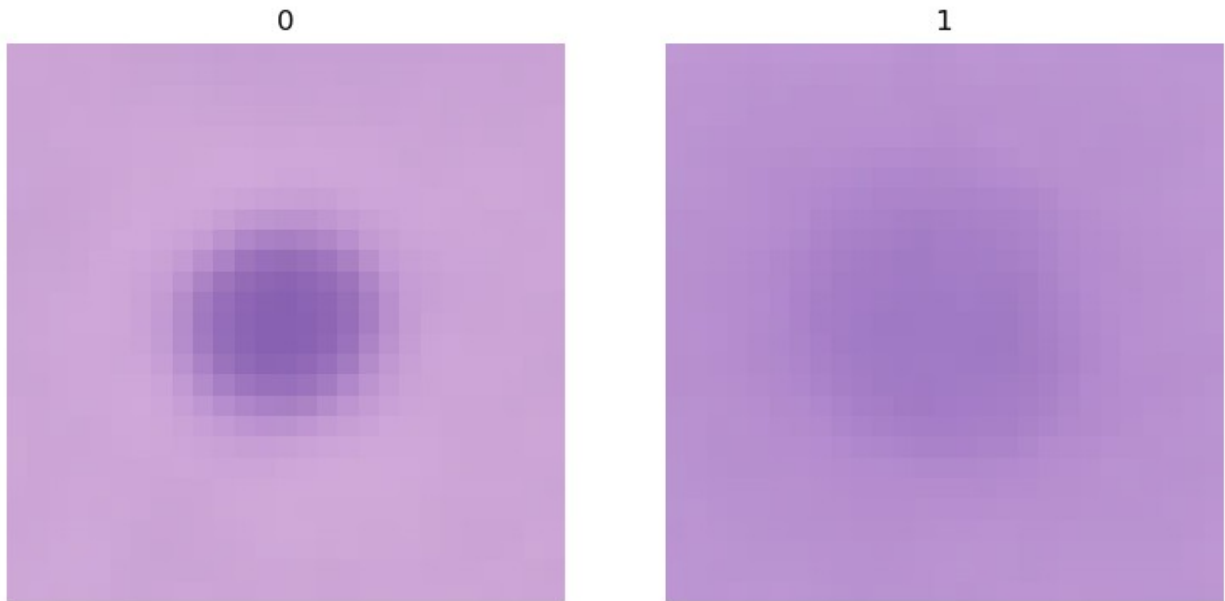
        avg_img = np.mean(images, axis=0)

        plt.subplot(1, len(classes), i + 1)
        plt.imshow(avg_img)
        plt.axis("off")
        plt.title(str(class_label))

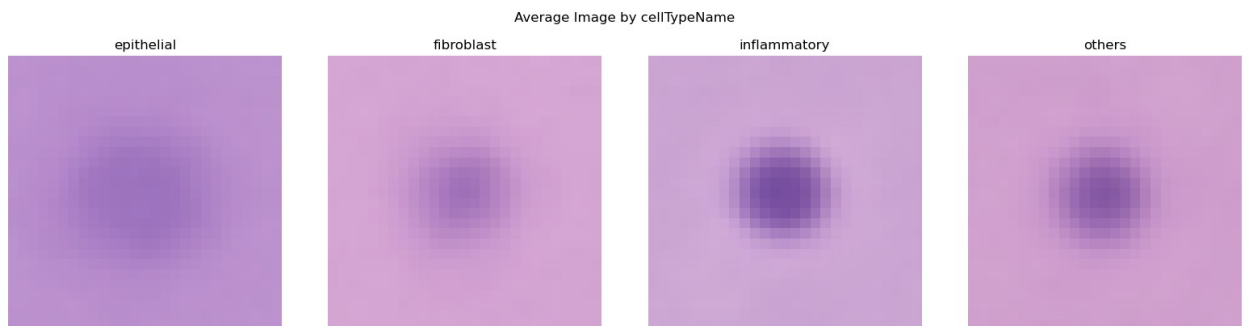
    plt.suptitle(f"Average Image by {target_col}", y=1.02)
    plt.tight_layout()
    plt.show()

plot_average_image_by_class(
    cancer_df,
    target_col="isCancerous",
    max_images_per_class=1000
)
```

Average Image by isCancerous



```
plot_average_image_by_class(  
    celltype_df,  
    target_col="cellTypeName",  
    max_images_per_class=1000  
)
```



Interpretation: the average images provide a high-level view of each class. However, averaging also removes much of the local texture and cell-level detail. Therefore, if the average images appear similar, it does not mean the classes are impossible to separate; it means the model likely needs to learn local patterns rather than relying only on global colour averages.

8.6 EDA Summary

The exploratory data analysis suggests several modelling challenges. First, the dataset is imbalanced, especially for the cell-type classification task. This means that macro-averaged metrics are needed alongside accuracy. Second, the number of images per patient varies. This confirms that patient-level splitting is necessary to avoid data leakage and to evaluate generalisation more realistically. Third, the image samples are small and visually similar. This

suggests that the classification tasks may require subtle colour, texture, and local spatial features. These observations support the modelling strategy used in this notebook. `LogisticRegression` provides a simple linear baseline, `RandomForestClassifier` provides a non-linear classical model, and the CNN provides an image-specific deep learning model that can learn local spatial patterns.

9. Unsupervised Learning EDA

This section uses unsupervised learning to further explore the structure of the image data.

Two unsupervised techniques are used:

- `PCA` for dimensionality reduction and visualisation.
- `KMeans` clustering to examine whether image samples naturally form groups.

The purpose of this section is not to replace supervised classification. Instead, it provides additional evidence about the difficulty of the dataset and whether the true labels are naturally separated in a lower-dimensional feature space.

9.1 Flattened Image Features for PCA and K-Means

PCA and K-Means are applied to flattened image vectors. Each $27 \times 27 \times 3$ image is converted into a one-dimensional vector of length 2187. A sample is used during development to keep the notebook efficient.

```
def load_flattened_images(df):  
    """  
    Load images, normalise pixel values to [0, 1],  
    and flatten each image into a one-dimensional vector.  
    """  
    X = []  
  
    for _, row in df.iterrows():  
        img = Image.open(row["image_path"]).convert("RGB")  
        arr = np.array(img).astype(np.float32) / 255.0  
        X.append(arr.flatten())  
  
    return np.array(X)  
  
#use samples for PCA/KMeans  
if QUICK_RUN:  
    cancer_unsup_sample = cancer_df.sample(n=min(3000,  
len(cancer_df)), random_state=RANDOM_STATE)  
    celltype_unsup_sample = celltype_df.sample(n=min(3000,  
len(celltype_df)), random_state=RANDOM_STATE)  
else:  
    cancer_unsup_sample = cancer_df.copy()  
    celltype_unsup_sample = celltype_df.copy()  
  
print("Cancer unsupervised sample size:", len(cancer_unsup_sample))
```

```

print("Cell-type unsupervised sample size:",
len(celltype_unsup_sample))

Cancer unsupervised sample size: 20280
Cell-type unsupervised sample size: 9896

X_cancer_unsup = load_flattened_images(cancer_unsup_sample)
X_celltype_unsup = load_flattened_images(celltype_unsup_sample)

print("Cancer flattened image matrix shape:", X_cancer_unsup.shape)
print("Cell-type flattened image matrix shape:",
X_celltype_unsup.shape)

Cancer flattened image matrix shape: (20280, 2187)
Cell-type flattened image matrix shape: (9896, 2187)

```

9.2 PCA Visualisation

Principal Component Analysis is used to project the high-dimensional image vectors into two dimensions. This helps visualise whether the classes show clear separation in a lower-dimensional feature space. If the classes overlap strongly, it suggests that simple linear projections of raw pixels are not enough to fully separate the labels.

```

def run_pca_visualisation(X, labels, title):
    """
    Run PCA and plot the first two principal components.
    """
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    pca = PCA(n_components=2, random_state=RANDOM_STATE)
    X_pca = pca.fit_transform(X_scaled)

    pca_df = pd.DataFrame({
        "PC1": X_pca[:, 0],
        "PC2": X_pca[:, 1],
        "label": labels.values
    })

    print(title)
    print("Explained variance ratio:", pca.explained_variance_ratio_)
    print("Total explained variance:",
pca.explained_variance_ratio_.sum())

    plt.figure(figsize=(7, 5))

    for class_label in sorted(pca_df["label"].unique()):
        class_data = pca_df[pca_df["label"] == class_label]
        plt.scatter(
            class_data["PC1"],

```

```

        class_data["PC2"],
        label=str(class_label),
        alpha=0.5,
        s=15
    )

    plt.title(title)
    plt.xlabel("Principal Component 1")
    plt.ylabel("Principal Component 2")
    plt.legend(title="Class")
    plt.tight_layout()
    plt.show()

    return pca_df, pca

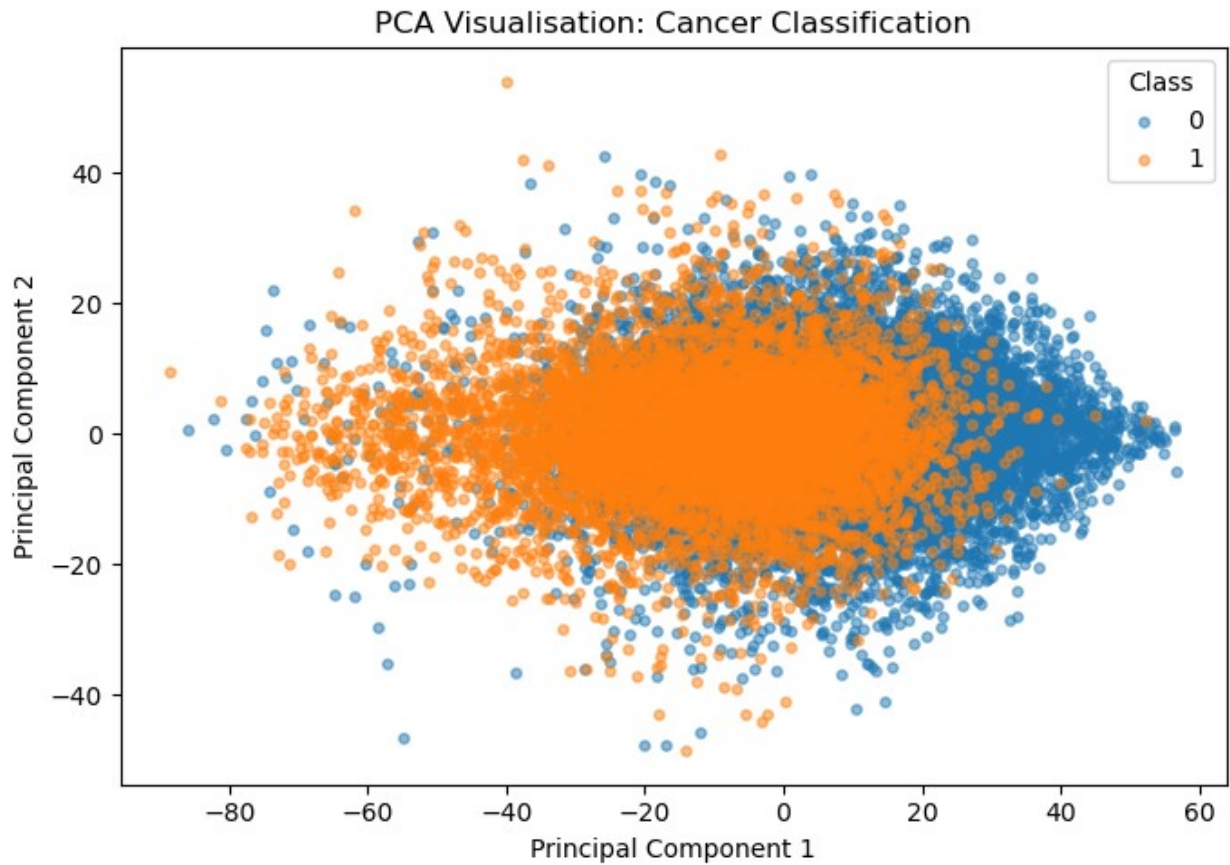
cancer_pca_df, cancer_pca_model = run_pca_visualisation(
    X_cancer_unsup,
    cancer_unsup_sample["isCancerous"],
    title="PCA Visualisation: Cancer Classification"
)

```

```

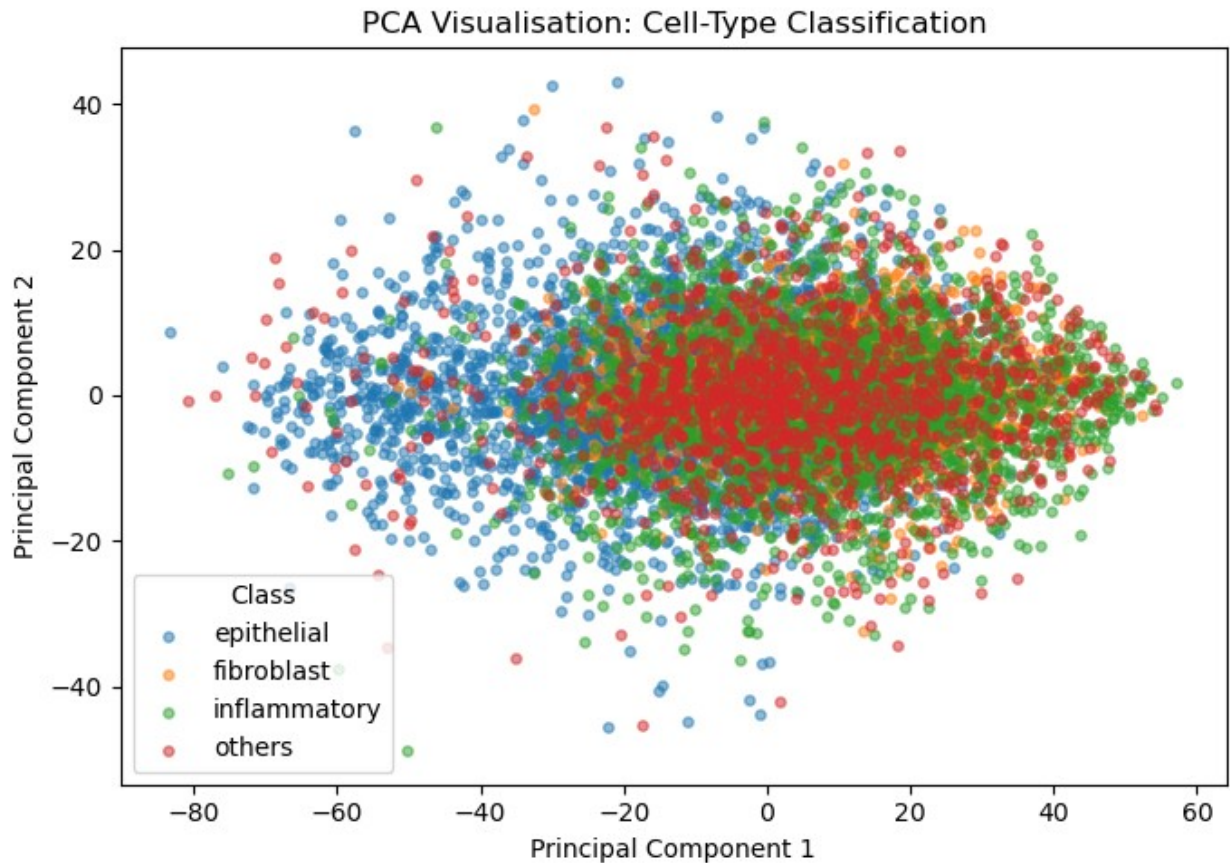
PCA Visualisation: Cancer Classification
Explained variance ratio: [0.1838282  0.04249262]
Total explained variance: 0.22632083

```



```
celltype_pca_df, celltype_pca_model = run_pca_visualisation(  
    X_celltype_unsup,  
    celltype_unsup_sample["cellTypeName"],  
    title="PCA Visualisation: Cell-Type Classification"  
)
```

PCA Visualisation: Cell-Type Classification
Explained variance ratio: [0.2100314 0.04023254]
Total explained variance: 0.25026393



Interpretation: the PCA plots show whether the classes form distinct groups when the high-dimensional pixel vectors are projected into two principal components. If the class points overlap, this suggests that the strongest linear variance directions do not fully separate the target labels. This does not mean classification is impossible. Instead, it suggests that supervised models may need to learn more complex decision boundaries or local spatial image patterns.

9.3 K-Means Clustering

K-Means clustering is used to examine whether the images naturally group into clusters without using the target labels. For the cancer classification task, $k = 2$ is used because there are two target classes. For the cell-type classification task, $k = 4$ is used because there are four target classes. The clustering results are compared with the true labels to see whether the natural cluster structure aligns with the supervised classification labels.

```
def run_kmeans_on_pca(pca_df, n_clusters, title):
    """
    Run K-Means on PCA coordinates and visualise the resulting
    clusters.
    """
    X_pca = pca_df[["PC1", "PC2"]].values
    kmeans = KMeans(
```

```

        n_clusters=n_clusters,
        random_state=RANDOM_STATE,
        n_init=10
    )

    cluster_labels = kmeans.fit_predict(X_pca)

    cluster_df = pca_df.copy()
    cluster_df["cluster"] = cluster_labels

    plt.figure(figsize=(7, 5))

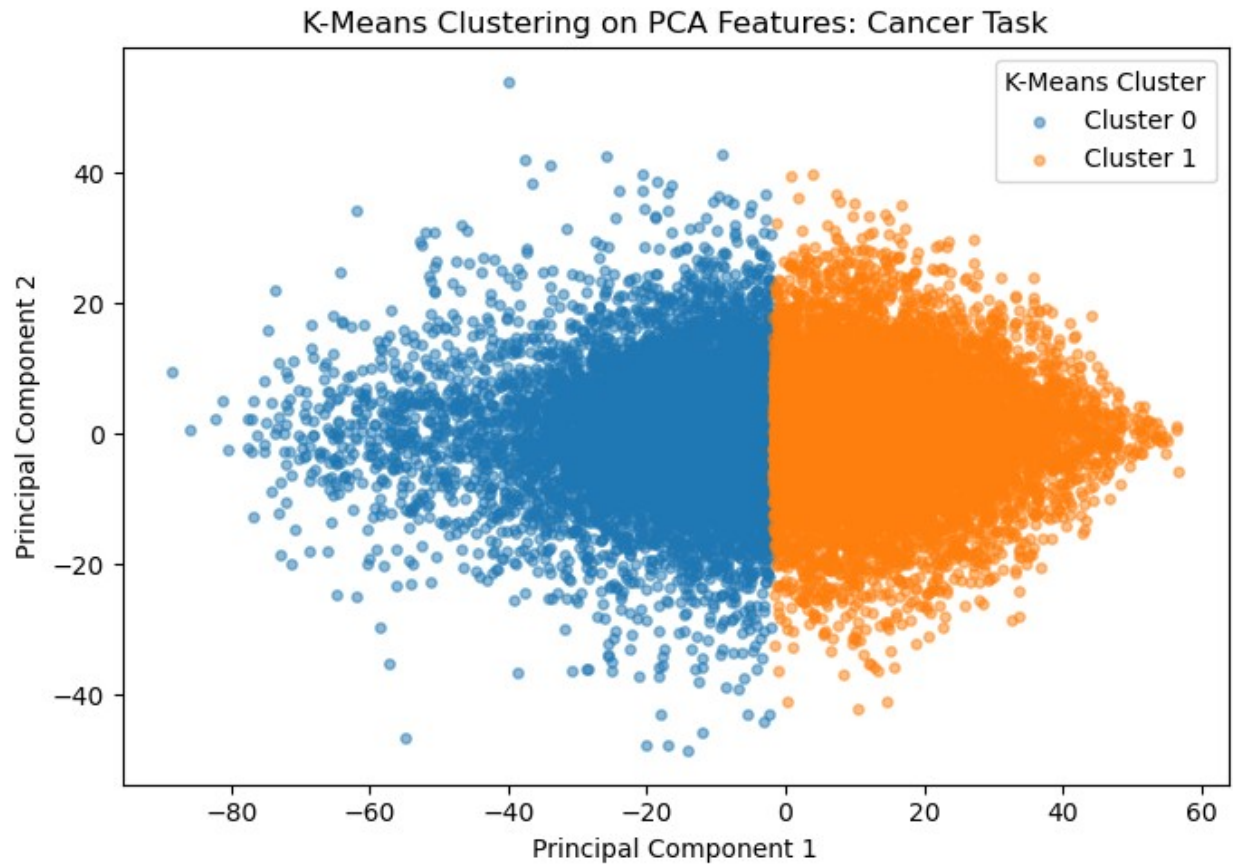
    for cluster in sorted(cluster_df["cluster"].unique()):
        cluster_data = cluster_df[cluster_df["cluster"] == cluster]
        plt.scatter(
            cluster_data["PC1"],
            cluster_data["PC2"],
            label=f"Cluster {cluster}",
            alpha=0.5,
            s=15
        )

    plt.title(title)
    plt.xlabel("Principal Component 1")
    plt.ylabel("Principal Component 2")
    plt.legend(title="K-Means Cluster")
    plt.tight_layout()
    plt.show()

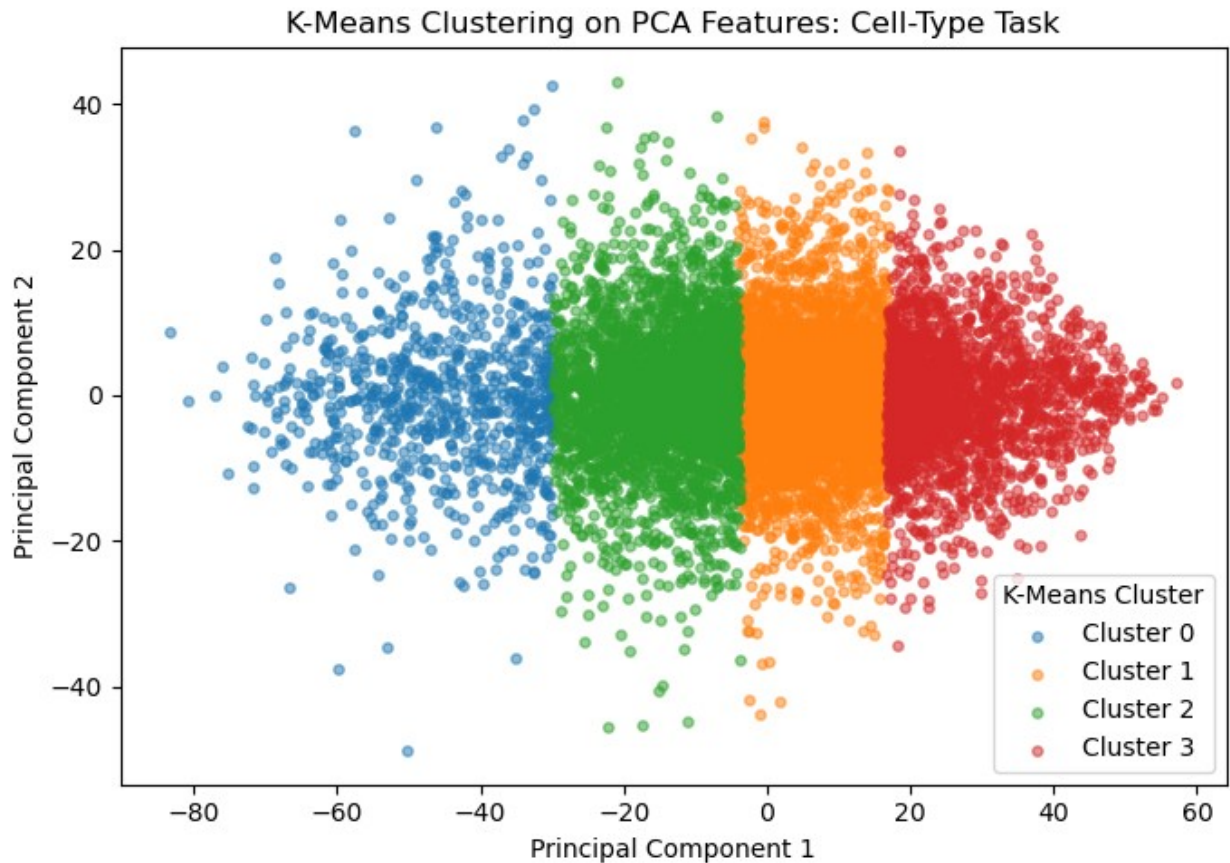
    return cluster_df, kmeans

cancer_cluster_df, cancer_kmeans = run_kmeans_on_pca(
    cancer_pca_df,
    n_clusters=2,
    title="K-Means Clustering on PCA Features: Cancer Task"
)

```



```
celltype_cluster_df, celltype_kmeans = run_kmeans_on_pca(  
    celltype_pca_df,  
    n_clusters=4,  
    title="K-Means Clustering on PCA Features: Cell-Type Task"  
)
```



9.4 Cluster and True Label Comparison

The K-Means clusters are compared with the true labels using cross-tabulation. If each cluster mainly contains one true class, this would suggest strong natural separation in the PCA feature space. If the clusters contain mixed labels, this suggests that the classes are not naturally separated by simple distance-based clustering.

```
print("Cancer task: K-Means cluster vs true label")
```

```
display(pd.crosstab(
    cancer_cluster_df["cluster"],
    cancer_cluster_df["label"],
    normalize="index"
).round(3))
```

Cancer task: K-Means cluster vs true label

label	0	1
cluster		
0	0.479	0.521
1	0.788	0.212

```
print("Cell-type task: K-Means cluster vs true label")

display(pd.crosstab(
    celltype_cluster_df["cluster"],
    celltype_cluster_df["label"],
    normalize="index"
).round(3))
```

Cell-type task: K-Means cluster vs true label

label cluster	epithelial	fibroblast	inflammatory	others
0	0.796	0.017	0.081	0.106
1	0.392	0.206	0.261	0.141
2	0.526	0.131	0.212	0.131
3	0.132	0.318	0.384	0.166

Interpretation: the cluster-label tables show whether unsupervised clusters align with the true supervised labels. If the clusters are mixed, this suggests that the class boundaries are not simple. This is expected for small histology images, where classes may overlap in colour and texture. These results support the use of supervised models rather than relying on unsupervised grouping alone.

9.5 Unsupervised EDA Summary

The unsupervised learning analysis provides additional evidence about the difficulty of the classification tasks. PCA gives a low-dimensional view of the image data, while K-Means checks whether samples naturally group in a way that resembles the true labels. If the PCA plots show overlap and the K-Means clusters contain mixed labels, this suggests that the tasks require supervised learning models. These findings support the next modelling stage. The notebook will compare `LogisticRegression`, `RandomForestClassifier`, and a CNN to examine how linear, non-linear, and image-specific models perform on the two classification tasks.

10. Data Pre-processing for Modelling

This section prepares the image data for supervised model development.

Two different input formats are prepared because the selected models use the image data in different ways:

1. **Flattened image vectors** are used for the classical machine learning models: `LogisticRegression` and `RandomForestClassifier`.
2. **Image tensors** are used for the Convolutional Neural Network (CNN).

All image pixel values are normalised to the range `[0, 1]`. This avoids using raw pixel values from 0 to 255 and helps make model training more stable. The target labels are also encoded into numeric form using `LabelEncoder`, so that they can be used by both scikit-learn and PyTorch models.

```

def load_images_flat(df):
    """
    load images as flattened vectors

    each 27 x 27 x 3 RGB image is converted into a vector of length
2187
    this format is used for classical machine learning models
    """
    X = []

    for _, row in df.iterrows():
        img = Image.open(row["image_path"]).convert("RGB")
        arr = np.array(img).astype(np.float32) / 255.0
        X.append(arr.flatten())

    return np.array(X)

def load_images_tensor(df):
    """
    load images as tensors for PyTorch

    PyTorch expects images in channels x height x width format
    each image is converted from 27 x 27 x 3 to 3 x 27 x 27
    """
    X = []

    for _, row in df.iterrows():
        img = Image.open(row["image_path"]).convert("RGB")
        arr = np.array(img).astype(np.float32) / 255.0

        # Convert from H x W x C to C x H x W
        arr = np.transpose(arr, (2, 0, 1))
        X.append(arr)

    return np.array(X)

```

10.1 Encode Target Labels

Machine learning models require numeric target labels. For the cancer classification task, the `isCancerous` target is already numeric, but it is still encoded consistently so that the workflow is standardised. For the cell-type classification task, `cellTypeName` is converted from text labels into integer class labels.

```

#label encoders for both tasks
cancer_label_encoder = LabelEncoder()
celltype_label_encoder = LabelEncoder()

#fit encoders on the full task datasets so the mapping is consistent
across splits

```

```

cancer_label_encoder.fit(cancer_df["isCancerous"])
celltype_label_encoder.fit(celltype_df["cellTypeName"])

print("Cancer label mapping:")
for class_label, encoded_label in zip(
    cancer_label_encoder.classes_,
    cancer_label_encoder.transform(cancer_label_encoder.classes_)
):
    print(f"{class_label} -> {encoded_label}")

print("\nCell-type label mapping:")
for class_label, encoded_label in zip(
    celltype_label_encoder.classes_,
    celltype_label_encoder.transform(celltype_label_encoder.classes_)
):
    print(f"{class_label} -> {encoded_label}")

Cancer label mapping:
0 -> 0
1 -> 1

Cell-type label mapping:
epithelial -> 0
fibroblast -> 1
inflammatory -> 2
others -> 3

```

10.2 Prepare Flattened Features for Classical Models

The classical machine learning models use flattened image vectors. Each $27 \times 27 \times 3$ image is converted into a vector of length 2187. This representation is suitable for `LogisticRegression` and `RandomForestClassifier`. A limitation of this representation is that the spatial structure of the image is not explicitly preserved. This means the classical models treat the pixels as numerical features, but they do not directly understand neighbouring pixel relationships.

```

#cancer classification arrays for classical machine learning
X_cancer_train_flat = load_images_flat(cancer_train_df)
X_cancer_val_flat = load_images_flat(cancer_val_df)
X_cancer_test_flat = load_images_flat(cancer_test_df)

y_cancer_train =
cancer_label_encoder.transform(cancer_train_df["isCancerous"])
y_cancer_val =
cancer_label_encoder.transform(cancer_val_df["isCancerous"])
y_cancer_test =
cancer_label_encoder.transform(cancer_test_df["isCancerous"])

print("Cancer classical ML arrays:")

```

```

print("X_train:", X_cancer_train_flat.shape)
print("X_val:", X_cancer_val_flat.shape)
print("X_test:", X_cancer_test_flat.shape)
print("y_train:", y_cancer_train.shape)
print("y_val:", y_cancer_val.shape)
print("y_test:", y_cancer_test.shape)

Cancer classical ML arrays:
X_train: (12319, 2187)
X_val: (4005, 2187)
X_test: (3956, 2187)
y_train: (12319,)
y_val: (4005,)
y_test: (3956,)

#cell-type classification arrays for classical machine learning
X_celltype_train_flat = load_images_flat(celltype_train_df)
X_celltype_val_flat = load_images_flat(celltype_val_df)
X_celltype_test_flat = load_images_flat(celltype_test_df)

y_celltype_train =
celltype_label_encoder.transform(celltype_train_df["cellTypeName"])
y_celltype_val =
celltype_label_encoder.transform(celltype_val_df["cellTypeName"])
y_celltype_test =
celltype_label_encoder.transform(celltype_test_df["cellTypeName"])

print("Cell-type classical ML arrays:")
print("X_train:", X_celltype_train_flat.shape)
print("X_val:", X_celltype_val_flat.shape)
print("X_test:", X_celltype_test_flat.shape)
print("y_train:", y_celltype_train.shape)
print("y_val:", y_celltype_val.shape)
print("y_test:", y_celltype_test.shape)

Cell-type classical ML arrays:
X_train: (6215, 2187)
X_val: (1842, 2187)
X_test: (1839, 2187)
y_train: (6215,)
y_val: (1842,)
y_test: (1839,)

```

10.3 Prepare Image Tensors for CNN

The CNN uses the image-shaped representation rather than flattened vectors.

PyTorch expects image tensors in the following format: channels × height × width

```

#cancer classification tensors for PyTorch
X_cancer_train_tensor = load_images_tensor(cancer_train_df)
X_cancer_val_tensor = load_images_tensor(cancer_val_df)
X_cancer_test_tensor = load_images_tensor(cancer_test_df)

print("Cancer PyTorch tensor arrays:")
print("X_train:", X_cancer_train_tensor.shape)
print("X_val:", X_cancer_val_tensor.shape)
print("X_test:", X_cancer_test_tensor.shape)

Cancer PyTorch tensor arrays:
X_train: (12319, 3, 27, 27)
X_val: (4005, 3, 27, 27)
X_test: (3956, 3, 27, 27)

# Cell-type classification tensors for CNN
X_celltype_train_tensor = load_images_tensor(celltype_train_df)
X_celltype_val_tensor = load_images_tensor(celltype_val_df)
X_celltype_test_tensor = load_images_tensor(celltype_test_df)

print("Cell-type PyTorch tensor arrays:")
print("X_train:", X_celltype_train_tensor.shape)
print("X_val:", X_celltype_val_tensor.shape)
print("X_test:", X_celltype_test_tensor.shape)

Cell-type PyTorch tensor arrays:
X_train: (6215, 3, 27, 27)
X_val: (1842, 3, 27, 27)
X_test: (1839, 3, 27, 27)

```

10.4 PyTorch Dataset and DataLoader

A custom PyTorch `Dataset` class is created so that image tensors and labels can be loaded in mini-batches during CNN training. Mini-batch training is used because it is more efficient than processing the entire dataset at once. During each training step, the CNN receives a batch of images, calculates the loss, computes gradients using backpropagation, and updates its weights using the optimiser.

```

class HistologyImageDataset(Dataset):
    """
    PyTorch Dataset for histology image classification
    """

    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)

    def __len__(self):
        return len(self.X)

```

```

def __getitem__(self, idx):
    return self.X[idx], self.y[idx]

#batch size setting
BATCH_SIZE = 64

#cancer datasets and loaders
cancer_train_dataset = HistologyImageDataset(X_cancer_train_tensor,
y_cancer_train)
cancer_val_dataset = HistologyImageDataset(X_cancer_val_tensor,
y_cancer_val)
cancer_test_dataset = HistologyImageDataset(X_cancer_test_tensor,
y_cancer_test)

cancer_train_loader = DataLoader(cancer_train_dataset,
batch_size=BATCH_SIZE, shuffle=True)
cancer_val_loader = DataLoader(cancer_val_dataset,
batch_size=BATCH_SIZE, shuffle=False)
cancer_test_loader = DataLoader(cancer_test_dataset,
batch_size=BATCH_SIZE, shuffle=False)

#cell-type datasets and loaders
celltype_train_dataset =
HistologyImageDataset(X_celltype_train_tensor, y_celltype_train)
celltype_val_dataset = HistologyImageDataset(X_celltype_val_tensor,
y_celltype_val)
celltype_test_dataset = HistologyImageDataset(X_celltype_test_tensor,
y_celltype_test)

celltype_train_loader = DataLoader(celltype_train_dataset,
batch_size=BATCH_SIZE, shuffle=True)
celltype_val_loader = DataLoader(celltype_val_dataset,
batch_size=BATCH_SIZE, shuffle=False)
celltype_test_loader = DataLoader(celltype_test_dataset,
batch_size=BATCH_SIZE, shuffle=False)

print("DataLoaders created successfully.")
print("Batch size:", BATCH_SIZE)

DataLoaders created successfully.
Batch size: 64

```

10.5 Pre-processing Summary

the data is now prepared in two formats. The flattened image vectors are used for `LogisticRegression` and `RandomForestClassifier`. These models provide classical machine learning baselines. The image tensors are used for the CNN. This allows the neural network to preserve the spatial image structure and learn local visual patterns. The target variables have been encoded into numeric labels, and all image pixel values have been normalised to `[0, 1]`.

11. Evaluation Framework

This section defines reusable functions for model evaluation.

The assignment requires supervised models to be compared using suitable classification metrics. This notebook reports:

- accuracy
- macro_f1
- weighted_f1
- macro_precision
- macro_recall
- classification report
- confusion matrix

The main metric used for model comparison is `macro_f1`. This is because the dataset is imbalanced, and macro-averaged F1-score gives equal importance to each class. Confusion matrices are also used because they show the specific types of errors made by each model. This is important for identifying whether a model is biased toward majority classes or struggles with particular cell types.

11.1 Metric Calculation Function

```
def calculate_classification_metrics(y_true, y_pred, model_name,
task_name):
    """
    Calculate core classification metrics and return them in a
    dictionary.
    """
    metrics = {
        "task": task_name,
        "model": model_name,
        "accuracy": accuracy_score(y_true, y_pred),
        "macro_f1": f1_score(y_true, y_pred, average="macro",
zero_division=0),
        "weighted_f1": f1_score(y_true, y_pred, average="weighted",
zero_division=0),
        "macro_precision": precision_score(y_true, y_pred,
average="macro", zero_division=0),
        "macro_recall": recall_score(y_true, y_pred, average="macro",
zero_division=0)
    }

    return metrics
```

11.2 Prediction Evaluation Function

```
def evaluate_predictions(
    y_true,
```

```

y_pred,
model_name,
task_name,
class_names=None,
show_report=True,
show_confusion_matrix=True
):
    """
    Evaluate predictions using classification metrics, a
    classification report,
    and a confusion matrix.
    """
    metrics = calculate_classification_metrics(
        y_true=y_true,
        y_pred=y_pred,
        model_name=model_name,
        task_name=task_name
    )

    print("=" * 80)
    print(f"Task: {task_name}")
    print(f"Model: {model_name}")
    print("=" * 80)
    print(f"Accuracy:           {metrics['accuracy']:.4f}")
    print(f"Macro F1:             {metrics['macro_f1']:.4f}")
    print(f"Weighted F1:          {metrics['weighted_f1']:.4f}")
    print(f"Macro Precision:      {metrics['macro_precision']:.4f}")
    print(f"Macro Recall:         {metrics['macro_recall']:.4f}")

    if show_report:
        print("\nClassification Report:")
        print(classification_report(
            y_true,
            y_pred,
            target_names=class_names,
            zero_division=0
        ))

    if show_confusion_matrix:
        cm = confusion_matrix(y_true, y_pred)

        disp = ConfusionMatrixDisplay(
            confusion_matrix=cm,
            display_labels=class_names
        )

        fig, ax = plt.subplots(figsize=(6, 5))
        disp.plot(ax=ax, values_format="d", cmap=None)
        plt.title(f"Confusion Matrix: {model_name} ({task_name})")
        plt.xticks(rotation=45, ha="right")

```

```
plt.tight_layout()
plt.show()

return metrics
```

11.3 Result Storage

The validation and test results are stored in lists so they can be converted into summary tables later. This makes model comparison more organised and reduces repeated manual work.

```
#storing all validation and test results
validation_results = []
test_results = []

print("Result storage lists created.")

Result storage lists created.
```

11.4 Classical Model Training Helper

This helper function trains a scikit-learn model, predicts on the validation set, evaluates the results, and returns the fitted model and metrics. It is used for both `LogisticRegression` and `RandomForestClassifier`.

```
def train_evaluate_sklearn_model(
    model,
    model_name,
    task_name,
    X_train,
    y_train,
    X_val,
    y_val,
    class_names
):
    """
    Train and evaluate a scikit-learn classification model.
    """
    print(f"Training {model_name} for {task_name}...")

    model.fit(X_train, y_train)
    y_val_pred = model.predict(X_val)

    metrics = evaluate_predictions(
        y_true=y_val,
        y_pred=y_val_pred,
        model_name=model_name,
        task_name=task_name,
        class_names=class_names,
        show_report=True,
```

```

        show_confusion_matrix=True
    )

    return model, metrics

```

11.5 Class Name Variables

```

#class names for readable reports and confusion matrices
cancer_class_names = [str(label) for label in
cancer_label_encoder.classes_]
celltype_class_names = [str(label) for label in
celltype_label_encoder.classes_]

print("Cancer class names:", cancer_class_names)
print("Cell-type class names:", celltype_class_names)

Cancer class names: ['0', '1']
Cell-type class names: ['epithelial', 'fibroblast', 'inflammatory',
'others']

```

11.6 PyTorch Prediction Function

The following function evaluates a PyTorch model on a given `DataLoader`. The model outputs raw class scores called logits. The predicted class is selected using `argmax`, which returns the class with the highest score.

```

def predict_pytorch_model(model, data_loader, device):
    """
    Generate predictions from a PyTorch model.
    """
    model.eval()

    all_preds = []
    all_true = []

    with torch.no_grad():
        for X_batch, y_batch in data_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            outputs = model(X_batch)
            preds = torch.argmax(outputs, dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_true.extend(y_batch.cpu().numpy())

    return np.array(all_true), np.array(all_preds)

```

11.7 PyTorch Training Function

This function trains a CNN using mini-batch training. For each epoch, the model performs a forward pass, calculates cross-entropy loss, applies backpropagation, and updates the model weights using the optimiser. Cross-entropy loss is used because both tasks are classification problems. Validation accuracy and validation macro F1-score are recorded after each epoch so that model learning can be inspected.

```
def train_pytorch_model(
    model,
    train_loader,
    val_loader,
    criterion,
    optimizer,
    device,
    num_epochs=10,
    model_name="CNN",
    task_name="Classification",
    class_names=None
):
    """
    Train a PyTorch classification model and keep the model state
    with the best validation macro F1-score.
    """
    model = model.to(device)

    history = {
        "train_loss": [],
        "val_accuracy": [],
        "val_macro_f1": []
    }

    best_val_macro_f1 = -1
    best_epoch = 0
    best_model_state = copy.deepcopy(model.state_dict())

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0

        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()

            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)

            loss.backward()
```

```

        optimizer.step()

        running_loss += loss.item() * X_batch.size(0)

    epoch_loss = running_loss / len(train_loader.dataset)

    y_val_true, y_val_pred = predict_pytorch_model(
        model=model,
        data_loader=val_loader,
        device=device
    )

    val_accuracy = accuracy_score(y_val_true, y_val_pred)
    val_macro_f1 = f1_score(
        y_val_true,
        y_val_pred,
        average="macro",
        zero_division=0
    )

    history["train_loss"].append(epoch_loss)
    history["val_accuracy"].append(val_accuracy)
    history["val_macro_f1"].append(val_macro_f1)

    if val_macro_f1 > best_val_macro_f1:
        best_val_macro_f1 = val_macro_f1
        best_epoch = epoch + 1
        best_model_state = copy.deepcopy(model.state_dict())

    print(
        f"Epoch [{epoch + 1}/{num_epochs}] "
        f"Train Loss: {epoch_loss:.4f} "
        f"Val Accuracy: {val_accuracy:.4f} "
        f"Val Macro F1: {val_macro_f1:.4f}"
    )

    print("\nBest validation epoch:", best_epoch)
    print("Best validation macro F1:", round(best_val_macro_f1, 4))

    #restore the best validation model before final validation
    evaluation
    model.load_state_dict(best_model_state)

    y_val_true, y_val_pred = predict_pytorch_model(
        model=model,
        data_loader=val_loader,
        device=device
    )

    metrics = evaluate_predictions(

```

```

        y_true=y_val_true,
        y_pred=y_val_pred,
        model_name=model_name,
        task_name=task_name,
        class_names=class_names,
        show_report=True,
        show_confusion_matrix=True
    )

    metrics["best_epoch"] = best_epoch
    metrics["best_val_macro_f1"] = best_val_macro_f1

    return model, history, metrics

```

11.8 Training Curve Plot Function

```

def plot_training_history(history, title):
    """
    Plot training loss and validation macro F1 across epochs
    """
    epochs = range(1, len(history["train_loss"]) + 1)

    best_epoch = int(np.argmax(history["val_macro_f1"]) + 1)
    best_f1 = max(history["val_macro_f1"])

    plt.figure(figsize=(7, 4))
    plt.plot(epochs, history["train_loss"], marker="o",
    label="Training Loss")
    plt.title(f"{title}: Training Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(7, 4))
    plt.plot(epochs, history["val_macro_f1"], marker="o",
    label="Validation Macro F1")
    plt.axvline(best_epoch, linestyle="--", label=f"Best epoch:
    {best_epoch}")
    plt.title(f"{title}: Validation Macro F1")
    plt.xlabel("Epoch")
    plt.ylabel("Macro F1")
    plt.legend()
    plt.tight_layout()
    plt.show()

    print(f"Best epoch for {title}: {best_epoch}")
    print(f"Best validation macro F1 for {title}: {best_f1:.4f}")

```

11.9 Evaluation Framework Summary

The evaluation framework is now ready for both classical machine learning models and CNN models. The same metrics will be used across all models, which makes the comparison consistent. The validation set is used for model comparison, while the test set remains separate until final evaluation.

12. Classical Machine Learning Baselines

This section trains three classical supervised learning models on flattened image features:

1. `LogisticRegression`
2. `SVC`
3. `RandomForestClassifier`

These models are used as classical baselines before training the CNN.

- `LogisticRegression` is included as a simple linear classification baseline. Although its name contains “regression”, it is used here for classification by modelling class probabilities and applying a decision boundary.
- `SVC` is included as a margin-based classifier. It can perform well with high-dimensional feature vectors, which is relevant because each image is flattened into a vector of length 2187.
- `RandomForestClassifier` is included as a non-linear tree-based ensemble model. It combines multiple decision trees and can capture non-linear relationships between pixel values.

All three models are trained for both prediction tasks so that the comparison remains consistent.

12.1 Logistic Regression for Cancer Classification

```
logreg_cancer = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(
        max_iter=1000,
        class_weight="balanced",
        random_state=RANDOM_STATE
    ))
])

logreg_cancer, logreg_cancer_metrics = train_evaluate_sklearn_model(
    model=logreg_cancer,
    model_name="Logistic Regression",
    task_name="Cancer Classification",
    X_train=X_cancer_train_flat,
    y_train=y_cancer_train,
    X_val=X_cancer_val_flat,
```

```
    y_val=y_cancer_val,  
    class_names=cancer_class_names  
)
```

```
validation_results.append(logreg_cancer_metrics)
```

Training Logistic Regression for Cancer Classification...

```
=====
```

Task: Cancer Classification

Model: Logistic Regression

```
=====
```

```
=====
```

Accuracy: 0.7730

Macro F1: 0.7710

Weighted F1: 0.7743

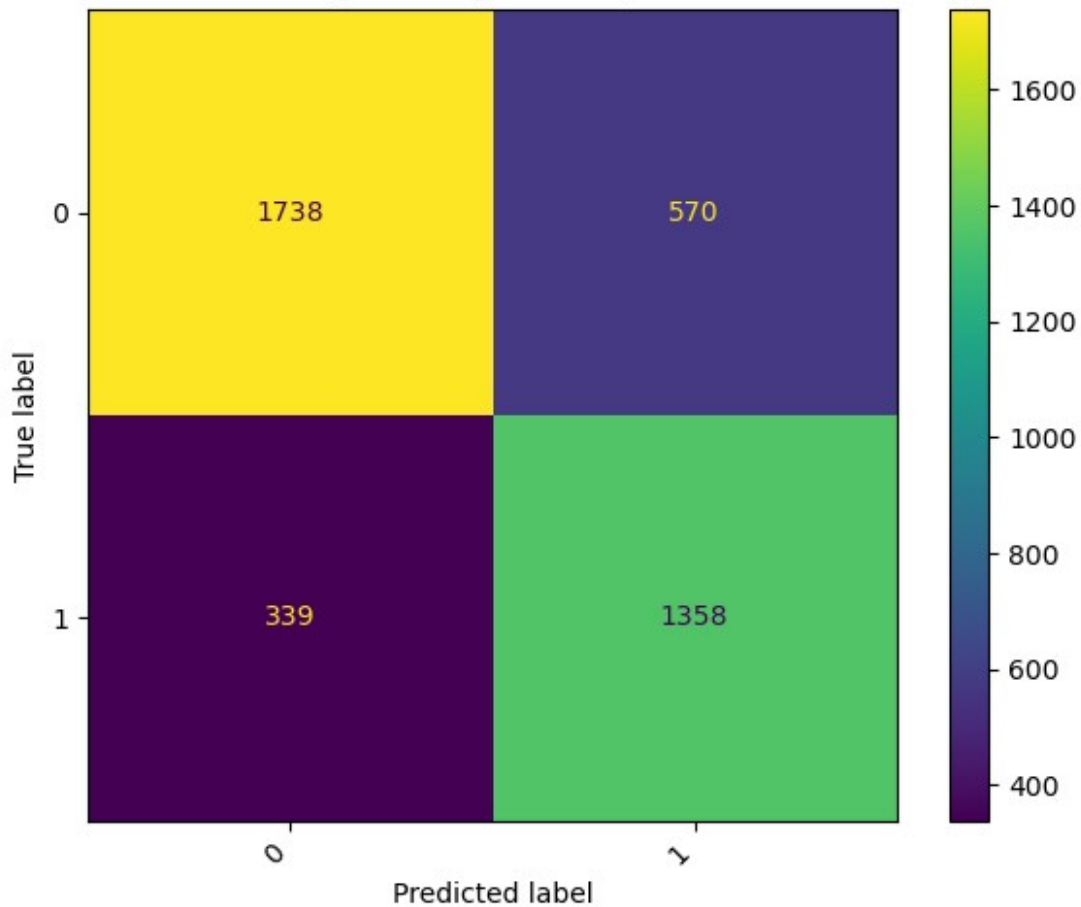
Macro Precision: 0.7706

Macro Recall: 0.7766

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.75	0.79	2308
1	0.70	0.80	0.75	1697
accuracy			0.77	4005
macro avg	0.77	0.78	0.77	4005
weighted avg	0.78	0.77	0.77	4005

Confusion Matrix: Logistic Regression (Cancer Classification)



12.2 Logistic Regression for Cell-Type Classification

```
logreg_celltype = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(
        max_iter=1000,
        class_weight="balanced",
        random_state=RANDOM_STATE
    ))
])

logreg_celltype, logreg_celltype_metrics =
train_evaluate_sklearn_model(
    model=logreg_celltype,
    model_name="Logistic Regression",
    task_name="Cell-Type Classification",
    X_train=X_celltype_train_flat,
    y_train=y_celltype_train,
    X_val=X_celltype_val_flat,
    y_val=y_celltype_val,
    class_names=celltype_class_names
```

```
)  
validation_results.append(logreg_celltype_metrics)
```

Training Logistic Regression for Cell-Type Classification...

```
=====
```

Task: Cell-Type Classification
Model: Logistic Regression

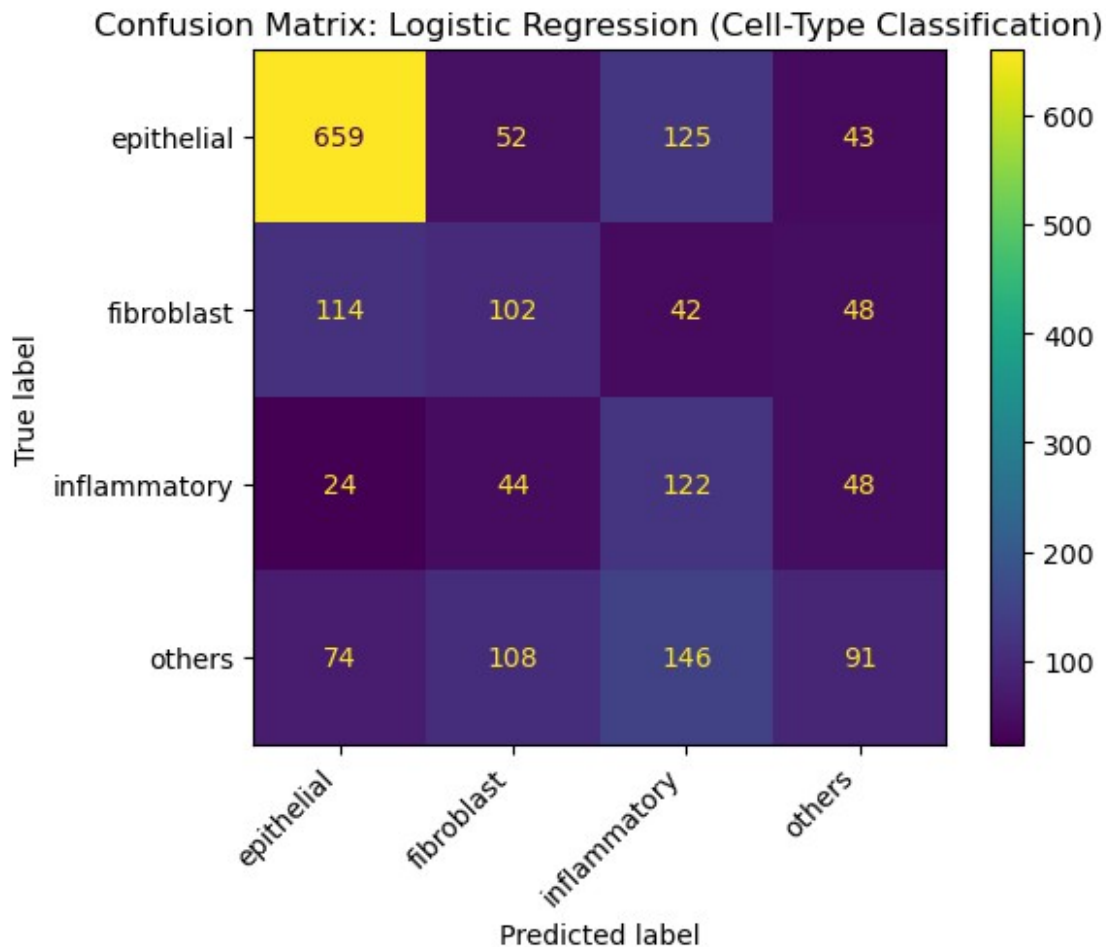
```
=====
```

```
=====
```

Accuracy: 0.5288
Macro F1: 0.4324
Weighted F1: 0.5254
Macro Precision: 0.4415
Macro Recall: 0.4532

Classification Report:

	precision	recall	f1-score	support
epithelial	0.76	0.75	0.75	879
fibroblast	0.33	0.33	0.33	306
inflammatory	0.28	0.51	0.36	238
others	0.40	0.22	0.28	419
accuracy			0.53	1842
macro avg	0.44	0.45	0.43	1842
weighted avg	0.54	0.53	0.53	1842



12.3 SVM RBF for Cancer Classification

The `SVC` model is trained using an RBF kernel. The RBF kernel allows the model to learn non-linear decision boundaries, which may be useful because the image classes are unlikely to be perfectly linearly separable in flattened pixel space. A `StandardScaler` is used because SVM models are sensitive to the scale of the input features.

```
svm_cancer = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC(
        kernel="rbf",
        C=10,
        gamma="scale",
        class_weight="balanced",
        random_state=RANDOM_STATE
    ))
])

svm_cancer, svm_cancer_metrics = train_evaluate_sklearn_model(
    model=svm_cancer,
```

```

    model_name="SVM RBF",
    task_name="Cancer Classification",
    X_train=X_cancer_train_flat,
    y_train=y_cancer_train,
    X_val=X_cancer_val_flat,
    y_val=y_cancer_val,
    class_names=cancer_class_names
)

validation_results.append(svm_cancer_metrics)

```

Training SVM RBF for Cancer Classification...

=====

Task: Cancer Classification

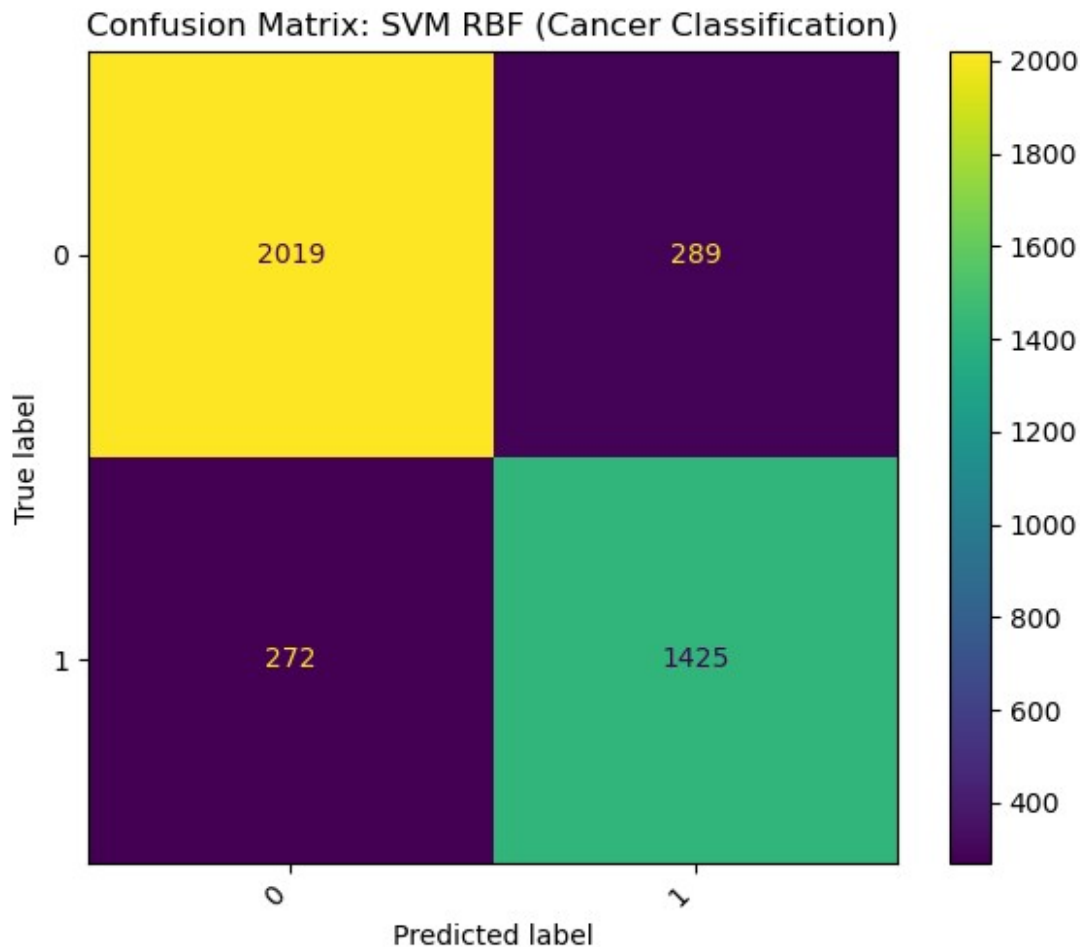
Model: SVM RBF

=====

Accuracy: 0.8599
 Macro F1: 0.8568
 Weighted F1: 0.8600
 Macro Precision: 0.8563
 Macro Recall: 0.8573

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.87	0.88	2308
1	0.83	0.84	0.84	1697
accuracy			0.86	4005
macro avg	0.86	0.86	0.86	4005
weighted avg	0.86	0.86	0.86	4005



12.4 SVM RBF for Cell-Type Classification

```
svm_celltype = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC(
        kernel="rbf",
        C=10,
        gamma="scale",
        class_weight="balanced",
        random_state=RANDOM_STATE
    ))
])

svm_celltype, svm_celltype_metrics = train_evaluate_sklearn_model(
    model=svm_celltype,
    model_name="SVM RBF",
    task_name="Cell-Type Classification",
    X_train=X_celltype_train_flat,
    y_train=y_celltype_train,
    X_val=X_celltype_val_flat,
    y_val=y_celltype_val,
```

```

    class_names=celltype_class_names
)

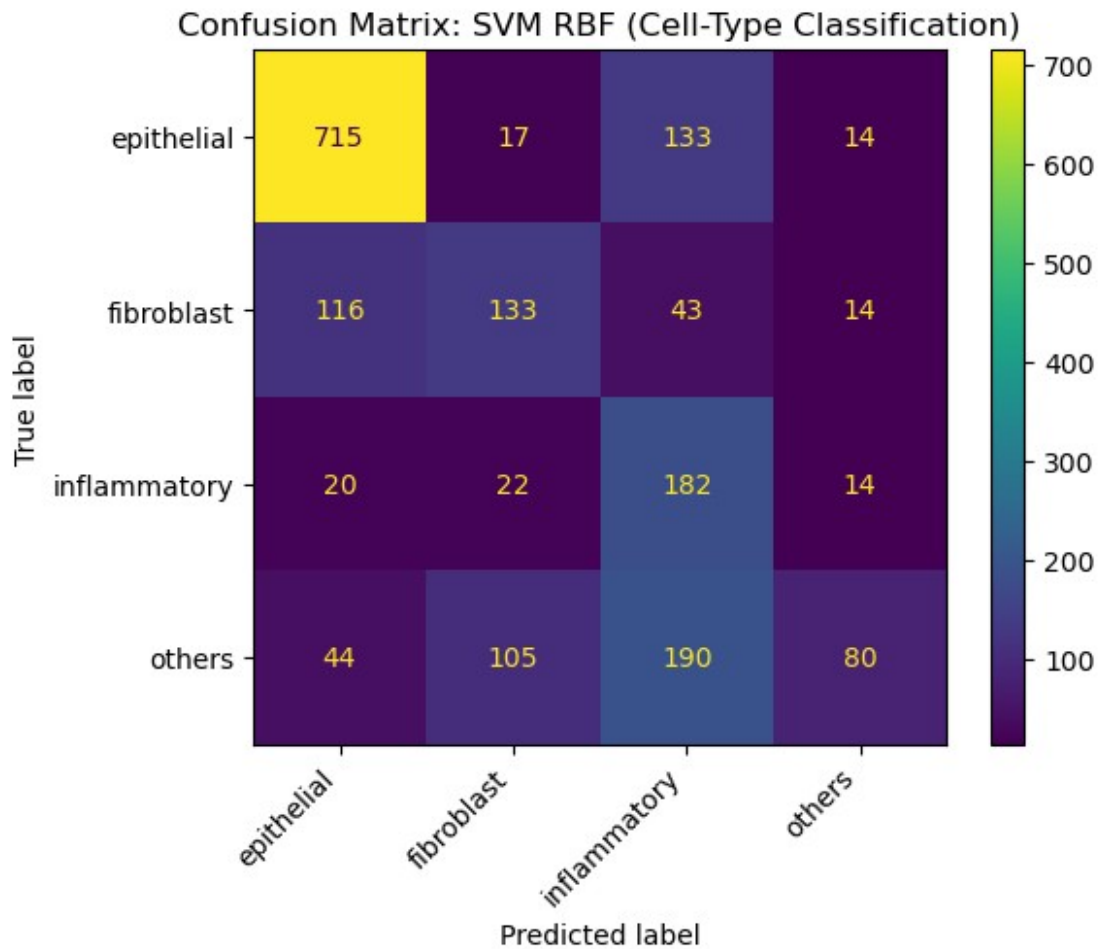
validation_results.append(svm_celltype_metrics)

Training SVM RBF for Cell-Type Classification...
=====
=====
Task: Cell-Type Classification
Model: SVM RBF
=====
=====
Accuracy:      0.6026
Macro F1:      0.5053
Weighted F1:   0.5876
Macro Precision: 0.5667
Macro Recall:  0.5509

Classification Report:

```

	precision	recall	f1-score	support
epithelial	0.80	0.81	0.81	879
fibroblast	0.48	0.43	0.46	306
inflammatory	0.33	0.76	0.46	238
others	0.66	0.19	0.30	419
accuracy			0.60	1842
macro avg	0.57	0.55	0.51	1842
weighted avg	0.65	0.60	0.59	1842



12.5 Random Forest for Cancer Classification

```
rf_cancer = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    class_weight="balanced",
    random_state=RANDOM_STATE,
    n_jobs=-1
)

rf_cancer, rf_cancer_metrics = train_evaluate_sklearn_model(
    model=rf_cancer,
    model_name="Random Forest",
    task_name="Cancer Classification",
    X_train=X_cancer_train_flat,
    y_train=y_cancer_train,
    X_val=X_cancer_val_flat,
    y_val=y_cancer_val,
    class_names=cancer_class_names
)
```

```
validation_results.append(rf_cancer_metrics)
```

```
Training Random Forest for Cancer Classification...
```

```
Task: Cancer Classification
```

```
Model: Random Forest
```

```
Accuracy: 0.8370
```

```
Macro F1: 0.8277
```

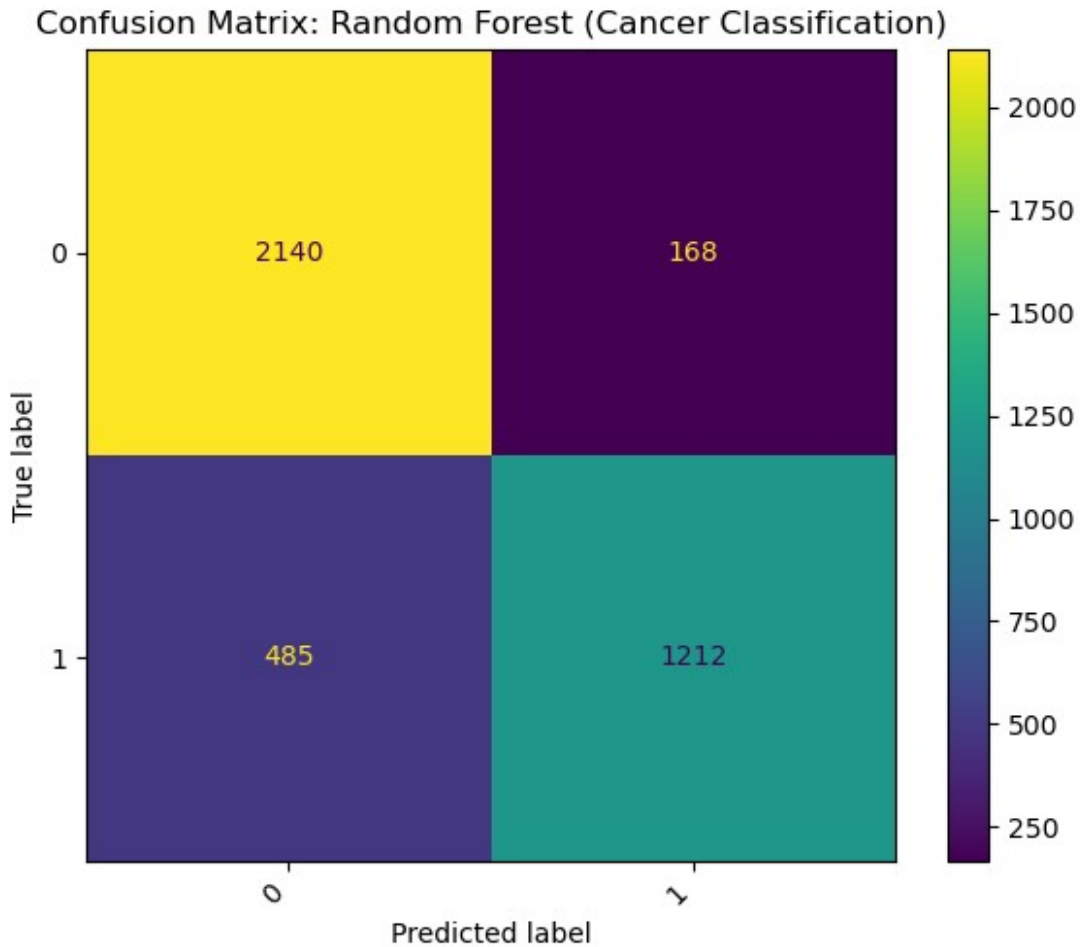
```
Weighted F1: 0.8338
```

```
Macro Precision: 0.8467
```

```
Macro Recall: 0.8207
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.82	0.93	0.87	2308
1	0.88	0.71	0.79	1697
accuracy			0.84	4005
macro avg	0.85	0.82	0.83	4005
weighted avg	0.84	0.84	0.83	4005



12.6 Random Forest for Cell-Type Classification

```
rf_celltype = RandomForestClassifier(  
    n_estimators=500,  
    max_depth=None,  
    min_samples_split=2,  
    min_samples_leaf=1,  
    max_features="sqrt",  
    class_weight="balanced",  
    random_state=RANDOM_STATE,  
    n_jobs=-1  
)  
  
rf_celltype, rf_celltype_metrics = train_evaluate_sklearn_model(  
    model=rf_celltype,  
    model_name="Random Forest",  
    task_name="Cell-Type Classification",  
    X_train=X_celltype_train_flat,  
    y_train=y_celltype_train,  
    X_val=X_celltype_val_flat,  
    y_val=y_celltype_val,
```

```
    class_names=celltype_class_names
)
```

```
validation_results.append(rf_celltype_metrics)
```

```
Training Random Forest for Cell-Type Classification...
```

```
=====
```

```
=====
```

```
Task: Cell-Type Classification
```

```
Model: Random Forest
```

```
=====
```

```
=====
```

```
Accuracy:      0.5744
```

```
Macro F1:      0.4345
```

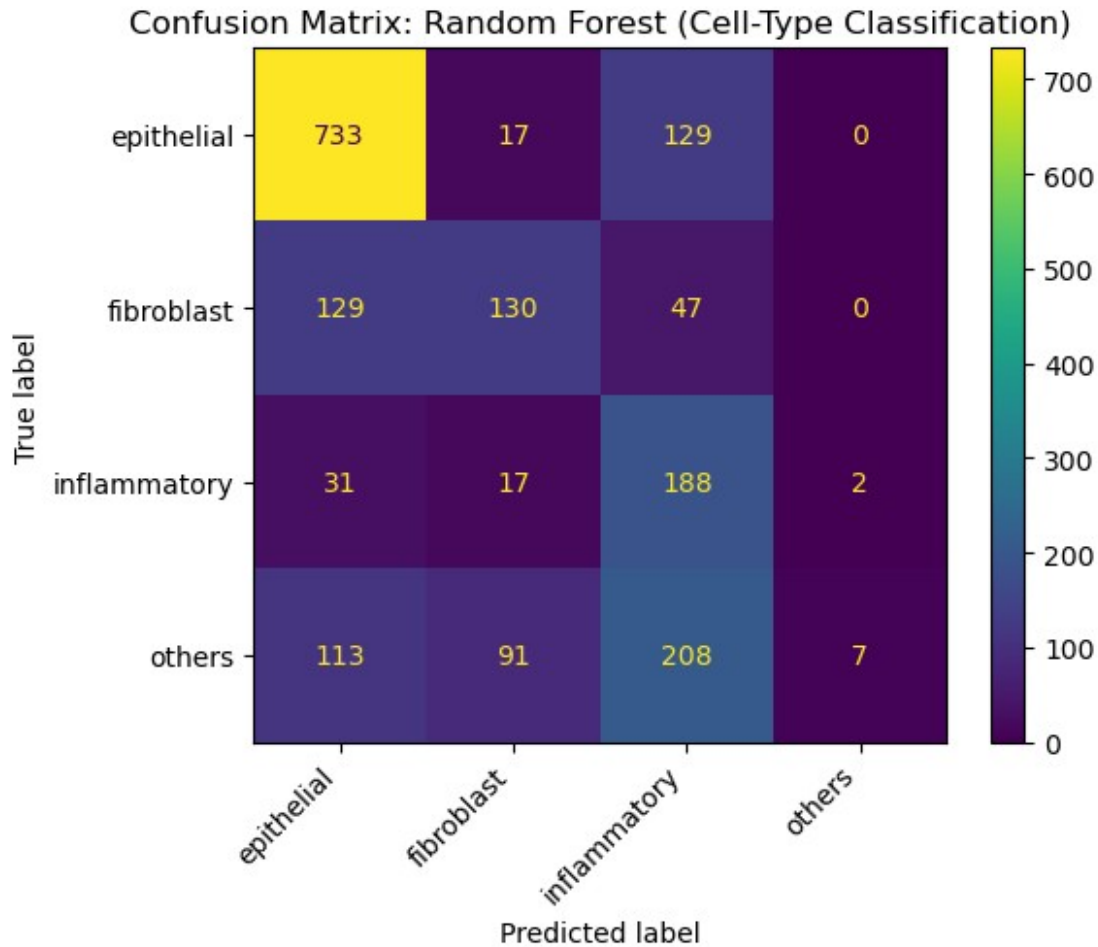
```
Weighted F1:   0.5155
```

```
Macro Precision: 0.5862
```

```
Macro Recall:  0.5163
```

```
Classification Report:
```

	precision	recall	f1-score	support
epithelial	0.73	0.83	0.78	879
fibroblast	0.51	0.42	0.46	306
inflammatory	0.33	0.79	0.46	238
others	0.78	0.02	0.03	419
accuracy			0.57	1842
macro avg	0.59	0.52	0.43	1842
weighted avg	0.65	0.57	0.52	1842



12.7 Classical Model Validation Summary

The validation results from the classical machine learning models are summarised below. The main comparison metric is `macro_f1` because it gives equal importance to each class. This is especially important for this assignment because the target performance thresholds are defined using macro-averaged F1-score.

```
classical_ml_results_df = pd.DataFrame(validation_results)
display(
    classical_ml_results_df.sort_values(
        by=["task", "macro_f1"],
        ascending=[True, False]
    )
)
```

	task	model	accuracy
macro_f1 \			
2	Cancer Classification	SVM RBF	0.859925 0.856775

4	Cancer Classification	Random Forest	0.836954	0.827703
0	Cancer Classification	Logistic Regression	0.773034	0.770972
3	Cell-Type Classification	SVM RBF	0.602606	0.505300
5	Cell-Type Classification	Random Forest	0.574376	0.434521
1	Cell-Type Classification	Logistic Regression	0.528773	0.432366

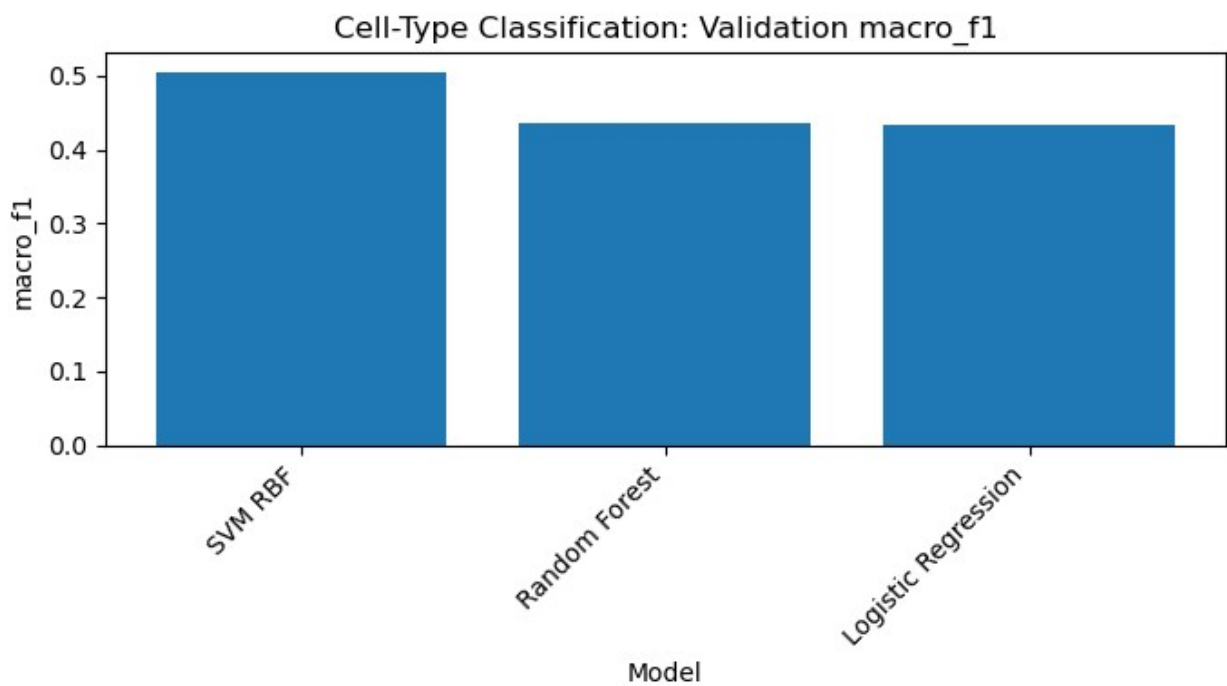
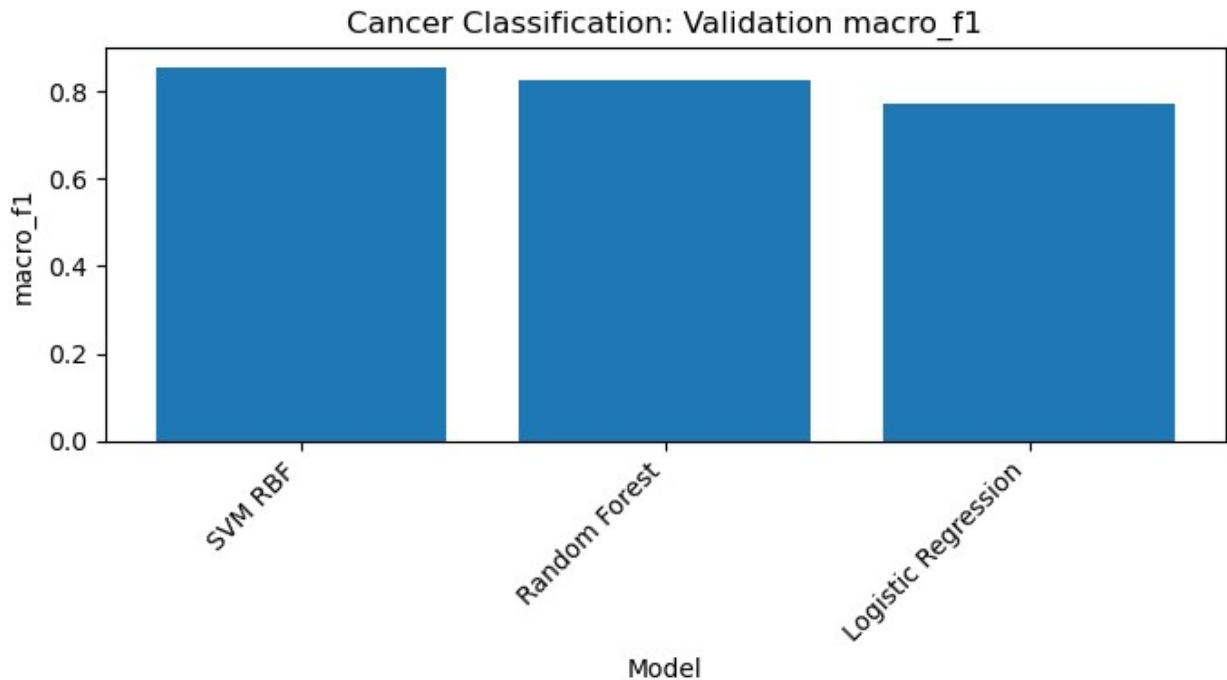
	weighted_f1	macro_precision	macro_recall
2	0.860015	0.856332	0.857250
4	0.833794	0.846749	0.820706
0	0.774287	0.770570	0.776634
3	0.587570	0.566720	0.550925
5	0.515536	0.586220	0.516340
1	0.525408	0.441512	0.453209

```
def plot_validation_results(results_df, task_name, metric="macro_f1"):
    """
    Plot validation metric for all models within a task.
    """
    task_df = results_df[results_df["task"] == task_name].copy()
    task_df = task_df.sort_values(by=metric, ascending=False)

    plt.figure(figsize=(7, 4))
    plt.bar(task_df["model"], task_df[metric])
    plt.title(f"{task_name}: Validation {metric}")
    plt.xlabel("Model")
    plt.ylabel(metric)
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()

plot_validation_results(
    classical_ml_results_df,
    task_name="Cancer Classification",
    metric="macro_f1"
)

plot_validation_results(
    classical_ml_results_df,
    task_name="Cell-Type Classification",
    metric="macro_f1"
)
```



12.8 Classical Model Interpretation

The classical models provide baseline evidence before training the CNN.

`LogisticRegression` shows whether a simple linear model can separate the classes using flattened image pixels. `SVM RBF` tests whether a more flexible margin-based classifier can improve performance in the same feature space. `RandomForestClassifier` provides a non-linear tree ensemble comparison. The confusion matrices should be used to identify where each

model struggles. For the cancer task, an important issue is whether cancerous images are missed. For the cell-type task, the key issue is which cell types are confused with each other. A limitation of all classical models in this section is that they use flattened image vectors. This means the spatial structure of the image is not explicitly preserved. The next section addresses this limitation by training a CNN using image-shaped tensors.

13. Deep Learning Model: Convolutional Neural Network

This section trains an improved Convolutional Neural Network (CNN) for both classification tasks. The CNN is used because the dataset consists of image patches. Unlike the classical models, the CNN keeps the image in tensor form and learns local spatial features using convolutional layers. This is appropriate for histology images because neighbouring pixels can form meaningful visual patterns such as texture, staining intensity, and cell structure.

13.1 CNN Model Rationale

The CNN is included as the image-specific deep learning model. The earlier EDA showed that the images are small, visually similar, and contain subtle colour and texture differences. Flattened classical models may lose spatial relationships between neighbouring pixels. A CNN addresses this limitation by applying convolutional filters to local regions of the image. The model is still kept relatively small because the images are only 27×27 pixels. This reduces unnecessary complexity while still allowing the network to learn local image patterns.

13.2 Improved CNN Architecture

```
class ImprovedCNN(nn.Module):
    """
    A CNN for 27 x 27 RGB histology image classification.
    """

    def __init__(self, num_classes):
        super(ImprovedCNN, self).__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU()
        )

        self.classifier = nn.Sequential(
```

```

        nn.Flatten(),
        nn.Linear(128 * 6 * 6, 128),
        nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(128, num_classes)
    )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

```

13.3 Class-Weighted Loss

Class-weighted cross-entropy loss is used to reduce the effect of class imbalance. This is important because the assignment evaluates macro-averaged F1-score. If minority classes are ignored, the model may still achieve reasonable accuracy but perform poorly on macro F1-score. The class weights are calculated from the training labels only.

```

def make_class_weights(y_train):
    """
    Create class weights for imbalanced classification tasks.
    """
    classes = np.unique(y_train)
    weights = compute_class_weight(
        class_weight="balanced",
        classes=classes,
        y=y_train
    )
    return torch.tensor(weights, dtype=torch.float32).to(device)

cancer_class_weights = make_class_weights(y_cancer_train)
celltype_class_weights = make_class_weights(y_celltype_train)

print("Cancer class weights:", cancer_class_weights)
print("Cell-type class weights:", celltype_class_weights)

Cancer class weights: tensor([0.7705, 1.4242])
Cell-type class weights: tensor([0.6830, 1.1762, 0.8296, 2.0828])

```

13.4 CNN for Cancer Classification

The improved CNN is first trained for the binary cancer classification task. The model is trained using class-weighted cross-entropy loss and the Adam optimiser. The best model is selected using validation macro F1-score rather than simply using the final epoch.

```

cnn_epochs = 30 if not QUICK_RUN else 5

```

```

cnn_cancer = ImprovedCNN(num_classes=len(cancer_class_names))
criterion_cancer = nn.CrossEntropyLoss(weight=cancer_class_weights)
optimizer_cancer = optim.Adam(
    cnn_cancer.parameters(),
    lr=0.001
)

cnn_cancer, cnn_cancer_history, cnn_cancer_metrics =
train_pytorch_model(
    model=cnn_cancer,
    train_loader=cancer_train_loader,
    val_loader=cancer_val_loader,
    criterion=criterion_cancer,
    optimizer=optimizer_cancer,
    device=device,
    num_epochs=cnn_epochs,
    model_name="Improved CNN",
    task_name="Cancer Classification",
    class_names=cancer_class_names
)

```

```
validation_results.append(cnn_cancer_metrics)
```

```

Epoch [1/30] Train Loss: 0.3283 Val Accuracy: 0.8604 Val Macro F1:
0.8587
Epoch [2/30] Train Loss: 0.2788 Val Accuracy: 0.8145 Val Macro F1:
0.8044
Epoch [3/30] Train Loss: 0.2449 Val Accuracy: 0.7793 Val Macro F1:
0.7744
Epoch [4/30] Train Loss: 0.2228 Val Accuracy: 0.7693 Val Macro F1:
0.7331
Epoch [5/30] Train Loss: 0.2085 Val Accuracy: 0.8797 Val Macro F1:
0.8764
Epoch [6/30] Train Loss: 0.1964 Val Accuracy: 0.8222 Val Macro F1:
0.8214
Epoch [7/30] Train Loss: 0.1669 Val Accuracy: 0.8312 Val Macro F1:
0.8311
Epoch [8/30] Train Loss: 0.1509 Val Accuracy: 0.8300 Val Macro F1:
0.8299
Epoch [9/30] Train Loss: 0.1358 Val Accuracy: 0.8005 Val Macro F1:
0.7919
Epoch [10/30] Train Loss: 0.1110 Val Accuracy: 0.8489 Val Macro F1:
0.8450
Epoch [11/30] Train Loss: 0.0962 Val Accuracy: 0.8370 Val Macro F1:
0.8357
Epoch [12/30] Train Loss: 0.0919 Val Accuracy: 0.8122 Val Macro F1:
0.7913
Epoch [13/30] Train Loss: 0.0650 Val Accuracy: 0.8529 Val Macro F1:

```

0.8439
Epoch [14/30] Train Loss: 0.0573 Val Accuracy: 0.8165 Val Macro F1: 0.8160
Epoch [15/30] Train Loss: 0.0555 Val Accuracy: 0.8479 Val Macro F1: 0.8405
Epoch [16/30] Train Loss: 0.0411 Val Accuracy: 0.8519 Val Macro F1: 0.8510
Epoch [17/30] Train Loss: 0.0359 Val Accuracy: 0.8577 Val Macro F1: 0.8550
Epoch [18/30] Train Loss: 0.0395 Val Accuracy: 0.7471 Val Macro F1: 0.7102
Epoch [19/30] Train Loss: 0.0307 Val Accuracy: 0.8277 Val Macro F1: 0.8213
Epoch [20/30] Train Loss: 0.0238 Val Accuracy: 0.7863 Val Macro F1: 0.7569
Epoch [21/30] Train Loss: 0.0309 Val Accuracy: 0.8529 Val Macro F1: 0.8438
Epoch [22/30] Train Loss: 0.0269 Val Accuracy: 0.8307 Val Macro F1: 0.8276
Epoch [23/30] Train Loss: 0.0236 Val Accuracy: 0.8594 Val Macro F1: 0.8572
Epoch [24/30] Train Loss: 0.0326 Val Accuracy: 0.8642 Val Macro F1: 0.8625
Epoch [25/30] Train Loss: 0.0170 Val Accuracy: 0.8280 Val Macro F1: 0.8270
Epoch [26/30] Train Loss: 0.0180 Val Accuracy: 0.8357 Val Macro F1: 0.8353
Epoch [27/30] Train Loss: 0.0239 Val Accuracy: 0.8474 Val Macro F1: 0.8383
Epoch [28/30] Train Loss: 0.0304 Val Accuracy: 0.8414 Val Macro F1: 0.8406
Epoch [29/30] Train Loss: 0.0224 Val Accuracy: 0.8652 Val Macro F1: 0.8613
Epoch [30/30] Train Loss: 0.0098 Val Accuracy: 0.8674 Val Macro F1: 0.8634

Best validation epoch: 5
Best validation macro F1: 0.8764

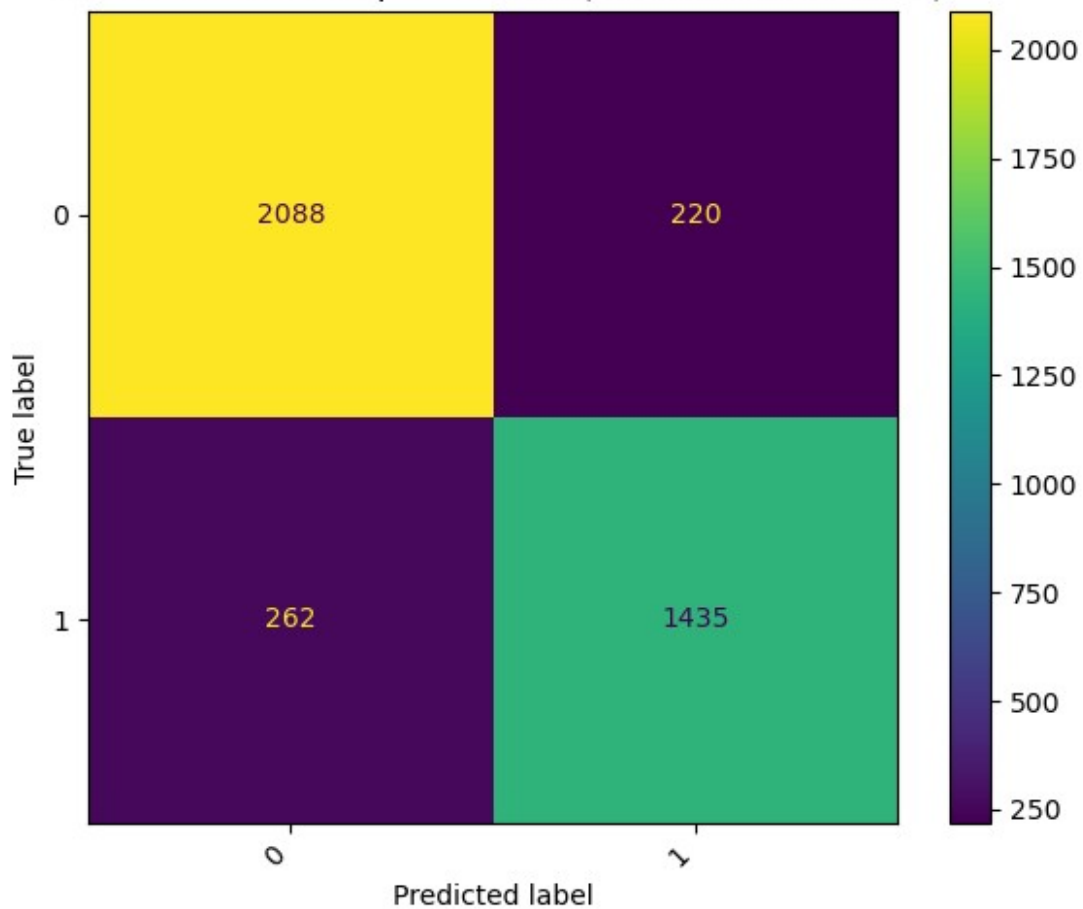
=====
=====
Task: Cancer Classification
Model: Improved CNN
=====

=====
Accuracy: 0.8797
Macro F1: 0.8764
Weighted F1: 0.8794
Macro Precision: 0.8778
Macro Recall: 0.8751

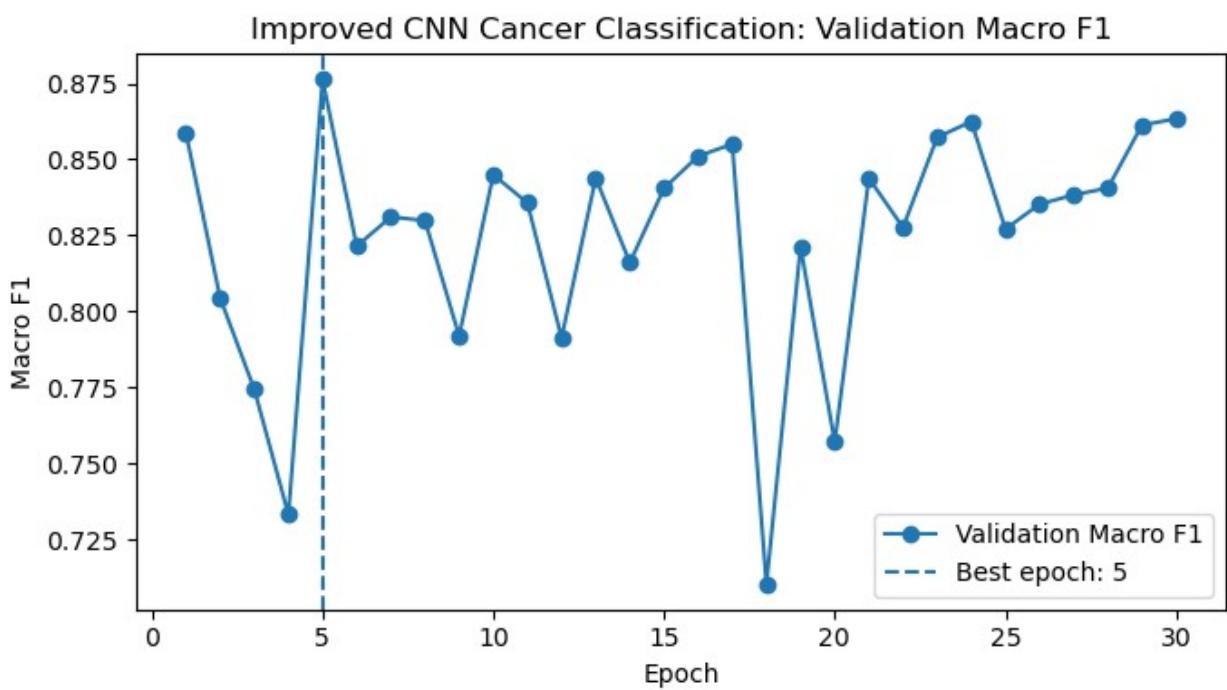
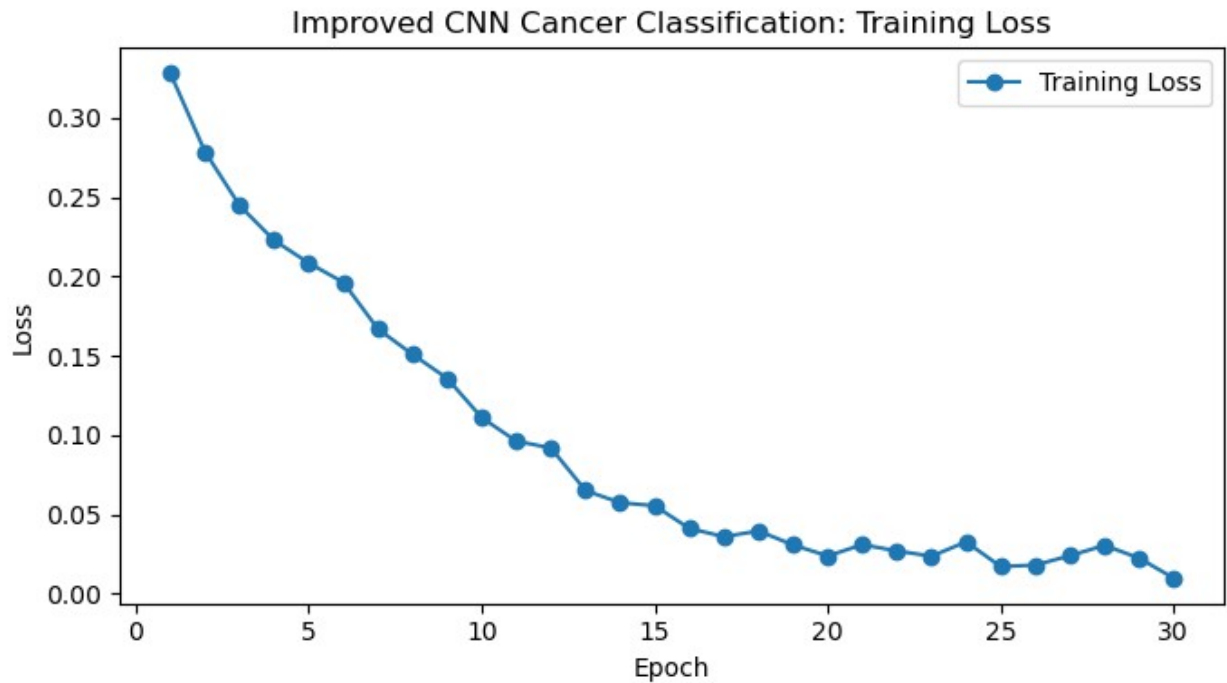
Classification Report:

	precision	recall	f1-score	support
0	0.89	0.90	0.90	2308
1	0.87	0.85	0.86	1697
accuracy			0.88	4005
macro avg	0.88	0.88	0.88	4005
weighted avg	0.88	0.88	0.88	4005

Confusion Matrix: Improved CNN (Cancer Classification)



```
plot_training_history(  
    cnn_cancer_history,  
    title="Improved CNN Cancer Classification"  
)
```



Best epoch for Improved CNN Cancer Classification: 5
Best validation macro F1 for Improved CNN Cancer Classification: 0.8764

13.5 CNN for Cell-Type Classification

The improved CNN is then trained for the multi-class cell-type classification task. This task is expected to be more difficult because the model must distinguish between four visually similar cell categories. Class-weighted loss is used to encourage the model to pay more attention to minority classes.

```
cnn_celltype = ImprovedCNN(num_classes=len(celltype_class_names))

criterion_celltype =
nn.CrossEntropyLoss(weight=celltype_class_weights)

optimizer_celltype = optim.Adam(
    cnn_celltype.parameters(),
    lr=0.001
)

cnn_celltype, cnn_celltype_history, cnn_celltype_metrics =
train_pytorch_model(
    model=cnn_celltype,
    train_loader=celltype_train_loader,
    val_loader=celltype_val_loader,
    criterion=criterion_celltype,
    optimizer=optimizer_celltype,
    device=device,
    num_epochs=cnn_epochs,
    model_name="Improved CNN",
    task_name="Cell-Type Classification",
    class_names=celltype_class_names
)

validation_results.append(cnn_celltype_metrics)
```

Epoch [1/30] Train Loss: 0.8244 Val Accuracy: 0.3925 Val Macro F1: 0.3461
Epoch [2/30] Train Loss: 0.6969 Val Accuracy: 0.5847 Val Macro F1: 0.4999
Epoch [3/30] Train Loss: 0.6412 Val Accuracy: 0.5489 Val Macro F1: 0.5082
Epoch [4/30] Train Loss: 0.6003 Val Accuracy: 0.5836 Val Macro F1: 0.4311
Epoch [5/30] Train Loss: 0.5437 Val Accuracy: 0.5608 Val Macro F1: 0.4390
Epoch [6/30] Train Loss: 0.5338 Val Accuracy: 0.6254 Val Macro F1: 0.5536
Epoch [7/30] Train Loss: 0.4986 Val Accuracy: 0.6460 Val Macro F1: 0.5862
Epoch [8/30] Train Loss: 0.4541 Val Accuracy: 0.6460 Val Macro F1: 0.6147
Epoch [9/30] Train Loss: 0.4299 Val Accuracy: 0.6053 Val Macro F1:

0.5242
Epoch [10/30] Train Loss: 0.3922 Val Accuracy: 0.5407 Val Macro F1:
0.4761
Epoch [11/30] Train Loss: 0.3534 Val Accuracy: 0.6868 Val Macro F1:
0.6380
Epoch [12/30] Train Loss: 0.2969 Val Accuracy: 0.4463 Val Macro F1:
0.3346
Epoch [13/30] Train Loss: 0.2881 Val Accuracy: 0.6086 Val Macro F1:
0.4581
Epoch [14/30] Train Loss: 0.3107 Val Accuracy: 0.6672 Val Macro F1:
0.6264
Epoch [15/30] Train Loss: 0.2213 Val Accuracy: 0.6357 Val Macro F1:
0.5614
Epoch [16/30] Train Loss: 0.1999 Val Accuracy: 0.5961 Val Macro F1:
0.4720
Epoch [17/30] Train Loss: 0.1903 Val Accuracy: 0.4826 Val Macro F1:
0.4034
Epoch [18/30] Train Loss: 0.1596 Val Accuracy: 0.6086 Val Macro F1:
0.5254
Epoch [19/30] Train Loss: 0.1115 Val Accuracy: 0.6661 Val Macro F1:
0.5756
Epoch [20/30] Train Loss: 0.1066 Val Accuracy: 0.6813 Val Macro F1:
0.6239
Epoch [21/30] Train Loss: 0.0881 Val Accuracy: 0.6661 Val Macro F1:
0.5736
Epoch [22/30] Train Loss: 0.0831 Val Accuracy: 0.6650 Val Macro F1:
0.6118
Epoch [23/30] Train Loss: 0.1072 Val Accuracy: 0.6010 Val Macro F1:
0.5376
Epoch [24/30] Train Loss: 0.1269 Val Accuracy: 0.5869 Val Macro F1:
0.4599
Epoch [25/30] Train Loss: 0.0827 Val Accuracy: 0.6553 Val Macro F1:
0.5898
Epoch [26/30] Train Loss: 0.0870 Val Accuracy: 0.6531 Val Macro F1:
0.5959
Epoch [27/30] Train Loss: 0.0507 Val Accuracy: 0.5814 Val Macro F1:
0.5388
Epoch [28/30] Train Loss: 0.0656 Val Accuracy: 0.6292 Val Macro F1:
0.5723
Epoch [29/30] Train Loss: 0.0474 Val Accuracy: 0.6498 Val Macro F1:
0.5387
Epoch [30/30] Train Loss: 0.1249 Val Accuracy: 0.6466 Val Macro F1:
0.5507

Best validation epoch: 11
Best validation macro F1: 0.638

=====

=====
Task: Cell-Type Classification

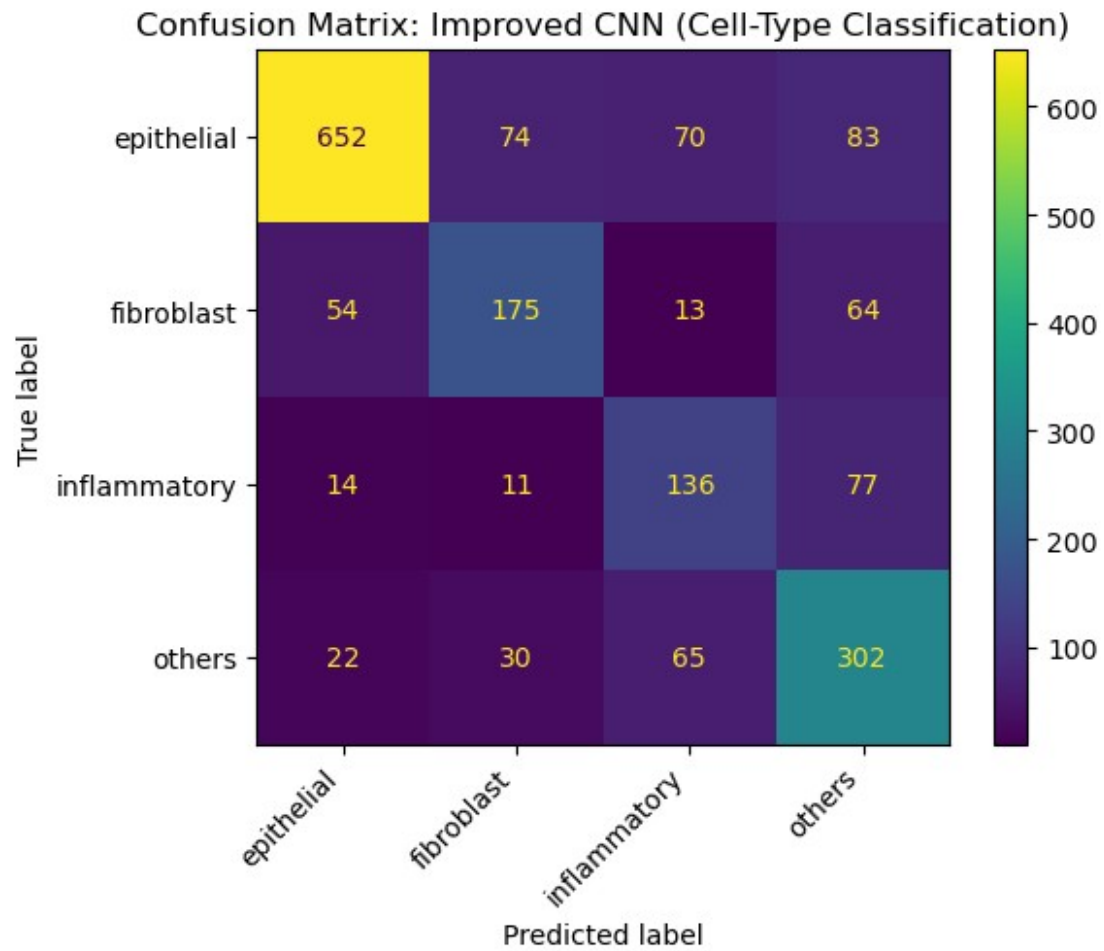
Model: Improved CNN

=====

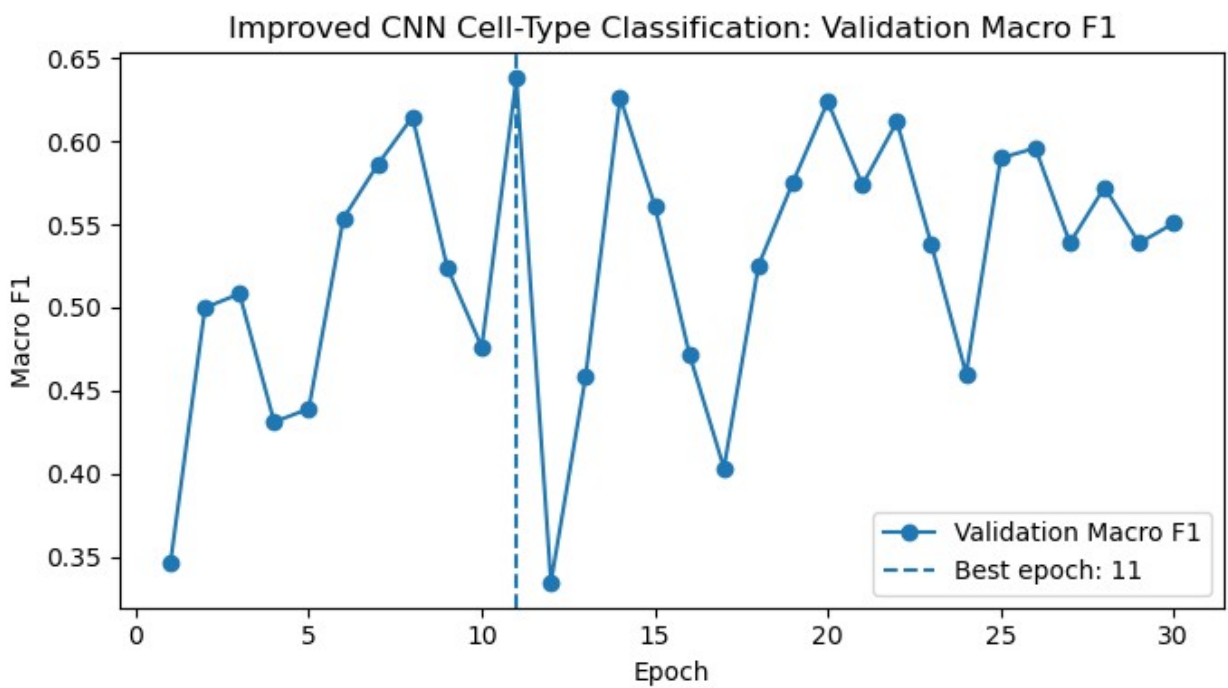
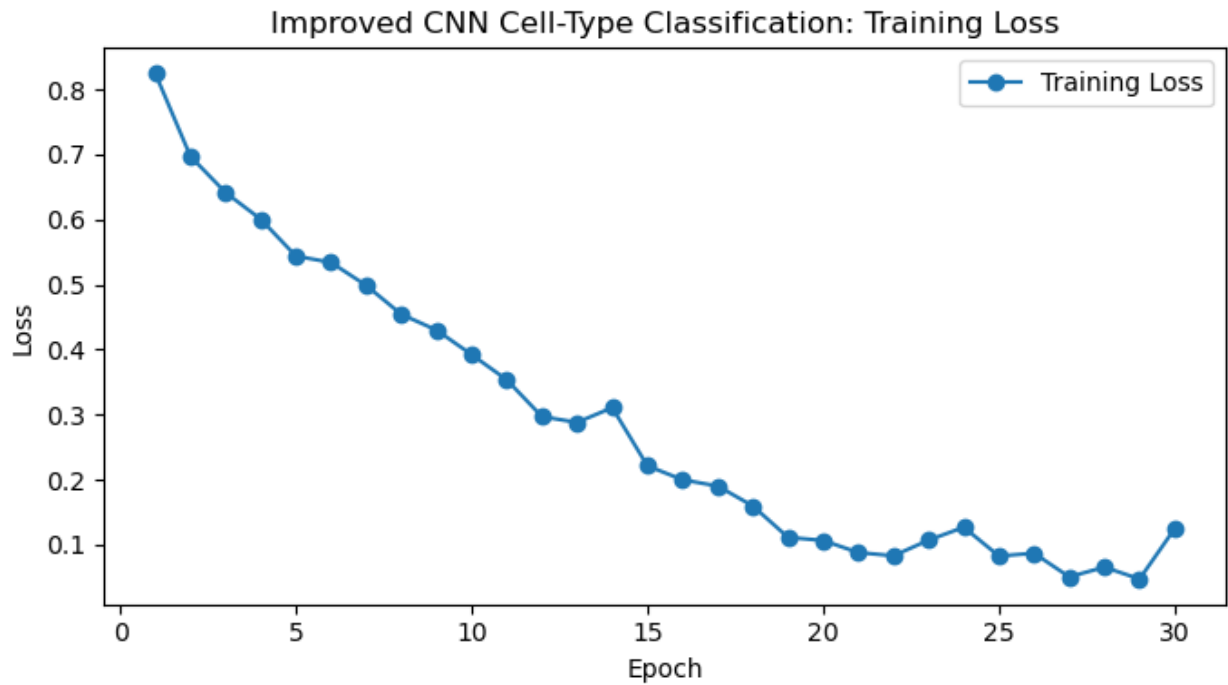
Accuracy: 0.6868
Macro F1: 0.6380
Weighted F1: 0.6941
Macro Precision: 0.6338
Macro Recall: 0.6515

Classification Report:

	precision	recall	f1-score	support
epithelial	0.88	0.74	0.80	879
fibroblast	0.60	0.57	0.59	306
inflammatory	0.48	0.57	0.52	238
others	0.57	0.72	0.64	419
accuracy			0.69	1842
macro avg	0.63	0.65	0.64	1842
weighted avg	0.71	0.69	0.69	1842



```
plot_training_history(  
    cnn_celltype_history,  
    title="Improved CNN Cell-Type Classification"  
)
```



Best epoch for Improved CNN Cell-Type Classification: 11
Best validation macro F1 for Improved CNN Cell-Type Classification: 0.6380

13.6 CNN Validation Summary

The CNN validation results are added to the existing validation results table so that all selected models can be compared together.

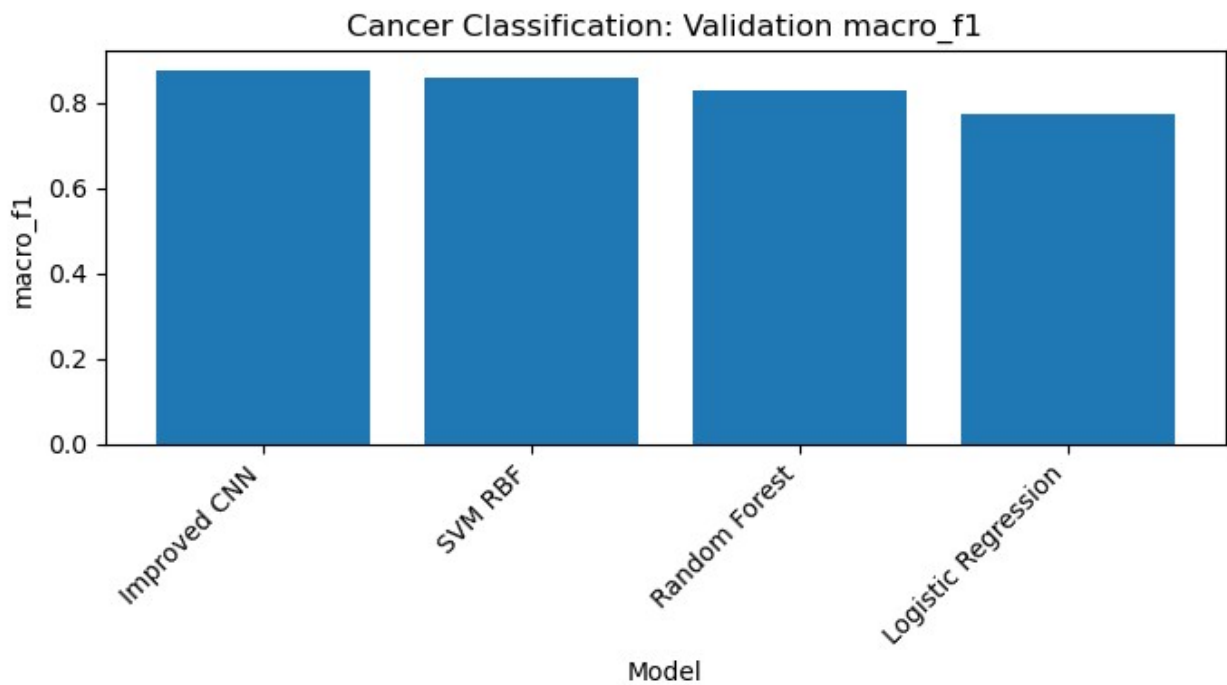
```
all_validation_results_df = pd.DataFrame(validation_results)

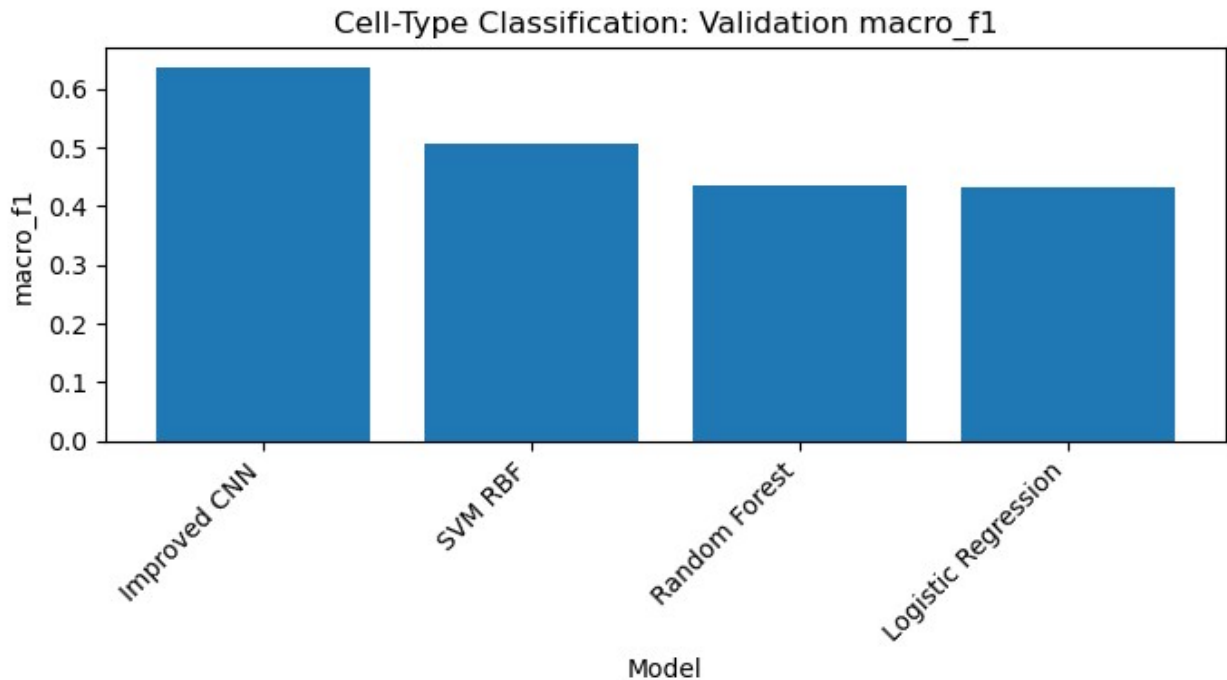
display(
    all_validation_results_df.sort_values(
        by=["task", "macro_f1"],
        ascending=[True, False]
    )
)
```

	task	model	accuracy	
macro_f1 \				
6	Cancer Classification	Improved CNN	0.879650	0.876364
2	Cancer Classification	SVM RBF	0.859925	0.856775
4	Cancer Classification	Random Forest	0.836954	0.827703
0	Cancer Classification	Logistic Regression	0.773034	0.770972
7	Cell-Type Classification	Improved CNN	0.686754	0.637979
3	Cell-Type Classification	SVM RBF	0.602606	0.505300
5	Cell-Type Classification	Random Forest	0.574376	0.434521
1	Cell-Type Classification	Logistic Regression	0.528773	0.432366

	weighted_f1	macro_precision	macro_recall	best_epoch
best_val_macro_f1				
6	0.879439	0.877790	0.875145	5.0
0.876364				
2	0.860015	0.856332	0.857250	NaN
NaN				
4	0.833794	0.846749	0.820706	NaN
NaN				
0	0.774287	0.770570	0.776634	NaN
NaN				
7	0.694149	0.633793	0.651460	11.0
0.637979				
3	0.587570	0.566720	0.550925	NaN
NaN				
5	0.515536	0.586220	0.516340	NaN
NaN				
1	0.525408	0.441512	0.453209	NaN
NaN				

```
plot_validation_results(  
    all_validation_results_df,  
    task_name="Cancer Classification",  
    metric="macro_f1"  
)  
  
plot_validation_results(  
    all_validation_results_df,  
    task_name="Cell-Type Classification",  
    metric="macro_f1"  
)
```





13.7 CNN Interpretation

The CNN results are interpreted using both the training curves and validation metrics. Although the training loss may decrease across epochs, validation macro F1 does not always improve at the same rate. This shows that lower training loss does not automatically mean better generalisation. For this reason, the CNN is selected using the best validation macro F1-score rather than simply the final epoch. The confusion matrix should also be inspected to determine whether the CNN improves performance across all classes or mainly benefits the majority class. If the CNN performs better than the classical models, this suggests that preserving spatial image structure is useful for the task. If it does not outperform the classical models, possible explanations include limited training data, class imbalance, small image size, or the need for further hyperparameter tuning and data augmentation.

14. Hyperparameter Tuning

The initial model comparison showed that the improved CNN is the strongest model for both prediction tasks. The cell-type CNN exceeded the required macro-averaged F1-score threshold of 0.60, but the cancer CNN remained below the required threshold of 0.90. Therefore, this section focuses on improving the cancer CNN. The goal is not to randomly test many settings, but to perform a controlled tuning experiment using hyperparameters that directly affect neural network training.

The following settings are explored:

- learning rate
- weight decay
- class-weighted loss

The validation set is used for tuning. The test set remains unseen and is not used until the final evaluation stage.

14.1 Tuning Setup

The cancer CNN is tuned first because it is close to the required macro F1-score threshold. The previous cancer CNN result showed that threshold tuning only produced a small improvement. This suggests that the main limitation is not the decision threshold, but the learned representation and generalisation ability of the CNN. The following tuning experiments compare different training settings while keeping the same patient-level train/validation split.

```
cnn_tuning_results = []
cnn_tuning_models = {}
cnn_tuning_histories = {}
```

14.2 CNN Tuning Helper Function

```
def train_cancer_cnn_experiment(
    experiment_name,
    learning_rate=0.001,
    weight_decay=0.0,
    use_class_weights=True,
    num_epochs=None
):
    """
    Train one cancer CNN experiment with specified hyperparameters.
    """
    if num_epochs is None:
        num_epochs = 30 if not QUICK_RUN else 5

    print("=" * 80)
    print("Experiment:", experiment_name)
    print("Learning rate:", learning_rate)
    print("Weight decay:", weight_decay)
    print("Use class weights:", use_class_weights)
    print("Epochs:", num_epochs)
    print("=" * 80)

    model = ImprovedCNN(num_classes=len(cancer_class_names))

    if use_class_weights:
        criterion = nn.CrossEntropyLoss(weight=cancer_class_weights)
    else:
        criterion = nn.CrossEntropyLoss()

    optimizer = optim.Adam(
        model.parameters(),
        lr=learning_rate,
        weight_decay=weight_decay
    )
```

```

model, history, metrics = train_pytorch_model(
    model=model,
    train_loader=cancer_train_loader,
    val_loader=cancer_val_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    num_epochs=num_epochs,
    model_name=experiment_name,
    task_name="Cancer Classification",
    class_names=cancer_class_names
)

metrics["learning_rate"] = learning_rate
metrics["weight_decay"] = weight_decay
metrics["use_class_weights"] = use_class_weights

cnn_tuning_results.append(metrics)
cnn_tuning_models[experiment_name] = model
cnn_tuning_histories[experiment_name] = history

return model, history, metrics

```

14.3 Learning Rate and Weight Decay Experiments

The learning rate controls the size of each optimiser update. If the learning rate is too large, the model may move past better solutions or show unstable validation performance. If it is too small, training may improve too slowly. Weight decay is also tested because it can reduce overfitting by discouraging overly large weights. The experiments below focus on the cancer CNN because this is the task that still needs to reach the required macro F1-score threshold.

```

#experiment 1: smaller learning rate, no class weights
cnn_cancer_lr0003_no_weights, history_lr0003_no_weights,
metrics_lr0003_no_weights = train_cancer_cnn_experiment(
    experiment_name="CNN lr=0.0003 no class weights",
    learning_rate=0.0003,
    weight_decay=1e-4,
    use_class_weights=False,
    num_epochs=30 if not QUICK_RUN else 5
)

```

```

=====
=====
Experiment: CNN lr=0.0003 no class weights
Learning rate: 0.0003
Weight decay: 0.0001
Use class weights: False
Epochs: 30

```

```
=====
=====
Epoch [1/30] Train Loss: 0.3131 Val Accuracy: 0.8529 Val Macro F1:
0.8510
Epoch [2/30] Train Loss: 0.2512 Val Accuracy: 0.8335 Val Macro F1:
0.8260
Epoch [3/30] Train Loss: 0.2313 Val Accuracy: 0.8589 Val Macro F1:
0.8537
Epoch [4/30] Train Loss: 0.2083 Val Accuracy: 0.8175 Val Macro F1:
0.8040
Epoch [5/30] Train Loss: 0.1790 Val Accuracy: 0.8667 Val Macro F1:
0.8655
Epoch [6/30] Train Loss: 0.1539 Val Accuracy: 0.8350 Val Macro F1:
0.8348
Epoch [7/30] Train Loss: 0.1330 Val Accuracy: 0.8604 Val Macro F1:
0.8587
Epoch [8/30] Train Loss: 0.1168 Val Accuracy: 0.8220 Val Macro F1:
0.8219
Epoch [9/30] Train Loss: 0.0886 Val Accuracy: 0.8322 Val Macro F1:
0.8182
Epoch [10/30] Train Loss: 0.0707 Val Accuracy: 0.8355 Val Macro F1:
0.8278
Epoch [11/30] Train Loss: 0.0512 Val Accuracy: 0.8409 Val Macro F1:
0.8406
Epoch [12/30] Train Loss: 0.0472 Val Accuracy: 0.8407 Val Macro F1:
0.8393
Epoch [13/30] Train Loss: 0.0379 Val Accuracy: 0.8337 Val Macro F1:
0.8333
Epoch [14/30] Train Loss: 0.0331 Val Accuracy: 0.8589 Val Macro F1:
0.8550
Epoch [15/30] Train Loss: 0.0347 Val Accuracy: 0.8424 Val Macro F1:
0.8355
Epoch [16/30] Train Loss: 0.0246 Val Accuracy: 0.8320 Val Macro F1:
0.8315
Epoch [17/30] Train Loss: 0.0171 Val Accuracy: 0.7810 Val Macro F1:
0.7808
Epoch [18/30] Train Loss: 0.0180 Val Accuracy: 0.8357 Val Macro F1:
0.8281
Epoch [19/30] Train Loss: 0.0175 Val Accuracy: 0.8357 Val Macro F1:
0.8255
Epoch [20/30] Train Loss: 0.0274 Val Accuracy: 0.7743 Val Macro F1:
0.7416
Epoch [21/30] Train Loss: 0.0214 Val Accuracy: 0.8097 Val Macro F1:
0.7925
Epoch [22/30] Train Loss: 0.0214 Val Accuracy: 0.8569 Val Macro F1:
0.8544
Epoch [23/30] Train Loss: 0.0204 Val Accuracy: 0.8529 Val Macro F1:
0.8509
Epoch [24/30] Train Loss: 0.0226 Val Accuracy: 0.8602 Val Macro F1:
```

0.8571
Epoch [25/30] Train Loss: 0.0153 Val Accuracy: 0.8317 Val Macro F1:
0.8260
Epoch [26/30] Train Loss: 0.0147 Val Accuracy: 0.8617 Val Macro F1:
0.8596
Epoch [27/30] Train Loss: 0.0074 Val Accuracy: 0.8442 Val Macro F1:
0.8425
Epoch [28/30] Train Loss: 0.0128 Val Accuracy: 0.8409 Val Macro F1:
0.8400
Epoch [29/30] Train Loss: 0.0053 Val Accuracy: 0.8462 Val Macro F1:
0.8419
Epoch [30/30] Train Loss: 0.0160 Val Accuracy: 0.8035 Val Macro F1:
0.7836

Best validation epoch: 5
Best validation macro F1: 0.8655

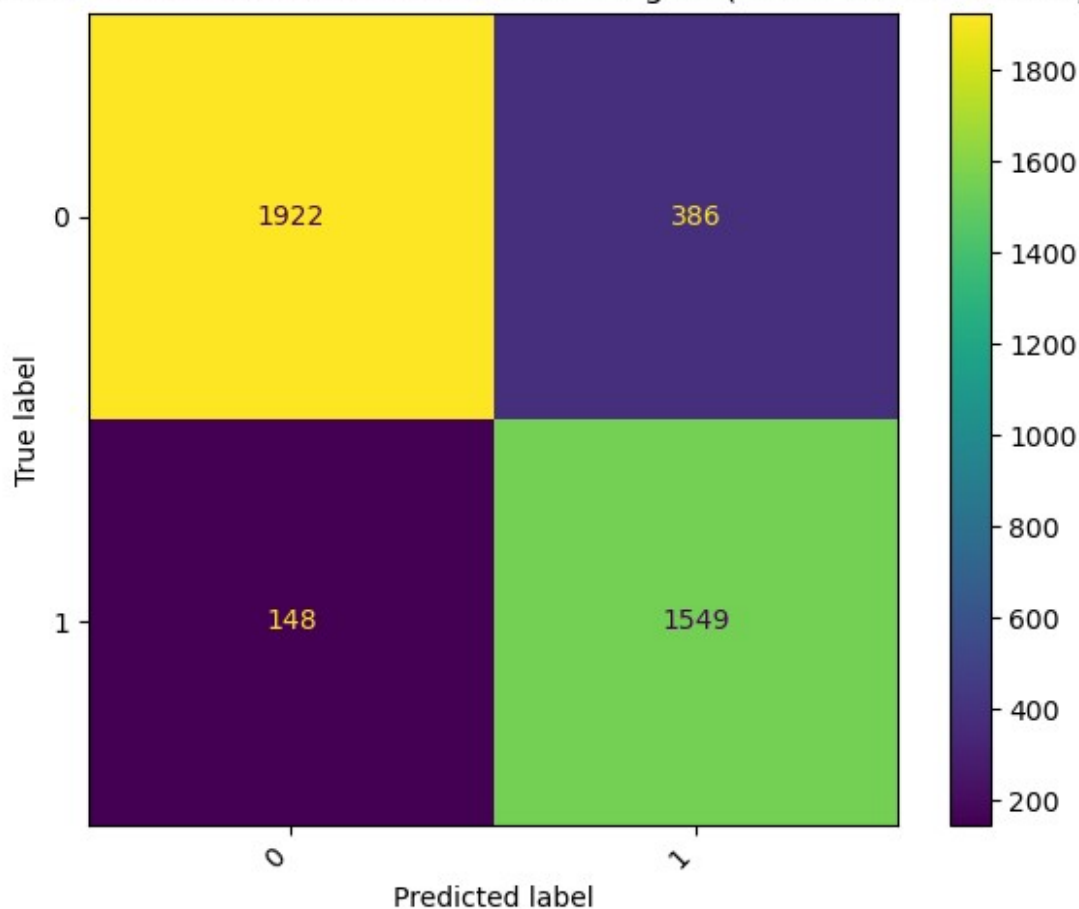
=====
=====
Task: Cancer Classification
Model: CNN lr=0.0003 no class weights
=====

=====
Accuracy: 0.8667
Macro F1: 0.8655
Weighted F1: 0.8674
Macro Precision: 0.8645
Macro Recall: 0.8728

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.83	0.88	2308
1	0.80	0.91	0.85	1697
accuracy			0.87	4005
macro avg	0.86	0.87	0.87	4005
weighted avg	0.87	0.87	0.87	4005

Confusion Matrix: CNN lr=0.0003 no class weights (Cancer Classification)



#experiment 2: smaller learning rate, with class weights

```
cnn_cancer_lr0003_weights, history_lr0003_weights,
metrics_lr0003_weights = train_cancer_cnn_experiment(
    experiment_name="CNN lr=0.0003 class weights",
    learning_rate=0.0003,
    weight_decay=1e-4,
    use_class_weights=True,
    num_epochs=30 if not QUICK_RUN else 5
)
```

```
=====
=====
Experiment: CNN lr=0.0003 class weights
Learning rate: 0.0003
Weight decay: 0.0001
Use class weights: True
Epochs: 30
=====
```

```
=====
Epoch [1/30] Train Loss: 0.3214 Val Accuracy: 0.8202 Val Macro F1:
```

0.8179
Epoch [2/30] Train Loss: 0.2690 Val Accuracy: 0.7366 Val Macro F1: 0.7339
Epoch [3/30] Train Loss: 0.2381 Val Accuracy: 0.8132 Val Macro F1: 0.7934
Epoch [4/30] Train Loss: 0.2220 Val Accuracy: 0.8265 Val Macro F1: 0.8264
Epoch [5/30] Train Loss: 0.1853 Val Accuracy: 0.8434 Val Macro F1: 0.8409
Epoch [6/30] Train Loss: 0.1658 Val Accuracy: 0.8494 Val Macro F1: 0.8389
Epoch [7/30] Train Loss: 0.1420 Val Accuracy: 0.8674 Val Macro F1: 0.8641
Epoch [8/30] Train Loss: 0.1180 Val Accuracy: 0.8442 Val Macro F1: 0.8403
Epoch [9/30] Train Loss: 0.0922 Val Accuracy: 0.8754 Val Macro F1: 0.8727
Epoch [10/30] Train Loss: 0.0770 Val Accuracy: 0.8699 Val Macro F1: 0.8678
Epoch [11/30] Train Loss: 0.0578 Val Accuracy: 0.8459 Val Macro F1: 0.8450
Epoch [12/30] Train Loss: 0.0437 Val Accuracy: 0.8507 Val Macro F1: 0.8493
Epoch [13/30] Train Loss: 0.0457 Val Accuracy: 0.7850 Val Macro F1: 0.7560
Epoch [14/30] Train Loss: 0.0308 Val Accuracy: 0.8589 Val Macro F1: 0.8574
Epoch [15/30] Train Loss: 0.0326 Val Accuracy: 0.8400 Val Macro F1: 0.8393
Epoch [16/30] Train Loss: 0.0299 Val Accuracy: 0.8222 Val Macro F1: 0.8083
Epoch [17/30] Train Loss: 0.0205 Val Accuracy: 0.8385 Val Macro F1: 0.8363
Epoch [18/30] Train Loss: 0.0225 Val Accuracy: 0.8599 Val Macro F1: 0.8570
Epoch [19/30] Train Loss: 0.0268 Val Accuracy: 0.8207 Val Macro F1: 0.8151
Epoch [20/30] Train Loss: 0.0205 Val Accuracy: 0.8522 Val Macro F1: 0.8495
Epoch [21/30] Train Loss: 0.0159 Val Accuracy: 0.8235 Val Macro F1: 0.8101
Epoch [22/30] Train Loss: 0.0278 Val Accuracy: 0.8280 Val Macro F1: 0.8279
Epoch [23/30] Train Loss: 0.0247 Val Accuracy: 0.8407 Val Macro F1: 0.8388
Epoch [24/30] Train Loss: 0.0170 Val Accuracy: 0.8165 Val Macro F1: 0.8072
Epoch [25/30] Train Loss: 0.0115 Val Accuracy: 0.8582 Val Macro F1: 0.8544

Epoch [26/30] Train Loss: 0.0124 Val Accuracy: 0.8230 Val Macro F1: 0.8228
Epoch [27/30] Train Loss: 0.0106 Val Accuracy: 0.8217 Val Macro F1: 0.8215
Epoch [28/30] Train Loss: 0.0161 Val Accuracy: 0.8467 Val Macro F1: 0.8455
Epoch [29/30] Train Loss: 0.0286 Val Accuracy: 0.8589 Val Macro F1: 0.8527
Epoch [30/30] Train Loss: 0.0241 Val Accuracy: 0.8402 Val Macro F1: 0.8354

Best validation epoch: 9
Best validation macro F1: 0.8727

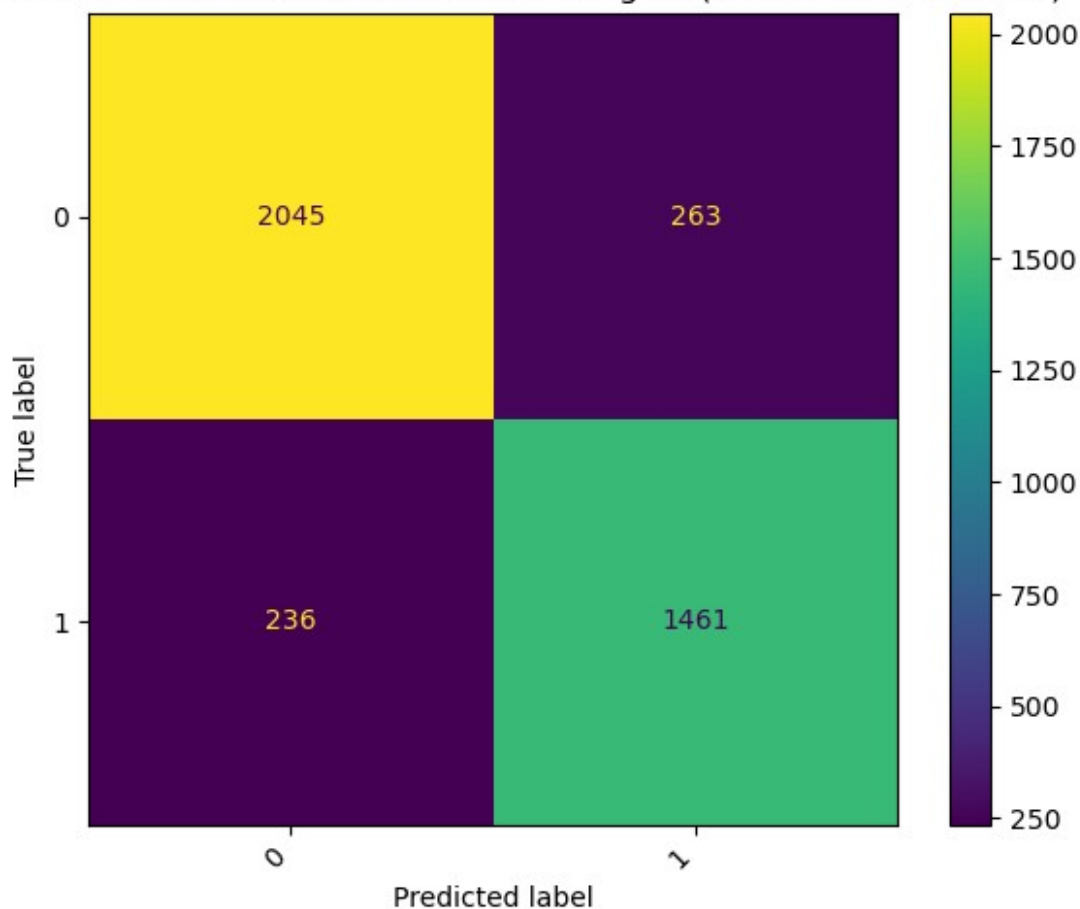
=====
=====
Task: Cancer Classification
Model: CNN lr=0.0003 class weights
=====

=====
Accuracy: 0.8754
Macro F1: 0.8727
Weighted F1: 0.8755
Macro Precision: 0.8720
Macro Recall: 0.8735

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.89	0.89	2308
1	0.85	0.86	0.85	1697
accuracy			0.88	4005
macro avg	0.87	0.87	0.87	4005
weighted avg	0.88	0.88	0.88	4005

Confusion Matrix: CNN lr=0.0003 class weights (Cancer Classification)



```
#experiment 3: very small learning rate, no class weights
cnn_cancer_lr0001_no_weights, history_lr0001_no_weights,
metrics_lr0001_no_weights = train_cancer_cnn_experiment(
    experiment_name="CNN lr=0.0001 no class weights",
    learning_rate=0.0001,
    weight_decay=1e-4,
    use_class_weights=False,
    num_epochs=30 if not QUICK_RUN else 5
)
```

```
=====
=====
Experiment: CNN lr=0.0001 no class weights
Learning rate: 0.0001
Weight decay: 0.0001
Use class weights: False
Epochs: 30
=====
```

```
=====
Epoch [1/30] Train Loss: 0.3114 Val Accuracy: 0.8414 Val Macro F1:
```

0.8314
Epoch [2/30] Train Loss: 0.2549 Val Accuracy: 0.8444 Val Macro F1: 0.8367
Epoch [3/30] Train Loss: 0.2342 Val Accuracy: 0.8704 Val Macro F1: 0.8674
Epoch [4/30] Train Loss: 0.2115 Val Accuracy: 0.8607 Val Macro F1: 0.8589
Epoch [5/30] Train Loss: 0.1869 Val Accuracy: 0.8634 Val Macro F1: 0.8615
Epoch [6/30] Train Loss: 0.1682 Val Accuracy: 0.8727 Val Macro F1: 0.8701
Epoch [7/30] Train Loss: 0.1423 Val Accuracy: 0.8579 Val Macro F1: 0.8570
Epoch [8/30] Train Loss: 0.1242 Val Accuracy: 0.8644 Val Macro F1: 0.8623
Epoch [9/30] Train Loss: 0.1061 Val Accuracy: 0.8385 Val Macro F1: 0.8382
Epoch [10/30] Train Loss: 0.0889 Val Accuracy: 0.8267 Val Macro F1: 0.8092
Epoch [11/30] Train Loss: 0.0735 Val Accuracy: 0.8582 Val Macro F1: 0.8549
Epoch [12/30] Train Loss: 0.0619 Val Accuracy: 0.8357 Val Macro F1: 0.8351
Epoch [13/30] Train Loss: 0.0487 Val Accuracy: 0.8275 Val Macro F1: 0.8271
Epoch [14/30] Train Loss: 0.0346 Val Accuracy: 0.8592 Val Macro F1: 0.8532
Epoch [15/30] Train Loss: 0.0292 Val Accuracy: 0.8524 Val Macro F1: 0.8514
Epoch [16/30] Train Loss: 0.0214 Val Accuracy: 0.8617 Val Macro F1: 0.8595
Epoch [17/30] Train Loss: 0.0174 Val Accuracy: 0.8607 Val Macro F1: 0.8561
Epoch [18/30] Train Loss: 0.0220 Val Accuracy: 0.8544 Val Macro F1: 0.8493
Epoch [19/30] Train Loss: 0.0134 Val Accuracy: 0.8452 Val Macro F1: 0.8441
Epoch [20/30] Train Loss: 0.0141 Val Accuracy: 0.8627 Val Macro F1: 0.8581
Epoch [21/30] Train Loss: 0.0158 Val Accuracy: 0.8539 Val Macro F1: 0.8523
Epoch [22/30] Train Loss: 0.0090 Val Accuracy: 0.8517 Val Macro F1: 0.8462
Epoch [23/30] Train Loss: 0.0107 Val Accuracy: 0.8395 Val Macro F1: 0.8384
Epoch [24/30] Train Loss: 0.0073 Val Accuracy: 0.8467 Val Macro F1: 0.8434
Epoch [25/30] Train Loss: 0.0144 Val Accuracy: 0.8487 Val Macro F1: 0.8474

Epoch [26/30] Train Loss: 0.0158 Val Accuracy: 0.8559 Val Macro F1: 0.8542
Epoch [27/30] Train Loss: 0.0064 Val Accuracy: 0.8502 Val Macro F1: 0.8490
Epoch [28/30] Train Loss: 0.0053 Val Accuracy: 0.8529 Val Macro F1: 0.8485
Epoch [29/30] Train Loss: 0.0116 Val Accuracy: 0.8449 Val Macro F1: 0.8424
Epoch [30/30] Train Loss: 0.0052 Val Accuracy: 0.8519 Val Macro F1: 0.8506

Best validation epoch: 6
Best validation macro F1: 0.8701

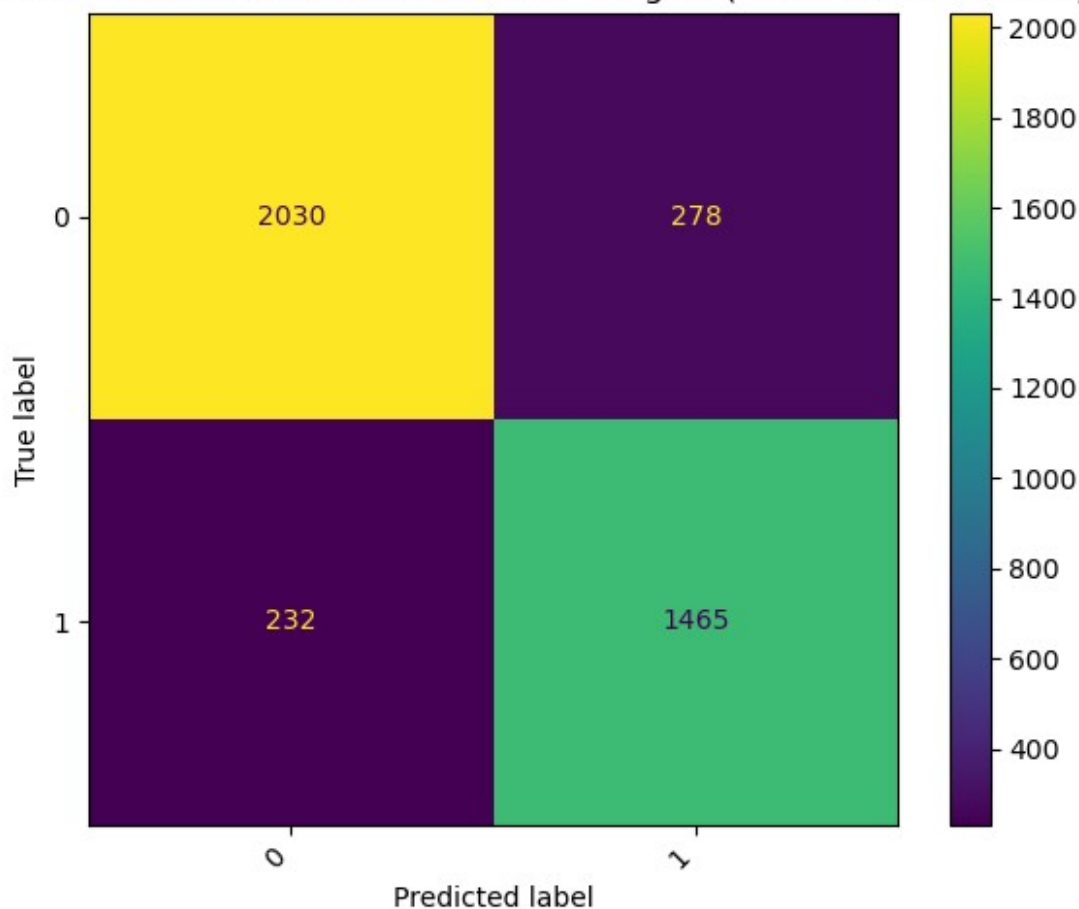
=====
=====
Task: Cancer Classification
Model: CNN lr=0.0001 no class weights
=====

=====
Accuracy: 0.8727
Macro F1: 0.8701
Weighted F1: 0.8729
Macro Precision: 0.8690
Macro Recall: 0.8714

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.88	0.89	2308
1	0.84	0.86	0.85	1697
accuracy			0.87	4005
macro avg	0.87	0.87	0.87	4005
weighted avg	0.87	0.87	0.87	4005

Confusion Matrix: CNN lr=0.0001 no class weights (Cancer Classification)



```
#experiment 4: original learning rate with weight decay
cnn_cancer_lr001_decay, history_lr001_decay, metrics_lr001_decay =
train_cancer_cnn_experiment(
    experiment_name="CNN lr=0.001 weight decay",
    learning_rate=0.001,
    weight_decay=1e-4,
    use_class_weights=True,
    num_epochs=30 if not QUICK_RUN else 5
)
```

```
=====
Experiment: CNN lr=0.001 weight decay
Learning rate: 0.001
Weight decay: 0.0001
Use class weights: True
Epochs: 30
=====
```

```
Epoch [1/30] Train Loss: 0.3324 Val Accuracy: 0.7263 Val Macro F1:
```

0.7231
Epoch [2/30] Train Loss: 0.2701 Val Accuracy: 0.8152 Val Macro F1: 0.8067
Epoch [3/30] Train Loss: 0.2401 Val Accuracy: 0.7203 Val Macro F1: 0.7166
Epoch [4/30] Train Loss: 0.2219 Val Accuracy: 0.8454 Val Macro F1: 0.8419
Epoch [5/30] Train Loss: 0.2136 Val Accuracy: 0.7316 Val Macro F1: 0.7287
Epoch [6/30] Train Loss: 0.1923 Val Accuracy: 0.8180 Val Macro F1: 0.8179
Epoch [7/30] Train Loss: 0.1791 Val Accuracy: 0.6819 Val Macro F1: 0.6086
Epoch [8/30] Train Loss: 0.1678 Val Accuracy: 0.8075 Val Macro F1: 0.7922
Epoch [9/30] Train Loss: 0.1435 Val Accuracy: 0.7076 Val Macro F1: 0.7028
Epoch [10/30] Train Loss: 0.1203 Val Accuracy: 0.8622 Val Macro F1: 0.8600
Epoch [11/30] Train Loss: 0.1084 Val Accuracy: 0.8587 Val Macro F1: 0.8567
Epoch [12/30] Train Loss: 0.0923 Val Accuracy: 0.8290 Val Macro F1: 0.8287
Epoch [13/30] Train Loss: 0.0846 Val Accuracy: 0.8514 Val Macro F1: 0.8448
Epoch [14/30] Train Loss: 0.0729 Val Accuracy: 0.8345 Val Macro F1: 0.8343
Epoch [15/30] Train Loss: 0.0555 Val Accuracy: 0.8639 Val Macro F1: 0.8581
Epoch [16/30] Train Loss: 0.0508 Val Accuracy: 0.8487 Val Macro F1: 0.8380
Epoch [17/30] Train Loss: 0.0537 Val Accuracy: 0.7169 Val Macro F1: 0.6765
Epoch [18/30] Train Loss: 0.0426 Val Accuracy: 0.8694 Val Macro F1: 0.8649
Epoch [19/30] Train Loss: 0.0354 Val Accuracy: 0.8507 Val Macro F1: 0.8498
Epoch [20/30] Train Loss: 0.0364 Val Accuracy: 0.7109 Val Macro F1: 0.6576
Epoch [21/30] Train Loss: 0.0350 Val Accuracy: 0.8400 Val Macro F1: 0.8331
Epoch [22/30] Train Loss: 0.0275 Val Accuracy: 0.8589 Val Macro F1: 0.8569
Epoch [23/30] Train Loss: 0.0284 Val Accuracy: 0.8594 Val Macro F1: 0.8571
Epoch [24/30] Train Loss: 0.0283 Val Accuracy: 0.7855 Val Macro F1: 0.7607
Epoch [25/30] Train Loss: 0.0219 Val Accuracy: 0.8270 Val Macro F1: 0.8120

Epoch [26/30] Train Loss: 0.0216 Val Accuracy: 0.8222 Val Macro F1: 0.8219
Epoch [27/30] Train Loss: 0.0326 Val Accuracy: 0.8414 Val Macro F1: 0.8404
Epoch [28/30] Train Loss: 0.0259 Val Accuracy: 0.8442 Val Macro F1: 0.8427
Epoch [29/30] Train Loss: 0.0242 Val Accuracy: 0.8569 Val Macro F1: 0.8500
Epoch [30/30] Train Loss: 0.0200 Val Accuracy: 0.8519 Val Macro F1: 0.8496

Best validation epoch: 18
Best validation macro F1: 0.8649

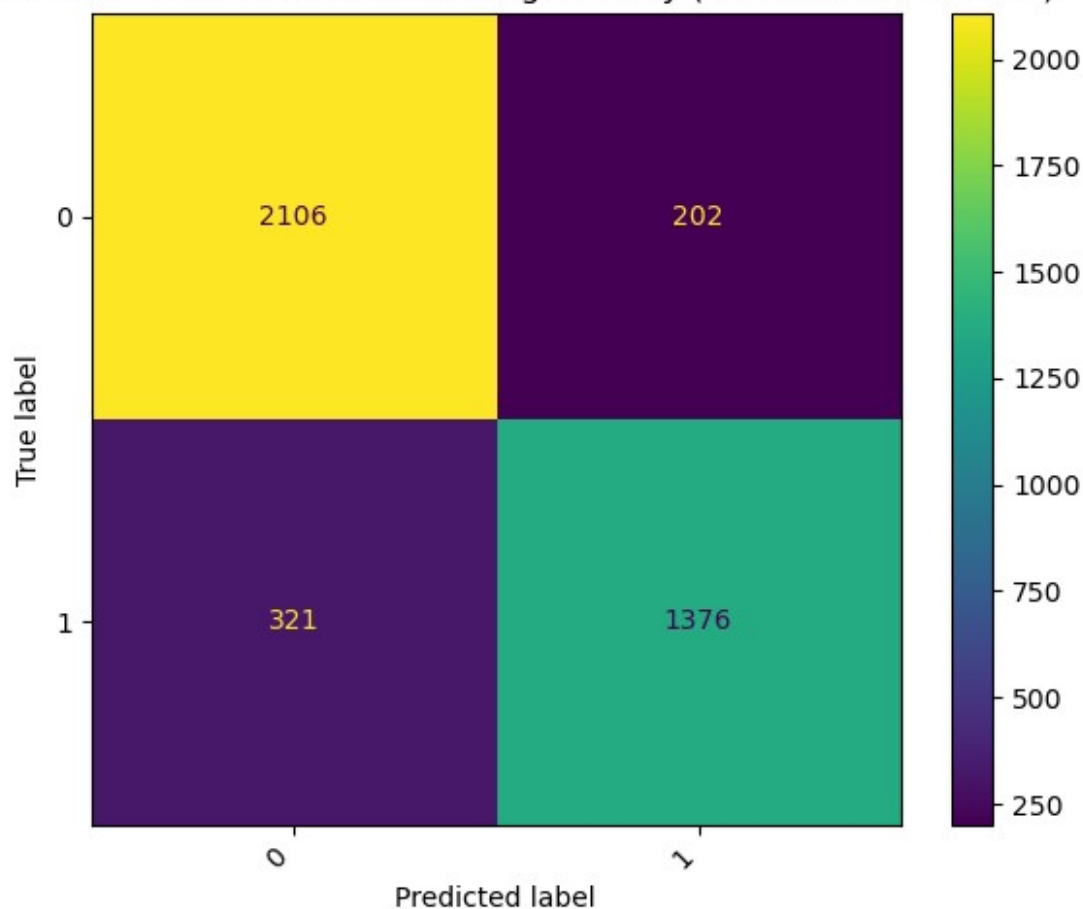
=====
=====
Task: Cancer Classification
Model: CNN lr=0.001 weight decay
=====

=====
Accuracy: 0.8694
Macro F1: 0.8649
Weighted F1: 0.8687
Macro Precision: 0.8699
Macro Recall: 0.8617

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.91	0.89	2308
1	0.87	0.81	0.84	1697
accuracy			0.87	4005
macro avg	0.87	0.86	0.86	4005
weighted avg	0.87	0.87	0.87	4005

Confusion Matrix: CNN lr=0.001 weight decay (Cancer Classification)



14.4 Hyperparameter Tuning Results

```
cnn_tuning_results_df = pd.DataFrame(cnn_tuning_results)
```

```
display(
    cnn_tuning_results_df.sort_values(
        by="macro_f1",
        ascending=False
    )
)
```

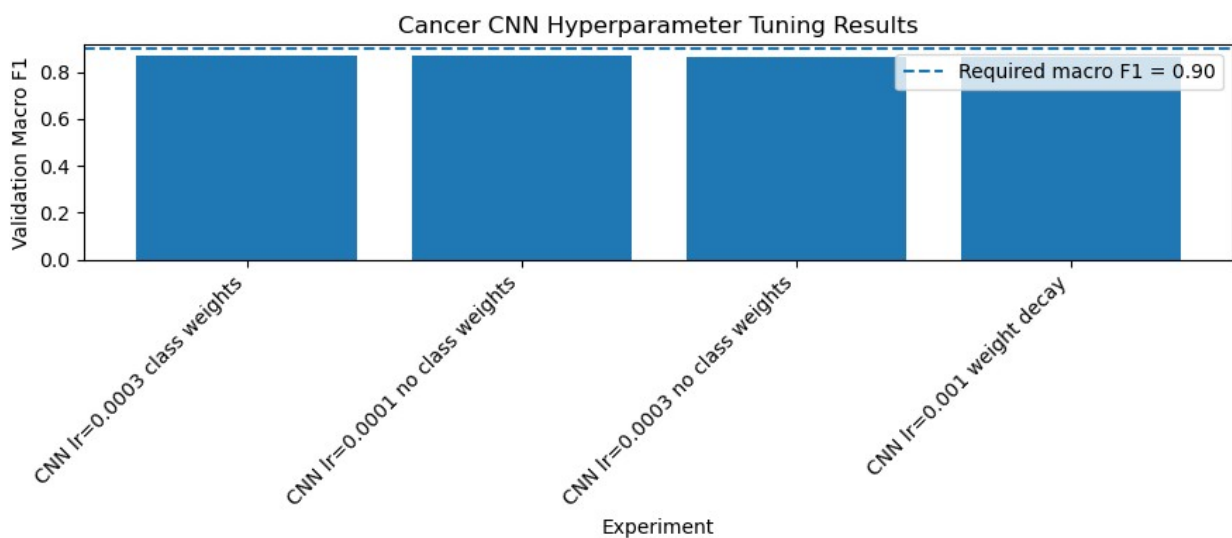
	task	model	accuracy
macro_f1 \			
1	Cancer Classification	CNN lr=0.0003 class weights	0.875406
			0.872699
2	Cancer Classification	CNN lr=0.0001 no class weights	0.872659
			0.870073
0	Cancer Classification	CNN lr=0.0003 no class weights	0.866667
			0.865500
3	Cancer Classification	CNN lr=0.001 weight decay	0.869413
			0.864926

	weighted_f1	macro_precision	macro_recall	best_epoch
best_val_macro_f1 \				
1	0.875531	0.871992	0.873490	9
0.872699				
2	0.872870	0.868970	0.871419	6
0.870073				
0	0.867411	0.864510	0.872771	5
0.865500				
3	0.868682	0.869864	0.861660	18
0.864926				

	learning_rate	weight_decay	use_class_weights
1	0.0003	0.0001	True
2	0.0001	0.0001	False
0	0.0003	0.0001	False
3	0.0010	0.0001	True

```
plt.figure(figsize=(9, 4))
plot_df = cnn_tuning_results_df.sort_values(by="macro_f1",
ascending=False)

plt.bar(plot_df["model"], plot_df["macro_f1"])
plt.axhline(0.90, linestyle="--", label="Required macro F1 = 0.90")
plt.title("Cancer CNN Hyperparameter Tuning Results")
plt.xlabel("Experiment")
plt.ylabel("Validation Macro F1")
plt.xticks(rotation=45, ha="right")
plt.legend()
plt.tight_layout()
plt.show()
```



14.5 Best Tuned CNN Selection

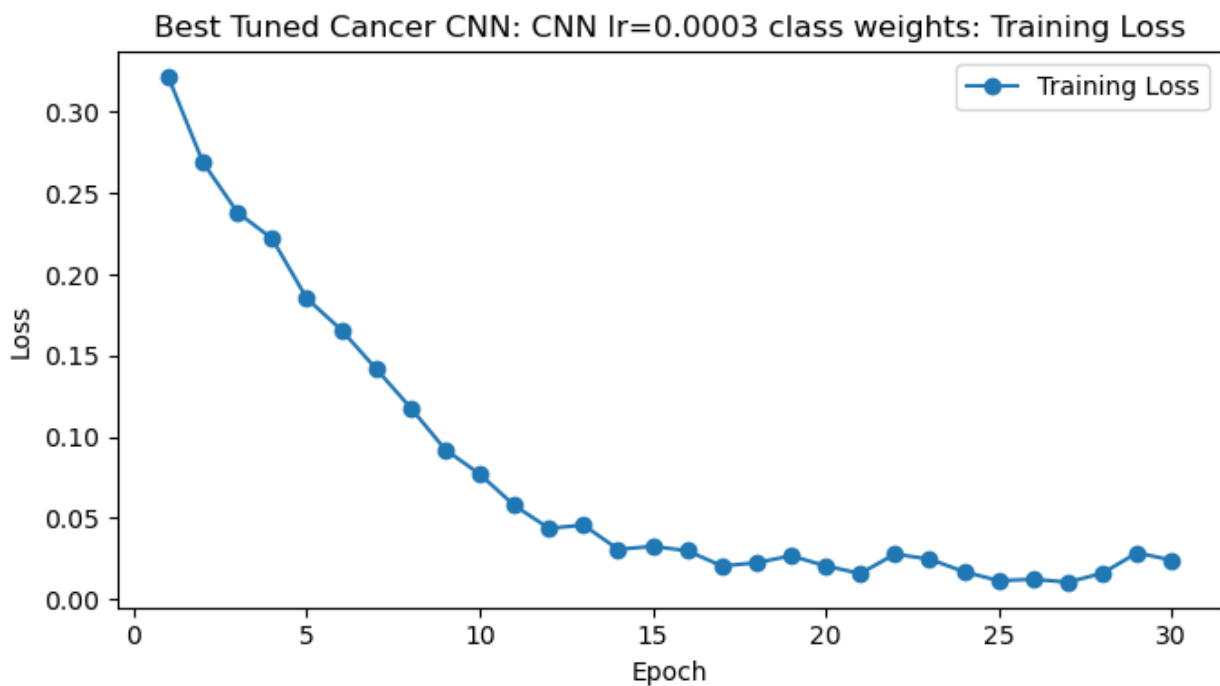
```
best_cancer_tuning_row = cnn_tuning_results_df.sort_values(
    by="macro_f1",
    ascending=False
).iloc[0]

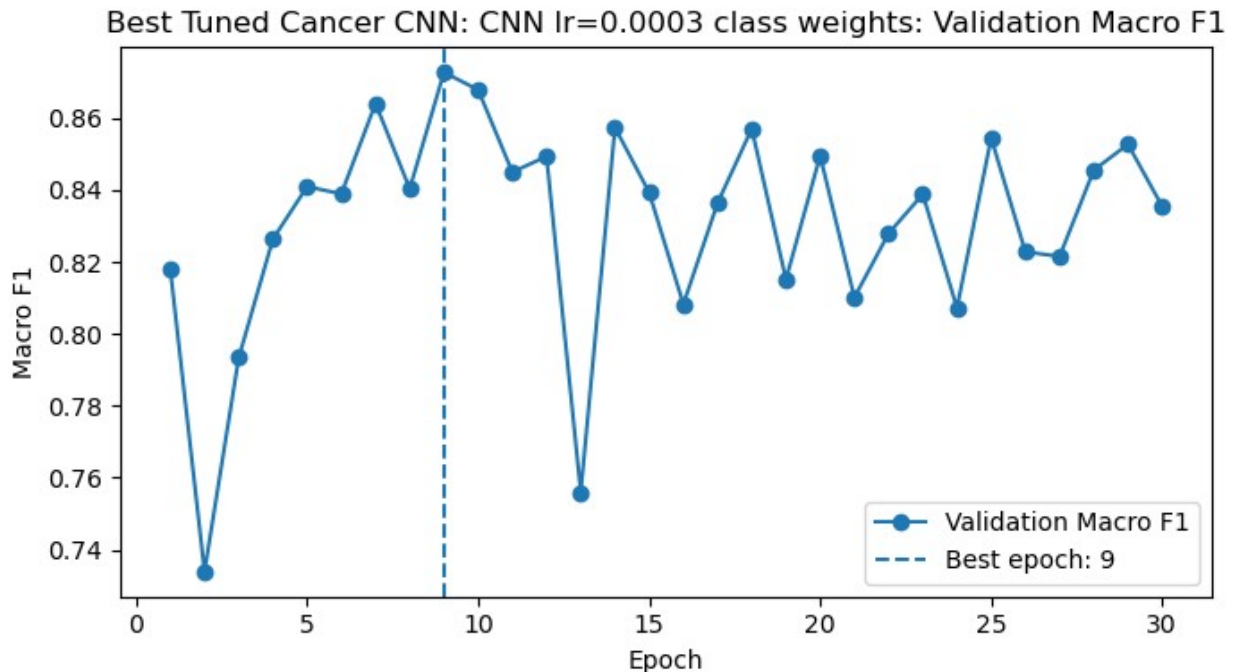
best_tuned_cancer_model_name = best_cancer_tuning_row["model"]
best_tuned_cancer_model =
cnn_tuning_models[best_tuned_cancer_model_name]
best_tuned_cancer_history =
cnn_tuning_histories[best_tuned_cancer_model_name]

print("Best tuned cancer CNN:", best_tuned_cancer_model_name)
print("Best validation macro F1:",
round(best_cancer_tuning_row["macro_f1"], 4))
print("Best epoch:", best_cancer_tuning_row.get("best_epoch", "N/A"))

Best tuned cancer CNN: CNN lr=0.0003 class weights
Best validation macro F1: 0.8727
Best epoch: 9

plot_training_history(
    best_tuned_cancer_history,
    title=f"Best Tuned Cancer CNN: {best_tuned_cancer_model_name}"
)
```





Best epoch for Best Tuned Cancer CNN: CNN lr=0.0003 class weights: 9
Best validation macro F1 for Best Tuned Cancer CNN: CNN lr=0.0003
class weights: 0.8727

14.6 Tuning Discussion

The tuning results show how learning rate, weight decay, and class-weighted loss affect the cancer CNN. The learning rate is important because it controls how quickly the model updates its weights. A smaller learning rate may train more slowly, but it can sometimes produce better validation performance because the optimiser makes more gradual updates. Weight decay is used as a regularisation technique. Its purpose is to reduce overfitting by discouraging overly large model weights. Class-weighted loss is tested because the cancer task is imbalanced. However, class weighting does not always guarantee better macro F1-score. It can improve recall for the minority class, but it may also reduce precision if the model becomes too biased toward that class. The best model is selected using validation macro F1-score rather than training loss. This is important because lower training loss does not automatically mean better generalisation to unseen patients.

14.7 Hyperparameter Tuning Interpretation

The hyperparameter tuning experiments did not improve the cancer CNN beyond the original improved CNN. The best tuned model achieved a validation macro F1-score of **0.8727**, while the original improved CNN achieved **0.8764**. This suggests that the cancer CNN's current limitation is not only the learning rate, weight decay, or class-weighted loss setting. The model may need a stronger generalisation technique rather than only optimiser-level tuning. For this reason, the next section applies data augmentation. This is appropriate because the input data consists of small histology image patches, and the model needs to generalise to unseen patients.

15. Advanced Technique: Data Augmentation

The previous tuning section showed that changing the learning rate, weight decay, and class-weighted loss did not improve the cancer CNN beyond the original improved CNN. Therefore, this section applies data augmentation as an advanced technique. Data augmentation is used because the dataset consists of small 27×27 histology image patches. The model must generalise to unseen patients, so exposing the CNN to slightly varied versions of the training images may reduce overfitting and improve validation performance. Only the training set is augmented. The validation set is not augmented, because it must remain a fair estimate of model performance on unchanged unseen-patient images.

The augmentation operations used are mild:

- random horizontal flipping
- random vertical flipping
- small random rotation
- mild colour and contrast changes

These transformations are reasonable for histology image patches because the target label should not depend strongly on the exact orientation of the image.

15.1 Import augmentation tools

```
from torchvision import transforms
```

15.2 Define augmentation transforms

```
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(
        brightness=0.10,
        contrast=0.10,
        saturation=0.10
    )
])

eval_transform = None

print("Training transform:")
print(train_transform)

Training transform:
Compose(
  RandomHorizontalFlip(p=0.5)
  RandomVerticalFlip(p=0.5)
  RandomRotation(degrees=[-15.0, 15.0], interpolation=nearest,
expand=False, fill=0)
  ColorJitter(brightness=(0.9, 1.1), contrast=(0.9, 1.1),
```

```
saturation=(0.9, 1.1), hue=None)
)
```

15.3 Create augmented dataset class

```
class AugmentedHistologyDataset(Dataset):
    """
    PyTorch Dataset with optional image augmentation.

    The image input is expected to be in C x H x W format
    with pixel values normalised to [0, 1].
    """

    def __init__(self, X, y, transform=None):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)
        self.transform = transform

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        image = self.X[idx]
        label = self.y[idx]

        if self.transform is not None:
            image = self.transform(image)

        return image, label
```

15.4 Create augmented cancer DataLoaders

```
aug_cancer_train_dataset = AugmentedHistologyDataset(
    X_cancer_train_tensor,
    y_cancer_train,
    transform=train_transform
)

aug_cancer_val_dataset = AugmentedHistologyDataset(
    X_cancer_val_tensor,
    y_cancer_val,
    transform=eval_transform
)

aug_cancer_train_loader = DataLoader(
    aug_cancer_train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True
)
```

```

aug_cancer_val_loader = DataLoader(
    aug_cancer_val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)

print("Augmented cancer training batches:",
len(aug_cancer_train_loader))
print("Cancer validation batches:", len(aug_cancer_val_loader))

Augmented cancer training batches: 193
Cancer validation batches: 63

```

15.5 Train augmented cancer CNN using original strong setting

```

aug_cnn_cancer_original_setting = ImprovedCNN(
    num_classes=len(cancer_class_names)
)

criterion_aug_cancer_original = nn.CrossEntropyLoss(
    weight=cancer_class_weights
)

optimizer_aug_cancer_original = optim.Adam(
    aug_cnn_cancer_original_setting.parameters(),
    lr=0.001
)

aug_cnn_cancer_original_setting, aug_cnn_cancer_original_history,
aug_cnn_cancer_original_metrics = train_pytorch_model(
    model=aug_cnn_cancer_original_setting,
    train_loader=aug_cancer_train_loader,
    val_loader=aug_cancer_val_loader,
    criterion=criterion_aug_cancer_original,
    optimizer=optimizer_aug_cancer_original,
    device=device,
    num_epochs=40 if not QUICK_RUN else 5,
    model_name="Improved CNN + Data Augmentation",
    task_name="Cancer Classification",
    class_names=cancer_class_names
)

validation_results.append(aug_cnn_cancer_original_metrics)

Epoch [1/40] Train Loss: 0.3727 Val Accuracy: 0.7810 Val Macro F1:
0.7508
Epoch [2/40] Train Loss: 0.3171 Val Accuracy: 0.6122 Val Macro F1:
0.5901
Epoch [3/40] Train Loss: 0.3071 Val Accuracy: 0.7513 Val Macro F1:
0.7497

```

Epoch [4/40] Train Loss: 0.2895 Val Accuracy: 0.7071 Val Macro F1: 0.7021
Epoch [5/40] Train Loss: 0.2831 Val Accuracy: 0.7670 Val Macro F1: 0.7661
Epoch [6/40] Train Loss: 0.2807 Val Accuracy: 0.8400 Val Macro F1: 0.8365
Epoch [7/40] Train Loss: 0.2728 Val Accuracy: 0.8649 Val Macro F1: 0.8600
Epoch [8/40] Train Loss: 0.2634 Val Accuracy: 0.8727 Val Macro F1: 0.8695
Epoch [9/40] Train Loss: 0.2642 Val Accuracy: 0.8195 Val Macro F1: 0.8018
Epoch [10/40] Train Loss: 0.2623 Val Accuracy: 0.7928 Val Macro F1: 0.7926
Epoch [11/40] Train Loss: 0.2550 Val Accuracy: 0.8315 Val Macro F1: 0.8192
Epoch [12/40] Train Loss: 0.2543 Val Accuracy: 0.8065 Val Macro F1: 0.7908
Epoch [13/40] Train Loss: 0.2493 Val Accuracy: 0.7056 Val Macro F1: 0.7003
Epoch [14/40] Train Loss: 0.2467 Val Accuracy: 0.8277 Val Macro F1: 0.8276
Epoch [15/40] Train Loss: 0.2461 Val Accuracy: 0.8831 Val Macro F1: 0.8804
Epoch [16/40] Train Loss: 0.2387 Val Accuracy: 0.8819 Val Macro F1: 0.8781
Epoch [17/40] Train Loss: 0.2375 Val Accuracy: 0.6797 Val Macro F1: 0.6732
Epoch [18/40] Train Loss: 0.2337 Val Accuracy: 0.8724 Val Macro F1: 0.8716
Epoch [19/40] Train Loss: 0.2310 Val Accuracy: 0.7513 Val Macro F1: 0.7502
Epoch [20/40] Train Loss: 0.2243 Val Accuracy: 0.8834 Val Macro F1: 0.8792
Epoch [21/40] Train Loss: 0.2232 Val Accuracy: 0.8564 Val Macro F1: 0.8560
Epoch [22/40] Train Loss: 0.2311 Val Accuracy: 0.8195 Val Macro F1: 0.8195
Epoch [23/40] Train Loss: 0.2255 Val Accuracy: 0.7556 Val Macro F1: 0.7538
Epoch [24/40] Train Loss: 0.2195 Val Accuracy: 0.6682 Val Macro F1: 0.6578
Epoch [25/40] Train Loss: 0.2167 Val Accuracy: 0.8884 Val Macro F1: 0.8854
Epoch [26/40] Train Loss: 0.2107 Val Accuracy: 0.8749 Val Macro F1: 0.8680
Epoch [27/40] Train Loss: 0.2151 Val Accuracy: 0.8432 Val Macro F1: 0.8431
Epoch [28/40] Train Loss: 0.2126 Val Accuracy: 0.7778 Val Macro F1:

0.7770
Epoch [29/40] Train Loss: 0.2047 Val Accuracy: 0.8287 Val Macro F1: 0.8286
Epoch [30/40] Train Loss: 0.2049 Val Accuracy: 0.8687 Val Macro F1: 0.8626
Epoch [31/40] Train Loss: 0.2020 Val Accuracy: 0.8557 Val Macro F1: 0.8483
Epoch [32/40] Train Loss: 0.1976 Val Accuracy: 0.8752 Val Macro F1: 0.8720
Epoch [33/40] Train Loss: 0.2012 Val Accuracy: 0.8307 Val Macro F1: 0.8168
Epoch [34/40] Train Loss: 0.1971 Val Accuracy: 0.8864 Val Macro F1: 0.8817
Epoch [35/40] Train Loss: 0.2013 Val Accuracy: 0.8759 Val Macro F1: 0.8751
Epoch [36/40] Train Loss: 0.1944 Val Accuracy: 0.8544 Val Macro F1: 0.8538
Epoch [37/40] Train Loss: 0.1896 Val Accuracy: 0.8300 Val Macro F1: 0.8287
Epoch [38/40] Train Loss: 0.1851 Val Accuracy: 0.7785 Val Macro F1: 0.7779
Epoch [39/40] Train Loss: 0.1848 Val Accuracy: 0.8684 Val Macro F1: 0.8667
Epoch [40/40] Train Loss: 0.1852 Val Accuracy: 0.8961 Val Macro F1: 0.8946

Best validation epoch: 40
Best validation macro F1: 0.8946

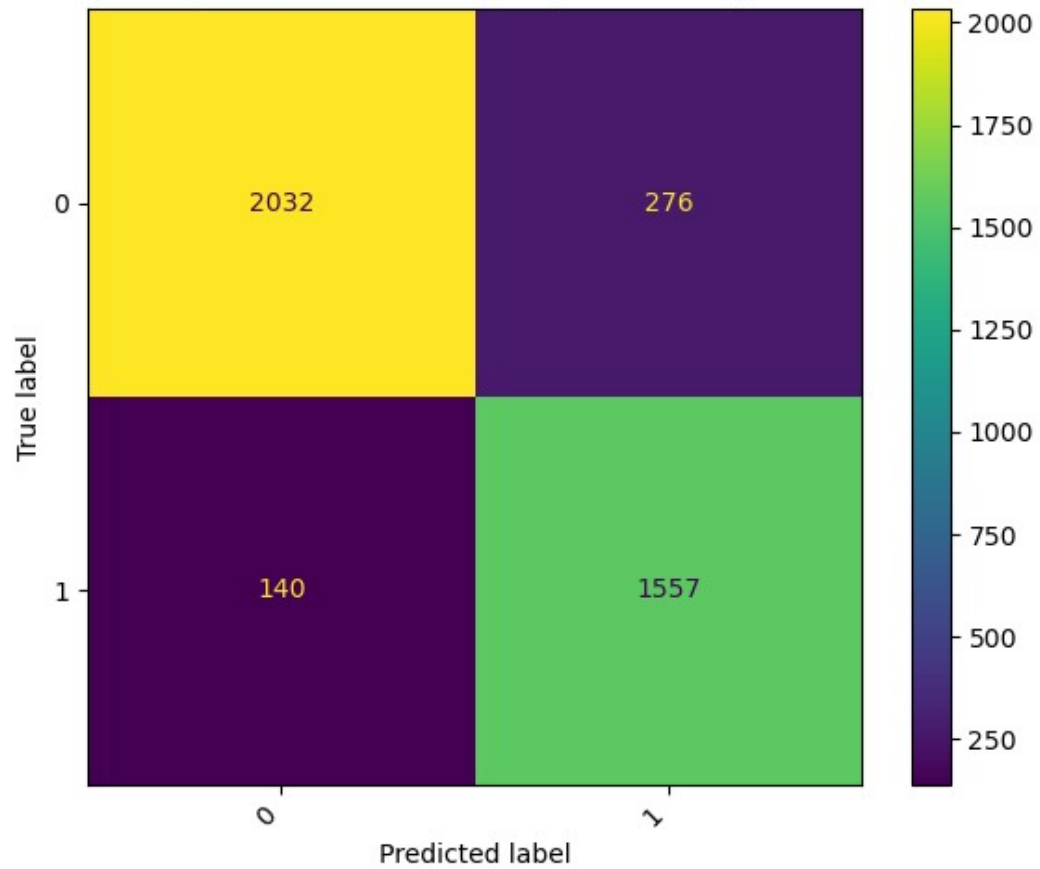
=====
=====
Task: Cancer Classification
Model: Improved CNN + Data Augmentation
=====

=====
Accuracy: 0.8961
Macro F1: 0.8946
Weighted F1: 0.8966
Macro Precision: 0.8925
Macro Recall: 0.8990

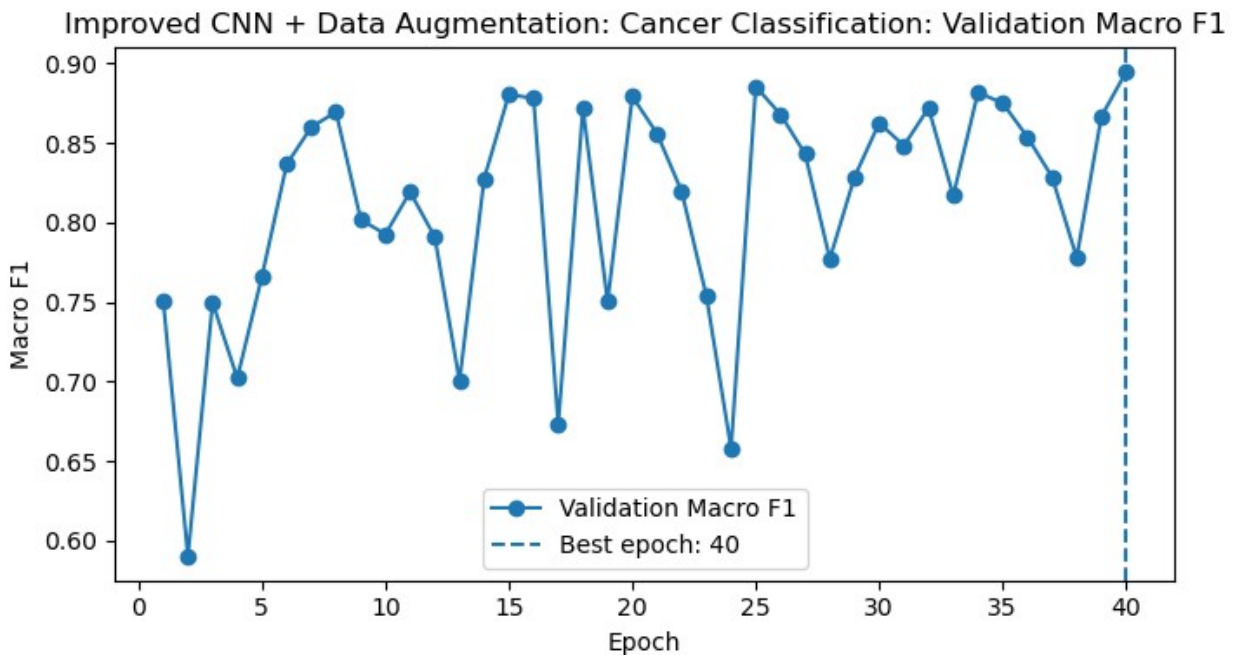
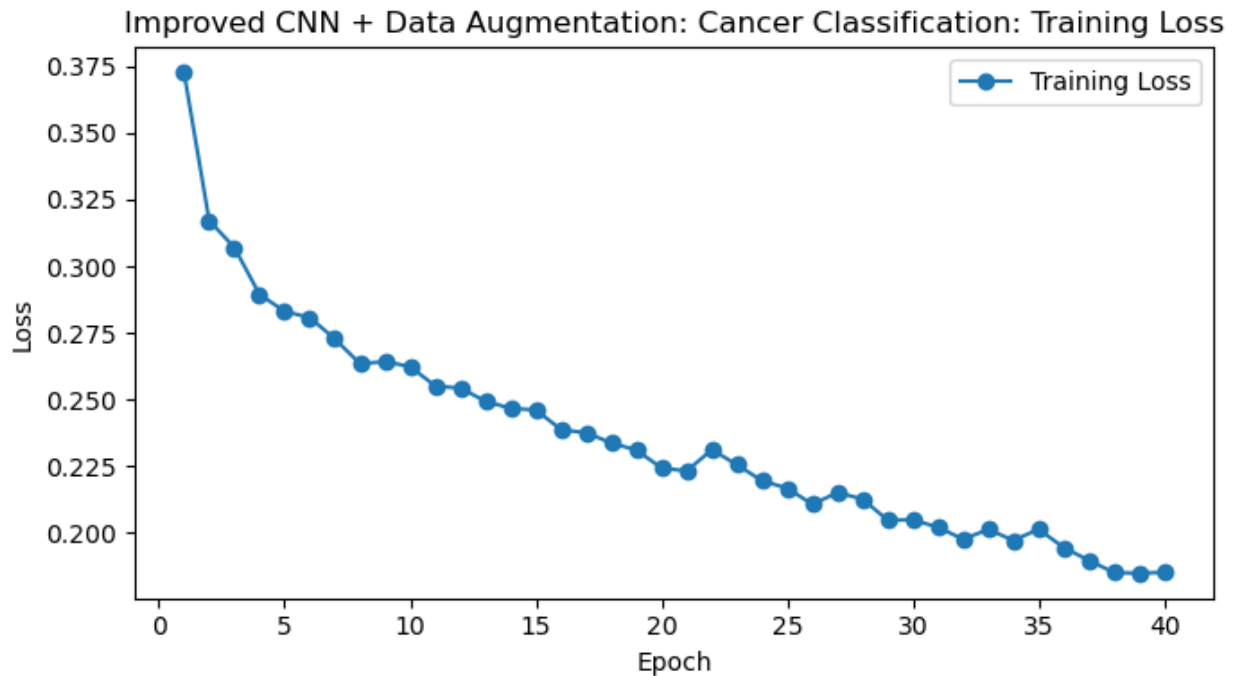
Classification Report:

	precision	recall	f1-score	support
0	0.94	0.88	0.91	2308
1	0.85	0.92	0.88	1697
accuracy			0.90	4005
macro avg	0.89	0.90	0.89	4005
weighted avg	0.90	0.90	0.90	4005

Confusion Matrix: Improved CNN + Data Augmentation (Cancer Classification)



```
plot_training_history(  
    aug_cnn_cancer_original_history,  
    title="Improved CNN + Data Augmentation: Cancer Classification"  
)
```



Best epoch for Improved CNN + Data Augmentation: Cancer Classification: 40
Best validation macro F1 for Improved CNN + Data Augmentation: Cancer Classification: 0.8946

15.6 Decision Threshold Tuning for the Augmented Cancer CNN

The augmented CNN improved the cancer classification validation macro F1-score to **0.8946**, which is close to the required threshold of **0.90**. Because cancer classification is a binary task, the model's probability output can be converted into class predictions using different decision thresholds. The default prediction rule is equivalent to selecting the class with the highest probability. However, this default threshold may not maximise macro F1-score. This section tunes the decision threshold using the validation set only. The test set remains unseen and is not used during threshold selection.

```
def predict_pytorch_probabilities(model, data_loader, device):
    """
    Generate true labels and predicted class probabilities from a
    PyTorch model.
    """
    model.eval()

    all_true = []
    all_probs = []

    with torch.no_grad():
        for X_batch, y_batch in data_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            outputs = model(X_batch)
            probs = torch.softmax(outputs, dim=1)

            all_true.extend(y_batch.cpu().numpy())
            all_probs.extend(probs.cpu().numpy())

    return np.array(all_true), np.array(all_probs)

y_aug_cancer_val_true, aug_cancer_val_probs =
predict_pytorch_probabilities(
    model=aug_cnn_cancer_original_setting,
    data_loader=aug_cancer_val_loader,
    device=device
)

print("Validation true labels shape:", y_aug_cancer_val_true.shape)
print("Validation probability matrix shape:",
aug_cancer_val_probs.shape)
print("First 5 probability rows:")
print(aug_cancer_val_probs[:5])

Validation true labels shape: (4005,)
Validation probability matrix shape: (4005, 2)
First 5 probability rows:
[[9.9862015e-01 1.3798230e-03]
```

```

[9.9952233e-01 4.7764962e-04]
[7.6028872e-01 2.3971134e-01]
[7.1130127e-01 2.8869873e-01]
[7.1629190e-01 2.8370804e-01]]

aug_threshold_results = []

class_1_probs = aug_cancer_val_probs[:, 1]

for threshold in np.arange(0.05, 0.96, 0.01):
    y_pred_threshold = (class_1_probs >= threshold).astype(int)

    aug_threshold_results.append({
        "threshold": threshold,
        "accuracy": accuracy_score(y_aug_cancer_val_true,
y_pred_threshold),
        "macro_f1": f1_score(y_aug_cancer_val_true, y_pred_threshold,
average="macro", zero_division=0),
        "weighted_f1": f1_score(y_aug_cancer_val_true,
y_pred_threshold, average="weighted", zero_division=0),
        "macro_precision": precision_score(y_aug_cancer_val_true,
y_pred_threshold, average="macro", zero_division=0),
        "macro_recall": recall_score(y_aug_cancer_val_true,
y_pred_threshold, average="macro", zero_division=0)
    })

aug_threshold_results_df = pd.DataFrame(aug_threshold_results)

display(
    aug_threshold_results_df.sort_values(
        by="macro_f1",
        ascending=False
    ).head(15)
)

```

	threshold	accuracy	macro_f1	weighted_f1	macro_precision
macro_recall					
60	0.65	0.910112	0.907927	0.910091	0.908122
0.907736					
61	0.66	0.910112	0.907824	0.910040	0.908507
0.907190					
58	0.63	0.909863	0.907793	0.909901	0.907457
0.908144					
59	0.64	0.909863	0.907722	0.909866	0.907690
0.907754					
62	0.67	0.909863	0.907516	0.909763	0.908458
0.906662					
63	0.68	0.909613	0.907237	0.909502	0.908294
0.906289					
64	0.69	0.909613	0.907160	0.909462	0.908616

0.905899					
65	0.70	0.909363	0.906833	0.909175	0.908667
0.905293					
57	0.62	0.908864	0.906828	0.908929	0.906267
0.907433					
56	0.61	0.908365	0.906388	0.908463	0.905564
0.907312					
66	0.71	0.908365	0.905726	0.908132	0.907998
0.903880					
55	0.60	0.907366	0.905423	0.907491	0.904409
0.906601					
67	0.72	0.907615	0.904831	0.907315	0.907806
0.902528					
68	0.73	0.907615	0.904730	0.907259	0.908309
0.902060					
70	0.75	0.907615	0.904539	0.907153	0.909317
0.901202					

```
best_aug_threshold_row = aug_threshold_results_df.sort_values(
    by="macro_f1",
    ascending=False
).iloc[0]
```

```
best_aug_cancer_threshold = best_aug_threshold_row["threshold"]
```

```
print("Best augmented cancer decision threshold:",
      round(best_aug_cancer_threshold, 2))
print("Best validation macro F1:",
      round(best_aug_threshold_row["macro_f1"], 4))
```

```
Best augmented cancer decision threshold: 0.65
Best validation macro F1: 0.9079
```

```
y_aug_cancer_val_pred_tuned = (class_1_probs >=
best_aug_cancer_threshold).astype(int)
```

```
aug_cnn_threshold_metrics = evaluate_predictions(
    y_true=y_aug_cancer_val_true,
    y_pred=y_aug_cancer_val_pred_tuned,
    model_name="Improved CNN + Data Augmentation + Threshold Tuning",
    task_name="Cancer Classification",
    class_names=cancer_class_names,
    show_report=True,
    show_confusion_matrix=True
)
```

```
aug_cnn_threshold_metrics["threshold"] = best_aug_cancer_threshold
```

```
validation_results.append(aug_cnn_threshold_metrics)
```

```
=====
Task: Cancer Classification
```

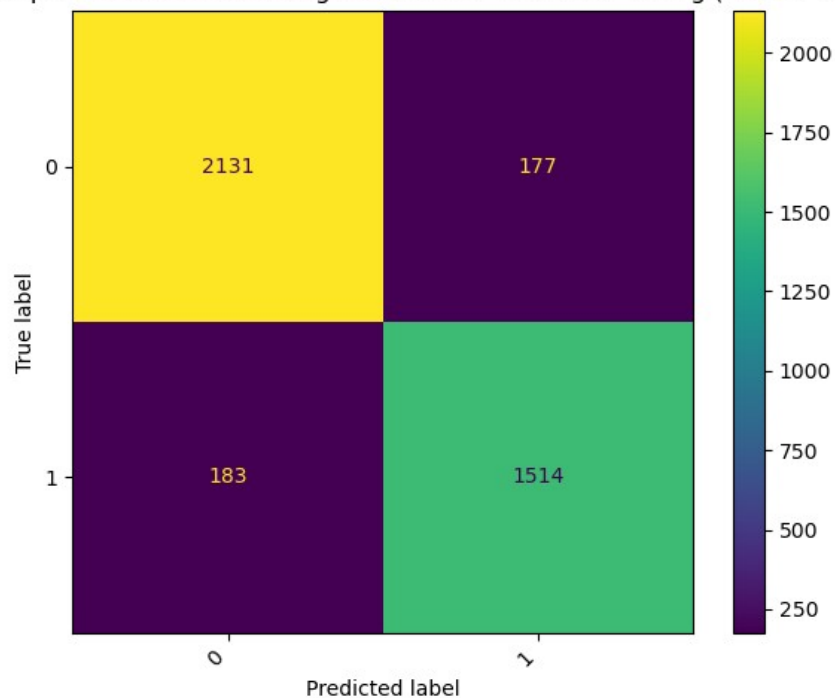
```
Model: Improved CNN + Data Augmentation + Threshold Tuning
=====
```

```
=====
Accuracy:      0.9101
Macro F1:      0.9079
Weighted F1:   0.9101
Macro Precision: 0.9081
Macro Recall:  0.9077
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.92	0.92	0.92	2308
1	0.90	0.89	0.89	1697
accuracy			0.91	4005
macro avg	0.91	0.91	0.91	4005
weighted avg	0.91	0.91	0.91	4005

Confusion Matrix: Improved CNN + Data Augmentation + Threshold Tuning (Cancer Classification)



```
plt.figure(figsize=(8, 4))
plt.plot(
    aug_threshold_results_df["threshold"],
```

```

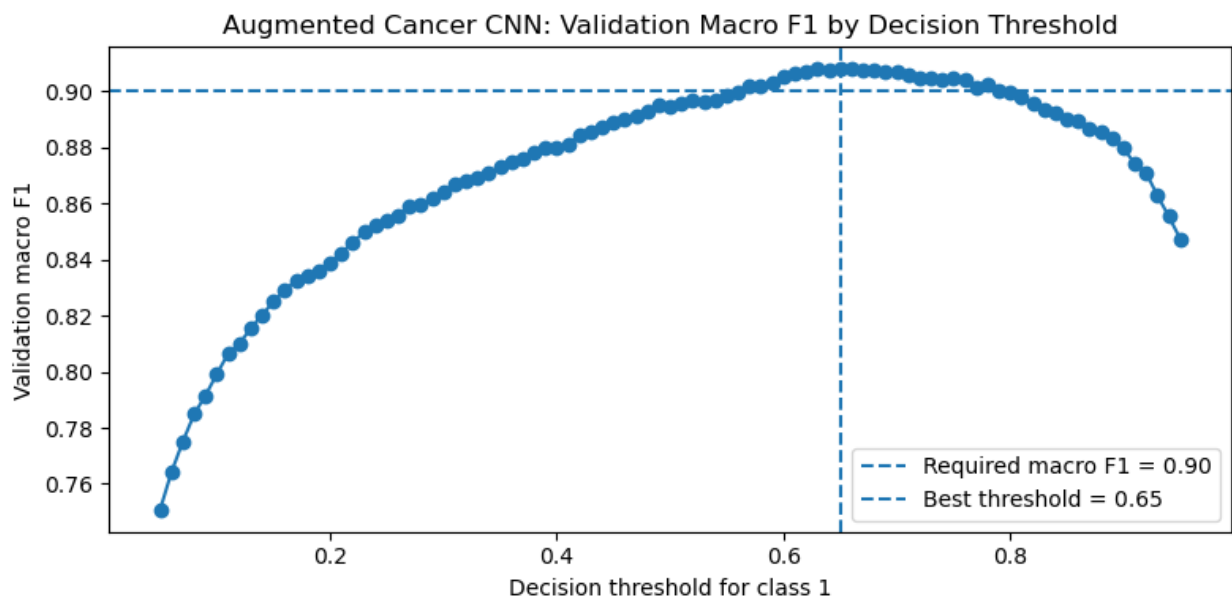
    aug_threshold_results_df["macro_f1"],
    marker="o"
)

plt.axhline(
    0.90,
    linestyle="--",
    label="Required macro F1 = 0.90"
)

plt.axvline(
    best_aug_cancer_threshold,
    linestyle="--",
    label=f"Best threshold = {best_aug_cancer_threshold:.2f}"
)

plt.title("Augmented Cancer CNN: Validation Macro F1 by Decision Threshold")
plt.xlabel("Decision threshold for class 1")
plt.ylabel("Validation macro F1")
plt.legend()
plt.tight_layout()
plt.show()

```



Data Augmentation Interpretation The augmented CNN achieved a validation macro F1-score of 0.8946, which was close to the required 0.90 threshold but still slightly below it. Decision threshold tuning improved the validation macro F1-score to 0.9079 using a threshold of 0.65 for the cancerous class. This result shows that the augmented CNN was learning useful probability outputs, but the default classification threshold was not the best decision rule for maximising macro-averaged F1-score. By tuning the threshold on the validation set, the model achieved a better balance between precision and recall across the cancerous and non-cancerous

classes. The final confusion matrix shows that the model correctly classified 2131 non-cancerous images and 1514 cancerous images. It made 177 false positive errors and 183 false negative errors. This is a more balanced error pattern than earlier models and supports the improved macro F1-score. The validation set was used for threshold selection, while the test set remains unseen. Therefore, the final test evaluation can still be used to estimate performance on unseen patients.

15.7 Advanced Technique Summary

Data augmentation and threshold tuning substantially improved the cancer classification result. The improved CNN baseline achieved a validation macro F1-score of 0.8764. After applying data augmentation, the score increased to 0.8946. After decision threshold tuning, the score increased further to 0.9079, exceeding the required macro F1-score threshold of 0.90. This suggests that the original CNN was limited partly by generalisation and partly by the default decision threshold. Data augmentation helped the CNN learn more stable visual features, while threshold tuning improved the balance between precision and recall. The best cancer classification model is therefore selected as Improved CNN + Data Augmentation + Threshold Tuning.

```
all_validation_results_df = pd.DataFrame(validation_results)

display(
    all_validation_results_df.sort_values(
        by=["task", "macro_f1"],
        ascending=[True, False]
    )
)
```

	task \
10	Cancer Classification
9	Cancer Classification
8	Cancer Classification
6	Cancer Classification
2	Cancer Classification
4	Cancer Classification
0	Cancer Classification
7	Cell-Type Classification
3	Cell-Type Classification
5	Cell-Type Classification
1	Cell-Type Classification

	model	accuracy
macro_f1 \		
10	Improved CNN + Data Augmentation + Threshold T...	0.910112
0.907927		
9	Improved CNN + Data Augmentation	0.896130
0.894648		
8	Improved CNN + Threshold Tuning	0.881898
0.878110		
6	Improved CNN	0.879650

0.876364		
2	SVM RBF	0.859925
0.856775		
4	Random Forest	0.836954
0.827703		
0	Logistic Regression	0.773034
0.770972		
7	Improved CNN	0.686754
0.637979		
3	SVM RBF	0.602606
0.505300		
5	Random Forest	0.574376
0.434521		
1	Logistic Regression	0.528773
0.432366		

	weighted_f1	macro_precision	macro_recall	best_epoch
best_val_macro_f1 \				
10	0.910091	0.908122	0.907736	NaN
NaN				
9	0.896554	0.892485	0.898959	40.0
0.894648				
8	0.881388	0.881918	0.875378	NaN
NaN				
6	0.879439	0.877790	0.875145	5.0
0.876364				
2	0.860015	0.856332	0.857250	NaN
NaN				
4	0.833794	0.846749	0.820706	NaN
NaN				
0	0.774287	0.770570	0.776634	NaN
NaN				
7	0.694149	0.633793	0.651460	11.0
0.637979				
3	0.587570	0.566720	0.550925	NaN
NaN				
5	0.515536	0.586220	0.516340	NaN
NaN				
1	0.525408	0.441512	0.453209	NaN
NaN				

	threshold
10	0.65
9	NaN
8	0.54
6	NaN
2	NaN
4	NaN
0	NaN

7	NaN
3	NaN
5	NaN
1	NaN

16. Final Model Selection

This section selects the final model for each prediction task using validation macro-averaged F1-score as the main criterion. Macro-averaged F1-score is used because the assignment evaluates model adequacy using macro F1 and because both tasks contain class imbalance. Accuracy is also reported, but it is not used as the main selection metric because it can hide weak performance on minority classes.

The selected models are:

- **Cancer Classification:** Improved CNN + Data Augmentation + Threshold Tuning
- **Cell-Type Classification:** Improved CNN

The cancer model is selected because it achieved a validation macro F1-score above the required 0.90 threshold. The cell-type model is selected because it achieved a validation macro F1-score above the required 0.60 threshold.

```
all_validation_results_df = pd.DataFrame(validation_results)

display(
    all_validation_results_df.sort_values(
        by=["task", "macro_f1"],
        ascending=[True, False]
    )
)
```

	task \
10	Cancer Classification
9	Cancer Classification
8	Cancer Classification
6	Cancer Classification
2	Cancer Classification
4	Cancer Classification
0	Cancer Classification
7	Cell-Type Classification
3	Cell-Type Classification
5	Cell-Type Classification
1	Cell-Type Classification

	model	accuracy
macro_f1 \		
10	Improved CNN + Data Augmentation + Threshold T...	0.910112
		0.907927

9	Improved CNN + Data Augmentation	0.896130
0.894648		
8	Improved CNN + Threshold Tuning	0.881898
0.878110		
6	Improved CNN	0.879650
0.876364		
2	SVM RBF	0.859925
0.856775		
4	Random Forest	0.836954
0.827703		
0	Logistic Regression	0.773034
0.770972		
7	Improved CNN	0.686754
0.637979		
3	SVM RBF	0.602606
0.505300		
5	Random Forest	0.574376
0.434521		
1	Logistic Regression	0.528773
0.432366		

	weighted_f1	macro_precision	macro_recall	best_epoch
best_val_macro_f1 \				
10	0.910091	0.908122	0.907736	NaN
NaN				
9	0.896554	0.892485	0.898959	40.0
0.894648				
8	0.881388	0.881918	0.875378	NaN
NaN				
6	0.879439	0.877790	0.875145	5.0
0.876364				
2	0.860015	0.856332	0.857250	NaN
NaN				
4	0.833794	0.846749	0.820706	NaN
NaN				
0	0.774287	0.770570	0.776634	NaN
NaN				
7	0.694149	0.633793	0.651460	11.0
0.637979				
3	0.587570	0.566720	0.550925	NaN
NaN				
5	0.515536	0.586220	0.516340	NaN
NaN				
1	0.525408	0.441512	0.453209	NaN
NaN				

	threshold
10	0.65
9	NaN

```

8      0.54
6      NaN
2      NaN
4      NaN
0      NaN
7      NaN
3      NaN
5      NaN
1      NaN

#select the best validation model for each task
best_validation_by_task = (
    all_validation_results_df
    .sort_values(by=["task", "macro_f1"], ascending=[True, False])
    .groupby("task")
    .head(1)
    .reset_index(drop=True)
)

display(best_validation_by_task)

      task \
0  Cancer Classification
1  Cell-Type Classification

      model  accuracy
macro_f1 \
0  Improved CNN + Data Augmentation + Threshold T...  0.910112
0.907927
1      Improved CNN  0.686754
0.637979

      weighted_f1  macro_precision  macro_recall  best_epoch
best_val_macro_f1 \
0      0.910091      0.908122      0.907736      NaN
NaN
1      0.694149      0.633793      0.651460      11.0
0.637979

      threshold
0      0.65
1      NaN

```

16.1 Final Model Justification

For the cancer classification task, the strongest model is the **Improved CNN + Data Augmentation + Threshold Tuning** model. The original improved CNN achieved a validation macro F1-score of **0.8764**. After data augmentation, the score increased to **0.8946**. After threshold tuning, the validation macro F1-score increased further to **0.9079**, exceeding the required threshold of **0.90**. This shows that the cancer model benefited from both stronger

generalisation through augmentation and a better probability decision threshold. For the cell-type classification task, the strongest model is the Improved CNN, which achieved a validation macro F1-score of 0.6380, exceeding the required threshold of 0.60. The classical models are still useful as baselines. LogisticRegression provides a simple linear comparison, SVM RBF provides a margin-based non-linear comparison, and RandomForestClassifier provides a tree-based ensemble comparison. However, the CNN models produced the strongest validation performance overall because they can preserve and learn from local spatial image patterns.

```
final_cancer_model = aug_cnn_cancer_original_setting
final_cancer_threshold = best_aug_cancer_threshold

final_celltype_model = cnn_celltype

print("Final cancer model: Improved CNN + Data Augmentation +
Threshold Tuning")
print("Final cancer threshold:", round(final_cancer_threshold, 2))

print("\nFinal cell-type model: Improved CNN")

Final cancer model: Improved CNN + Data Augmentation + Threshold
Tuning
Final cancer threshold: 0.65

Final cell-type model: Improved CNN
```

17. Final Test Set Evaluation

The final selected models are evaluated on the held-out test sets. The data was split by patientID, so the test set contains unseen patients rather than randomly selected images from the same patients. This makes the evaluation more realistic and reduces the risk of data leakage. The test set was not used during model training, validation, hyperparameter tuning, augmentation decisions, or threshold selection.

17.1 Cancer Test Evaluation

```
y_cancer_test_true, cancer_test_probs = predict_pytorch_probabilities(
    model=final_cancer_model,
    data_loader=cancer_test_loader,
    device=device
)

cancer_test_class_1_probs = cancer_test_probs[:, 1]

y_cancer_test_pred = (
    cancer_test_class_1_probs >= final_cancer_threshold
).astype(int)

cancer_test_metrics = evaluate_predictions(
    y_true=y_cancer_test_true,
    y_pred=y_cancer_test_pred,
```

```
    model_name="Final Cancer Model: Improved CNN + Data Augmentation +  
Threshold Tuning",  
    task_name="Cancer Classification Test Set",  
    class_names=cancer_class_names,  
    show_report=True,  
    show_confusion_matrix=True  
)
```

```
cancer_test_metrics["threshold"] = final_cancer_threshold
```

```
=====
```

```
Task: Cancer Classification Test Set  
Model: Final Cancer Model: Improved CNN + Data Augmentation +  
Threshold Tuning
```

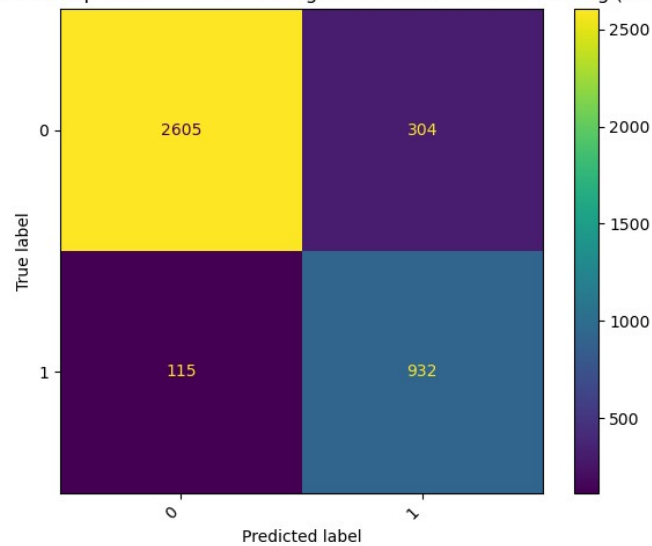
```
=====
```

```
Accuracy:      0.8941  
Macro F1:      0.8710  
Weighted F1:   0.8967  
Macro Precision: 0.8559  
Macro Recall:  0.8928
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.96	0.90	0.93	2909
1	0.75	0.89	0.82	1047
accuracy			0.89	3956
macro avg	0.86	0.89	0.87	3956
weighted avg	0.90	0.89	0.90	3956

Confusion Matrix: Final Cancer Model: Improved CNN + Data Augmentation + Threshold Tuning (Cancer Classification Test Set)



17.2 Cell-Type Test Evaluation

```
y_celltype_test_true, y_celltype_test_pred = predict_pytorch_model(
    model=final_celltype_model,
    data_loader=celltype_test_loader,
    device=device
)
```

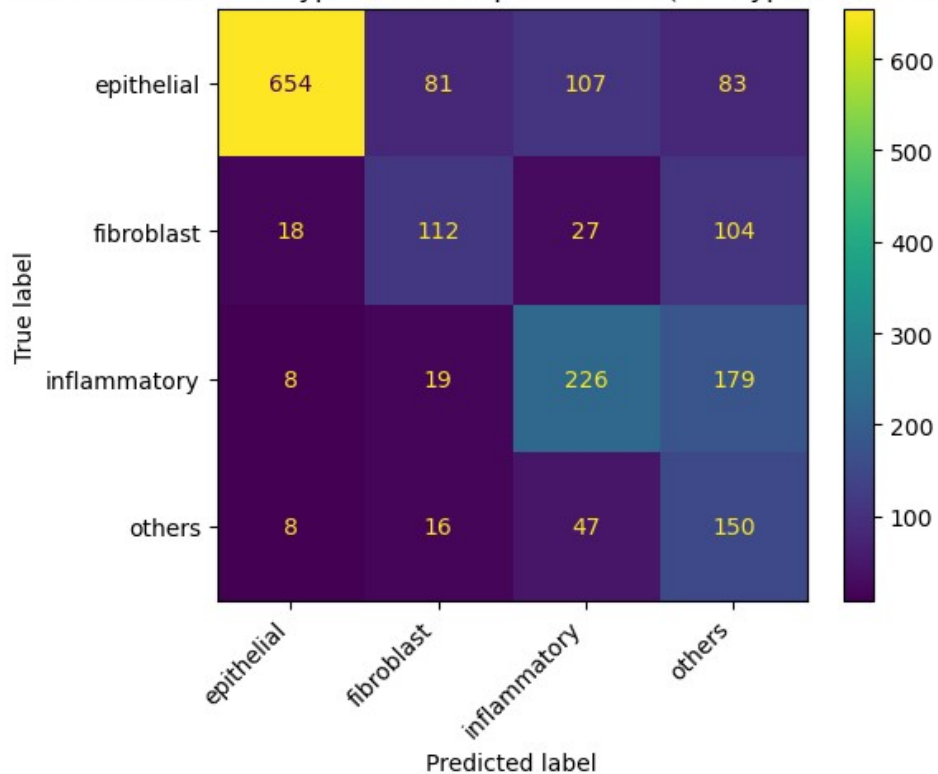
```
celltype_test_metrics = evaluate_predictions(
    y_true=y_celltype_test_true,
    y_pred=y_celltype_test_pred,
    model_name="Final Cell-Type Model: Improved CNN",
    task_name="Cell-Type Classification Test Set",
    class_names=celltype_class_names,
    show_report=True,
    show_confusion_matrix=True
)
```

```
=====
=====
Task: Cell-Type Classification Test Set
Model: Final Cell-Type Model: Improved CNN
=====
=====
Accuracy:      0.6210
Macro F1:      0.5537
Weighted F1:   0.6484
Macro Precision: 0.5719
Macro Recall:  0.5845

Classification Report:
      precision    recall  f1-score   support
```

epithelial	0.95	0.71	0.81	925
fibroblast	0.49	0.43	0.46	261
inflammatory	0.56	0.52	0.54	432
others	0.29	0.68	0.41	221
accuracy			0.62	1839
macro avg	0.57	0.58	0.55	1839
weighted avg	0.71	0.62	0.65	1839

Confusion Matrix: Final Cell-Type Model: Improved CNN (Cell-Type Classification Test Set)



17.3 Test Results Summary

```
test_results_df = pd.DataFrame([
    {
        "task": "Cancer Classification",
        "model": "Improved CNN + Data Augmentation + Threshold
Tuning",
        "accuracy": cancer_test_metrics["accuracy"],
        "macro_f1": cancer_test_metrics["macro_f1"],
        "weighted_f1": cancer_test_metrics["weighted_f1"],
        "macro_precision": cancer_test_metrics["macro_precision"],
        "macro_recall": cancer_test_metrics["macro_recall"],
        "threshold": final_cancer_threshold
    },
])
```

```

{
    "task": "Cell-Type Classification",
    "model": "Improved CNN",
    "accuracy": celltype_test_metrics["accuracy"],
    "macro_f1": celltype_test_metrics["macro_f1"],
    "weighted_f1": celltype_test_metrics["weighted_f1"],
    "macro_precision": celltype_test_metrics["macro_precision"],
    "macro_recall": celltype_test_metrics["macro_recall"],
    "threshold": None
}
})

display(test_results_df)

```

	task \	model	accuracy	
0	Cancer Classification			
1	Cell-Type Classification			
macro_f1 \				
0	Improved CNN + Data Augmentation + Threshold T...		0.894085	
			0.871017	
1		Improved CNN	0.620990	
			0.553695	
	weighted_f1	macro_precision	macro_recall	threshold
0	0.896691	0.855883	0.892830	0.65
1	0.648366	0.571947	0.584507	NaN

17.4 Test Evaluation Interpretation

The test results provide the final estimate of model performance on unseen patients. For cancer classification, the final model should be judged using macro F1-score, precision, recall, and the confusion matrix. False negatives are especially important because they represent cancerous images that were predicted as non-cancerous. False positives are also relevant because they may lead to unnecessary follow-up, but false negatives are generally more serious in a cancer-screening context. For cell-type classification, the confusion matrix is useful for identifying which cell types are most commonly confused. This task is more difficult because it contains four visually similar classes, and some categories have fewer examples than others. If the test macro F1-scores are lower than the validation macro F1-scores, this suggests that patient-level variation remains challenging. If the test results are similar to the validation results, this supports the generalisation ability of the selected models.

18. Discussion, Limitations, Ethics, and Conclusion

This final section summarises the modelling outcomes, discusses the adequacy of the models, and identifies limitations and ethical considerations.

18.1 Overall Findings

This notebook developed and evaluated machine learning models for two image classification tasks:

1. `isCancerous` classification
2. `cellTypeName` classification

The final selected cancer classification model was an `Improved CNN + Data Augmentation + Threshold Tuning` model. This model was selected because it achieved the strongest validation macro F1-score and exceeded the required target of `0.90`. The final selected cell-type classification model was the `Improved CNN`. This model was selected because it exceeded the required validation macro F1-score target of `0.60` and outperformed the classical machine learning baselines. Overall, the results show that CNN-based models are better suited to this image classification problem than models trained only on flattened image vectors. This is because CNNs can learn local spatial patterns from the image patches.

18.2 Do More Complex Models Always Perform Better?

The results show that more complex models do not automatically perform better in every setting. For the cancer classification task, the CNN became the strongest model only after additional improvement through data augmentation and threshold tuning. This suggests that model complexity alone was not enough; the training strategy and decision rule also mattered. For the cell-type classification task, the CNN outperformed the classical baselines, but the task remained challenging. The class-level results and confusion matrix show that visually similar classes were still difficult to separate. This matches the theoretical expectation that complex models can learn richer patterns, but they can also overfit or generalise poorly if they are not regularised or evaluated carefully. In this notebook, validation macro F1-score, confusion matrices, learning curves, and patient-level splitting were used to assess whether each model was generalising rather than only fitting the training data.

18.3 Model Adequacy in the Real World

The final models show promising performance, especially after using CNN-based feature learning, data augmentation, and threshold tuning. However, real-world medical use requires caution. For cancer classification, false negatives are especially important because they represent cancerous images predicted as non-cancerous. A model with strong macro F1-score can still make clinically serious mistakes. Therefore, this model should not be used as a standalone diagnostic system. A more realistic use case would be as a decision-support tool that helps prioritise images for review by medical experts. The model could support screening or triage, but final decisions should remain with qualified clinicians. For cell-type classification, the task is more difficult because the classes are visually similar. The model may provide useful automated support, but errors between visually overlapping cell categories should be expected.

18.4 Limitations

Several limitations should be considered:

- The images are very small (`27 × 27` pixels), which limits the amount of visual detail available to the models.

- The dataset is imbalanced, so macro F1-score is more informative than accuracy.
- The patient-level split is stricter than an image-level split. This is appropriate, but it makes the task harder because the models must generalise to unseen patients.
- Some classes appear visually similar, especially in the cell-type task.
- Threshold tuning improves validation performance, but the selected threshold must still be checked on the unseen test set.
- The models are trained on a limited dataset and may not generalise to images collected from different hospitals, scanners, staining procedures, or patient populations.

18.5 Ethical and Professional Considerations

Because this assignment involves medical image classification, ethical and professional responsibilities are important. The main risk is over-reliance on model predictions. Even if a model achieves strong macro F1-score, it can still make mistakes. In cancer classification, false negatives are especially concerning because missed cancerous cases may delay diagnosis. There may also be dataset bias. If the training data does not represent the full diversity of real patient populations or imaging conditions, the model may perform worse in real clinical settings. For these reasons, the model should be interpreted as a machine learning experiment and potential decision-support tool, not as an autonomous diagnostic system. Further validation on larger and more diverse datasets would be required before real-world deployment.

18.6 Conclusion

This assignment developed a complete image classification workflow for colon cell images. The workflow included data loading, patient-level splitting, exploratory data analysis, unsupervised learning EDA, preprocessing, classical supervised learning baselines, CNN modelling, hyperparameter tuning, data augmentation, threshold tuning, final model selection, and test evaluation. The best cancer classification model was the **Improved CNN + Data Augmentation + Threshold Tuning** model. This model exceeded the required validation macro F1-score threshold of **0.90**. The best cell-type classification model was the **Improved CNN**, which exceeded the required validation macro F1-score threshold of **0.60**. The results suggest that CNNs are better suited to this image classification problem than classical models using flattened pixels. However, the limitations and ethical risks mean that these models should be viewed as decision-support tools rather than replacements for expert clinical judgement.